Radboud Repository
sharing science

Spiking neural networks on FPGA: A survey of methodologies and recent advancements

Karamimanesh, M.; Abiri, E.; Shahsavari, M.; Hassanli, K.; Schaik, F.A. van; Eshraghian, J.

2025, Article / Letter to editor (Neural Networks, 186, (2025), article 107256)

Doi link to publisher: https://doi.org/10.1016/j.neunet.2025.107256

Version of the following full text: Publisher's version

Published under the terms of article 25fa of the Dutch copyright act. Please follow this link for the

Terms of Use: https://repository.ubn.ru.nl/page/termsofuse

Downloaded from: https://hdl.handle.net/2066/319311

Download date: 2026-06-08

Note:

To cite this publication please use the final published version (if applicable).

Neural Networks 186 (2025) 107256

Contents lists available at ScienceDirect

Neural Networks

journal homepage: www.elsevier.com/locate/neunet

Review

Spiking neural networks on FPGA: A survey of methodologies and recent advancements

Mehrzad Karamimanesh [a], Ebrahim Abiri [a,*], Mahyar Shahsavari [b], Kourosh Hassanli [a], André van Schaik [c], Jason Eshraghian [d]

[a] Department of Electrical Engineering, Shiraz University of Technology, Shiraz, Iran
[b] AI Department, Donders Institute for Brain Cognition and Behaviour, Radboud University, Nijmegen, The Netherlands
[c] The MARCS Institute, International Centre for Neuromorphic Systems, Western Sydney University, Australia
[d] Department of Electrical Engineering, University of California Santa Cruz, Santa Cruz, CA, USA

ARTICLE INFO

Keywords:
Spiking neural network
Neuromorphic computing
Field-programmable gate array
Brain-inspired computing
Accelerator

ABSTRACT

The mimicry of the biological brain's structure in information processing enables spiking neural networks (SNNs) to exhibit significantly reduced power consumption compared to conventional systems. Consequently, these networks have garnered heightened attention and spurred extensive research endeavors in recent years, proposing various structures to achieve low power consumption, high speed, and improved recognition ability. However, researchers are still in the early stages of developing more efficient neural networks that more closely resemble the biological brain. This development and research require suitable hardware for execution with appropriate capabilities, and field-programmable gate array (FPGA) serves as a highly qualified candidate compared to existing hardware such as central processing unit (CPU) and graphics processing unit (GPU). FPGA, with parallel processing capabilities similar to the brain, lower latency and power consumption, and higher throughput, is highly eligible hardware for assisting in the development of spiking neural networks. In this review, an attempt has been made to facilitate researchers' path to further develop this field by collecting and examining recent works and the challenges that hinder the implementation of these networks on FPGA.

Contents

* Corresponding author.
E-mail addresses: m.karamimanesh@sutech.ac.ir (M. Karamimanesh), abiri@sutech.ac.ir (E. Abiri), mahyar.shahsavari@ru.nl (M. Shahsavari), hassanli@sutech.ac.ir (K. Hassanli), a.vanschaik@westernsydney.edu.au (A. van Schaik), jeshragh@ucsc.edu (J. Eshraghian).

https://doi.org/10.1016/j.neunet.2025.107256
Received 28 June 2024; Received in revised form 28 December 2024; Accepted 5 February 2025
Available online 14 February 2025
0893-6080/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

1. Introduction

With the rapidly expanding use of artificial intelligence (AI) across all domains, optimizing models for energy consumption and latency only becomes of increasing importance. While a variety of approaches exist to make deep learning more efficient, the approach taken in neuromorphic computing is to draw upon principles that underpin the brain's remarkable efficiency. Spiking neural networks (SNNs), which mimic the brain by encoding and processing data as electrical pulses, or spikes, have the potential to drive down the energy cost of neural networks by reducing data bottlenecks through sparse activations and event-triggered computation, along with reduced bit-widths which simplify computation (Eshraghian et al., 2023; Liang, Zhang, & Chen, 2021; Maass, 1997). In event-triggered computation, processing occurs only when spikes (events) are generated, minimizing unnecessary operations and saving energy (Shahsavari, Thomas, van Gerven, Brown, & Luk, 2023). Compared to conventional artificial neural networks (ANNs), SNNs have the potential to be far more efficient when processing sequential data (Azghadi et al., 2020; Lemaire et al., 2020; Ottati et al., 2023). Such techniques to drive down the power consumption of embedded AI is critical due to the growing prevalence of portable, resource-constrained devices (He et al., 2021).

Spiking neurons transmit data via spike emission. Spikes that are distributed in space can be processed in parallel, and those that are triggered across different times can be serialized. In the absence of spiking activity, less memory accesses are required, though computing spike-based workloads still benefit from parallel processing. CPUs have limited scope for parallelization, while GPUs have a large number of cores which can be adapted for highly parallelizable workloads. This fact has been exploited by the deep learning community in batch processing tensors between layers. A tensor is a multidimensional array used to represent data, often in the form of matrices or higher-order generalizations, which allows efficient handling of large-scale computations in deep learning. The sparsity of spike-based activations results in large sub-blocks of tensors remaining inactive, and GPUs are not effectively optimized for sparsity, nor can they be reprogrammed in a way to address this (Liu, Chen, Ye, & Gui, 2022). Additionally, the von Neumann architecture prevalent in CPU and GPU processing enables general purpose compute at the cost of high power wastage in transferring data from memory to the processing unit (Dabbagh, Karamimanesh, Hassanli, & Abiri, 2025; He et al., 2021). Integrating memory and processor can accelerate computation while reducing power consumption at the cost of handling general workloads, but this is a trade-off that is typically necessary as the demands of deep learning processing continue to grow (Eshraghian, Wang and Lu, 2022; Yang, Gao et al., 2021).

Integrating memory and processor has been thought of as following the 'neuromorphic' computing paradigm because the brain does not isolate memory and computation. Rather, the computational units (neurons) and memory storage units (synapses) are intertwined and tightly integrated. Fig. 1 depicts an overview of the von Neumann architecture (Fig. 1(a)) and a general concept of neuromorphic processing (Fig. 1(b)) (An, Ehsan, Zhou, & Yi, 2017). Given that the brain is the most efficient computing system in the world, harnessing its processes can potentially lead us to reducing power consumption for cognitive computing. This requires custom hardware that is architected similarly to the brain, and a common theme of neuromorphic engineering is to do exactly this in the form of application-specific integrated circuits (ASICs) or FPGAs (Shahsavari, Devienne, & Boulet, 2019; Shahsavari, Faisal Nadeem, Arash Ostadzadeh, Devienne, & Boulet, 2015).

FPGAs are extremely useful for rapid testing and iteration of custom SNN acceleration due to its high flexibility, programmability, parallel processing, and low power consumption when compared to CPUs and GPUs (Gupta, Vyas, & Trivedi, 2020; Wang, Hao, Wang, & Deng, 2023). A general comparison between different platforms is provided in Fig. 1(c) (Capra et al., 2020). They can also be leveraged to test exotic and emerging spiking architectures, and to provide more accurate estimates of the energy efficiency that can be gained using co-design approaches. However, FPGA implementations bring on challenges that are not as straightforward as software programming, such as Python. Therefore, in this review article, we provide an overview of the implementation of SNNs on FPGAs, spanning from the execution methods, current focuses, challenges, specific applications, and research gaps that need to be addressed in the future. This review can enable deep learning practitioners who need lightweight hardware solutions to better understand the silicon their models will be processed on, and it can likewise assist FPGA engineers in understanding the future models that they will be tasked with accelerating.

1.1. A meta-review on SNNs

We provide a meta-review on the variety of review articles that complement this review and are summarized in Table 1. Moving from the bottom of the computing stack, Yang et al. (2020) provide a review on SNNs with an emphasis on emerging devices. Basu et al. (2022) offer a comprehensive review of key spike-based custom integrated circuits (ICs), and Bouvier et al. (2019) moves up to hardware architectures for SNNs. Javanshir et al. (2022) and Nguyen et al. (2021) examine recent advancements on the implementation of SNNs on various hardware platforms.

Moving to a software-centric perspective, Pietrzak et al. (2023) focuses on learning algorithms and their complexity for hardware execution, while Huynh et al. (2022) explores software frameworks that encompass platform-based design and hardware–software co-design. Finally, Frenkel et al. (2023) investigate the convergence of top-down and bottom-up design methodologies for custom ASIC implementations of SNNs with a deep dive into co-design, and Roy et al. (2019) provide a high-level overview across spike-based computation. With respect to FPGA-focused works, Pham et al. (2021) provides a high-level account of a variety of work that deploys SNNs on FPGA, and provides an application-level overview. Yang and Chen (2024) present

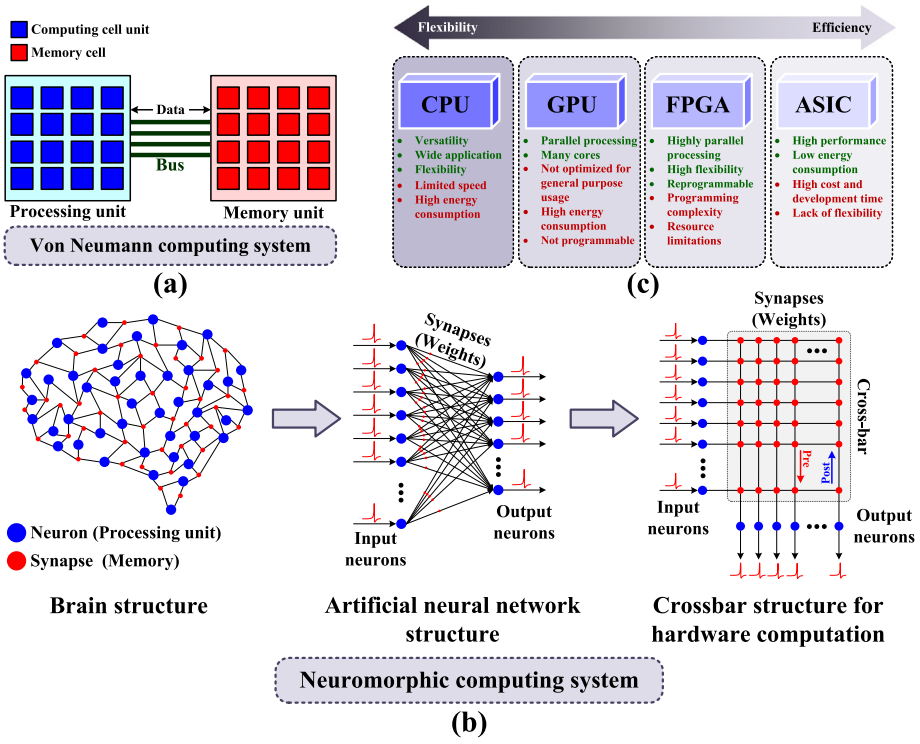

Fig. 1. (a) Overview schematic of the von Neumann architecture and how the memory and processing units interact; (b) An illustration of the overall concept of neuromorphic processing inspired by the brain structure and the interaction of neurons and synapses within it, and how it can be implemented in hardware as a unified processing and memory unit by a crossbar structure; (c) Comparison of hardware platforms.

one of the recent comprehensive resources that extensively examine various aspects of neuromorphic intelligence systems. While this book primarily emphasizes neuroscientific and theoretical aspects of learning and explores different architectures, it does not extensively cover FPGA implementations. We aim to provide a more recent account of learning and inference on FPGAs with a deeper dive and synthesis of what makes these designs efficient and effective.

Given the rapid pace at which algorithms are advancing and the long design cycles associated with ASIC design, there is a significant need for FPGA-based exploration for software-hardware co-design. This review aims to go in great depth of implementation details, execution of the various parts of an SNN, the tradeoffs between different architectures, and FPGA-specific techniques in order to assist SNN developers to better benefit from the fast design iterations that are possible with FPGAs.

1.2. Methodology and organization

In this review, our primary focus is on well-established techniques and methods that have been used, reused and broadly accepted amongst FPGA implementations across SNNs. We first present fundamental concepts to understand SNNs, and then move onto their implementation on FPGAs by analyzing its execution in various prior works across its various components.

The FPGA implementation of SNNs in this paper has been categorized into four main aspects, which should be considered in most designs. These include: (1) encoding of different types of input data into spikes; (2) the implementation of various spiking neurons; (3) learning rules, which involves how synaptic weights are updated; (4) network architecture and system design, which encompasses the design and execution of the SNN on FPGA and is dependent on the first three categories. Additionally, this section discusses sparsity, one of the key features of SNNs, and how to exploit it on FPGAs. After covering these four points, we dive deeper into technical methods that have implemented specific applications, and based on these articles, the existing gaps in the field are identified with a perspective on where future work could be directed.

The organization of this paper is as follows:

Basic concepts of SNN are first examined in Section 2. Their implementation on FPGA is examined in Section 3, and the specific applications in real world application and how they have been applied will be discussed in Section 4. The gaps and challenges that face the SNN/FPGA communities will be described in Section 5, with consideration on domains that should have more attention in future works, before concluding the review in Section 6.

2. Preliminaries

In this section, we delve into SNN-centric topics: namely, synaptic plasticity, various neuron models, principles of learning, and data encoding.

2.1. Synaptic plasticity

Synapses provide pathways through which neurons transmit information between each other, and play a critical role in the information processing of the nervous system in biological organisms (Abbott & Regehr, 2004). Numerous empirical studies have established that synapses show activity-dependent properties during neural activity, and it is broadly accepted that synaptic plasticity, the ability of synapses to strengthen or weaken over time in response to increases or decreases in their activity, forms the foundation of learning and memory in biological systems (Sjostrom, Rancz, Roth, & Hausser, 2008). The strength of synaptic connections between neurons is known to be affected by the timing of pre-synaptic and post-synaptic activity across various timescales (Bi & Poo, 1998). Synaptic plasticity can be broadly categorized into two types: (1) *long-term plasticity*, which includes changes in synapses that last for hours or more, and (2) *short-term plasticity*, which indicates some activity-dependent properties on a time scale of

Table 1
Recent published reviews with the main focus on the SNN hardware implementation.

Ref.	Main focus	Year
Yang and Chen (2024)	Basic to advanced review of neuromorphic learning algorithms and architectures	2024
Frenkel, Bol, and Indiveri (2023)	Various approaches to neuromorphic hardware design	2023
Pietrzak, Szczesny, Huderek, and Przyborowski (2023)	Learning algorithms and computational complexity	2023
Huynh et al. (2022)	Software frameworks for hardware-software co-design	2022
Basu, Deng, Frenkel, and Zhang (2022)	SNN integrated circuits	2022
Chen, Xiong and Liu (2022)	SNN chips based on stochastic computing	2022
Javanshir, Nguyen, Mahmud, and Kouzani (2022)	Various types of SNN hardware implementation	2022
Nguyen, Tran, and Iacopi (2021)	Various types of SNN hardware implementation	2021
Pham et al. (2021)	FPGA implementation of SNN	2021
Yang et al. (2020)	Neuromorphic hardware implementation with emerging devices	2020
Bouvier et al. (2019)	Emerging SNN hardware architectures	2019
Roy, Jaiswal, and Panda (2019)	Neuromorphic computing algorithm–hardware co-design	2019

tens of milliseconds to minutes (Lynch, Dunwiddie, & Gribkoff, 1977; Yang et al., 2020).

Long-term plasticity can be categorized into long-term potentiation (LTP) and long-term depression (LTD). LTP is a persistent enhancement in signal transmission between two neurons, caused by stimulating synapses with a highly potent stimulus. In contrast, LTD exhibits the opposite effect of LTP where connectivity is weakened, or 'depressed' (Bear & Malenka, 1994; Yang et al., 2020).

Short-term plasticity is crucial in the computations of neural systems. Much like long-term plasticity, short-term plasticity can be divided into two types based on changes in synaptic strength: short-term potentiation (STP) and short-term depression (STD).

Spike time-dependent plasticity (STDP) posits that pre-synaptic and post-synaptic activity will determine how the strength of synaptic connections are varied via LTP and LTD. STDP is often thought to play a vital role in performing time-dependent computational tasks in the nervous system, and at the very least contributes to higher-level, time-dependent learning. The next subsections will dive deeper into models of STDP, and while it is amongst the most commonly referenced mode of plasticity, there are other types of plasticity necessary for synaptic computation and neural signal processing. These include synaptic redistribution, spike-rate-dependent plasticity (SRDP), associative learning, non-associative learning, and synaptic scaling (Abbott & Nelson, 2000; Brenowitz & Regehr, 2005; Citri & Malenka, 2008; Yang et al., 2020). It is worth noting that understanding the operational mechanisms of biological nervous systems necessitates recognizing synaptic plasticity. STDP is one of the most commonly referenced forms of plasticity for imitating and executing biological nervous systems. To better understand STDP, it is important to explain its origins in Hebb's law.

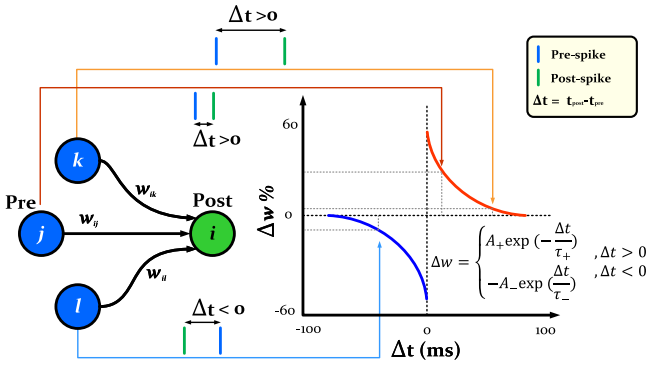

Fig. 2. The functioning of the STDP rule by adjusting synaptic weights to either strengthen or weaken them, depending on the timing between pre- and post-synaptic spikes, where w is synaptic weight, A_+ and A_- are two constants that represent learning rates, τ is time constant, and t_{pre} and t_{post} indicate when the pre-synaptic spike and post-synaptic spike happen, respectively.

2.1.1. Hebb's rule

Over the past five decades (and more), numerous scientific experiments have been conducted to study the mechanisms that underpin synaptic plasticity. Most of this research is based on Hebb's rule, which suggests that the connection between a pair of neurons, dubbed neurons A and B, should be modified in the following way (Hebb, 2005):

"*When an axon of cell A is near enough to excite cell B and repeatedly or persistently takes part in firing it, some growth process or metabolic change takes place in one or both cells such that A's efficiency, as one of the cells firing B, is increased*".

Shatz (1992) made summarized this more succinctly in her 1992 study of the visual system: "*cells that fire together wire together*". To put it simply, Hebb's law explains that when two neurons are physically close to each other and the pre-synaptic neuron causes the post-synaptic neuron to fire, their connection strengthens. On the other hand, if the post-synaptic neuron fires before the pre-synaptic neuron, their connection weakens. This latter case indicates that the two neurons lack causality, and the post-synaptic neuron was likely stimulated by another neuron instead of the intended pre-synaptic neuron. This principle has inspired researchers to attempt to replicate the brain's mechanisms by using it as a model and designing hardware that functions similarly. Several models have been proposed to achieve this goal, and one of the most prevalent is STDP.

2.1.2. Spike-timing-dependent plasticity (STDP)

STDP gained significant traction as it experimentally verified Hebb's rule, and relies completely on the temporal relationship between pre- and post-synaptic neurons, as illustrated in Fig. 2 (Shouval, Wang, & Wittenberg, 2010). In this way, STDP can be thought of as a specific implementation of Hebb's more general rule. STDP shows that if the pre-synaptic neuron fires before the post-synaptic neuron with a small time difference, their connection is strengthened, which is the same as LTP activity. As the time difference increases, the change in connection strength exponentially drops off. This supports and adds quantitative value to Hebb's law, which states that a small time difference between the neurons implies a small distance between them and that the pre-synaptic neuron was likely to be responsible for stimulating the postsynaptic neuron.

Conversely, if the post-synaptic neuron fires before the pre-synaptic neuron, its connection strength should decrease, leading to LTD. The extent of the decrease in connection strength is greater when the time difference is smaller. The equation for computing the time difference and determining the change magnitude in connection strength is presented in Fig. 2 (Bi & Poo, 1998). In the context of neural networks, this change in connection strength corresponds to the model weights.

2.2. Spiking neuron models

Plasticity between neurons is dependent on how neurons activate and talk to each other, and so the neuron models used will have a determinative effect on how a network evolves over time. A large number of neuron models have been developed, some of which strive for biological accuracy, while others a tailored for computational simplicity. Using the appropriate model depends on the desired level of accuracy or computational efficiency. In the following subsections, the most commonly used models are discussed.

2.2.1. Hodgkin–Huxley model

The Hodgkin and Huxley (1952) (HH) model is amongst the most common models that retains the rich dynamical behaviors of ion-channels, and is derived from experiments on large squid neurons. In the HH system of equations, two types of Na^+ and K^+ ion channels are included in the generation of neuron output current and action potential:

$$I_{ion}(t) = G_{Na}m^3h(V_m - V_{Na}) + G_k n^4(V_m - V_K) + G_L(V_m - V_L), \tag{1}$$

$$C_m \frac{dV_m(t)}{dt} = I_{ion}(t) + I_{syn}(t), \tag{2}$$

where C_m is membrane capacitance, V_m is membrane potential, I_{syn} is synaptic input current, G_K and G_{Na} represent conductance of K^+ and Na^+ channels respectively, V_K and V_{Na} are the reversal potentials of K^+ and Na^+ ions, V_L represents leak reversal potential, and G_L is leak conductance. Gating parameters n control the K^+ channel, while m and h control the Na^+ channel. These parameters are obtained by the following equations:

$$\begin{aligned}
\frac{dm}{dt} &= \alpha_m(V_m)(1-m) - \beta_m(V_m)m,\\
\frac{dn}{dt} &= \alpha_n(V_m)(1-n) - \beta_n(V_m)n,\\
\frac{dh}{dt} &= \alpha_h(V_m)(1-h) - \beta_h(V_m)h,
\end{aligned} \tag{3}$$

where the constants are parameterized by experimental data:

$$\begin{aligned}
\alpha_m(V_m) &= \frac{0.1(25-V_m)}{\exp((25-V_m)/10)-1},\\
\beta_m(V_m) &= 4\exp(-V_m/18),\\
\alpha_h(V_m) &= 0.07\exp(-V_m/20),\\
\beta_h(V_m) &= \frac{1}{\exp((30-V_m)/10)+1},\\
\alpha_n(V_m) &= \frac{0.01(10-V_m)}{\exp((10-V_m)/10)-1},\\
\beta_n(V_m) &= 0.125\exp(-V_m/80).
\end{aligned} \tag{4}$$

By solving the above system of equations, this model can emulate the behavior of the membrane potential during spike generation. Although this model is biologically accurate, it is computationally heavy and consumes a lot of resources to implement large networks (Yamazaki, Vo-Ho, Bulsara, & Le, 2022), especially for FPGA implementation where resources are limited. Additionally, according to the claim by Paugam-Moisy and Bohte (2012), the HH model requires approximately 1200 floating-point operations per second (FLOPS) for simulation in each millisecond.

2.2.2. Izhikevich model

Inspired by the HH method with the goal of simplifying it, Izhikevich (2003) proposed a model with fewer computations and satisfactory accuracy. This model reduces many parameters of the HH neuron model to a 2-D system of ordinary differential equations provided below:

$$\begin{aligned}
\frac{dv(t)}{dt} &= 0.04v^2 + 5v + 140 - u + I(t),\\
\frac{du(t)}{dt} &= a(bv - u),
\end{aligned} \tag{5}$$

with a discrete, after-spike reset condition:

$$\text{if } v \geq 30 \text{ then } \begin{cases} v \leftarrow c \\ u \leftarrow u + d, \end{cases} \tag{6}$$

where v is the membrane potential, u is the activation of K^+ and the inactivation of Na^+ channels, which are responsible for the recovery of the neuron membrane potential, and I is synaptic input current which is integrated at the neuron. Once the membrane potential reaches +30, the recovery variable and the membrane voltage are reset according to Eq. (6). The parameter a determines the recovery speed and is typically set to 0.02. The parameter b typically assumes a value of 0.2, representing the sensitivity of the recovery variable u to subthreshold oscillations of the membrane potential. The value of the membrane potential v and the recovery variable u after a spike is determined by the parameters c and d, which are typically set to -65 and 2, respectively. One of the benefits of the Izhikevich neuron is that by changing the values of a, b, c, and d, it is possible to model and simulate various dynamic behaviors of the neuron. Due to the simplification of calculations and appropriate biological accuracy, this model is suitable for constructing large-scale spiking networks and only needs 13 FLOPS per 1 ms of simulation (Paugam-Moisy & Bohte, 2012).

2.2.3. Leaky integrate and fire (LIF)

Simplifying the spiking neuron model down to its bare essentials (i.e., a single-dimensional differential equation) yields the integrate-and-fire (IF) neuron. The principle behind this model is that the neuron integrates the voltage from the incoming spikes, and once the membrane potential reaches a certain threshold level, a spike is generated, following which the neuron returns to its resting state (Gerstner, Kistler, Naud, & Paninski, 2014). This is a relatively more common model in digital circuits due to its simplicity (Nitzsche, Pachideh, Luhn, & Becker, 2021).

The leaky integrate-and-fire (LIF) model slightly extends the IF-inspired model where a leak term is included in the neuron membrane potential. The behavior of this model is described by the following equation:

$$\begin{aligned}
&C_m \frac{dV_m}{dt} = -G_L(V_m - V_L) + I_{syn}(t),\\
&\text{if } V_m \geq V_{th}, V_m \leftarrow V_{peak} \text{ then } V_m \leftarrow V_{reset},
\end{aligned} \tag{7}$$

where V_{th} is the threshold voltage of membrane, V_{peak} is the action potential and V_{rest} is the resetting state of membrane. When the membrane voltage crosses the threshold value, the neuron emits a spike (V_{peak}) and is set to the value of V_{reset} by a refractory period of τ_{ref}, during which the firing rate of the neuron is suppressed. When the synapse input current is constant ($I_{syn} = I$) and the reset voltage is zero ($V_{reset} = 0$), the membrane voltage is given by:

$$V_m(t) = R_m I(1 - \exp(-\frac{t}{\tau_m})), \tag{8}$$

where R_m indicates the resistance of the membrane ($M\Omega$) and $\tau_m = R_m C_m$ represents the time constant. Therefore, the steady-state firing rate according to the first spike time when $V_m(t) = V_{th}$, can be derived as:

$$f = (\tau_{ref} + \underbrace{\tau_m \ln \frac{R_m I}{R_m I - V_{th}}}_{\text{first spike time } (t^{(1)})})^{-1}. \tag{9}$$

Given that Eq. (9) provides non-linearity and is differentiable, it is possible to train a deep network and provides a reasonable approximation for spiking neurons (Hunsberger & Eliasmith, 2016). The LIF model is widely used due to its low computational cost, high simulation speed, and ability to capture the most essential temporal dynamics of a biological neuron, and only requires 5 FLOPS for simulation at millisecond precision.

Table 2
Comparison of neuronal models presented in the section.

Model	Computational complexity	Biological accuracy	Biological plausibility	Dynamics type	#Fixed parameters	#Dynamic variables	Memory requirements	Ease of implementation	FPGA Suitability	Ref.
HH	High	Very high	Very high	Nonlinear	7	11	High	Complex	Low	Hodgkin and Huxley (1952)
Izhikevich	Low	Medium-High	High	Nonlinear	4	2	Low	Simple	High	Izhikevich (2003)
LIF	Very low	Low	Low	Simple Nonlinear (Exponential Decay)	6	1	Very Low	Very simple	Very high	Gerstner et al. (2014)
AdEx	Medium	High	High	Nonlinear (Exponential)	10	2	Medium	Moderate	Medium	Brette and Gerstner (2005)

2.2.4. Adaptive exponential integrate and fire (AdEx) model

In 2005, Brette and Gerstner (2005) proposed a two-dimensional integrate-and-fire model called the adaptive exponential integrate-and-fire (AdEx) model, and is based on a combination of the exponential integrate-and-fire model (EIF) (Fourcaud-Trocmé, Hansel, Van Vreeswijk, & Brunel, 2003) and adaptation equation. The AdEx builds on features of the EIF model and the 2-D state variable model from Izhikevich. The equation that describes the behavior of the AdEx neuron is as follows:

$$C_m \frac{dV_m}{dt} = -G_L(V_m - V_L) + G_L \Delta_T \exp \frac{V_m - V_T}{\Delta_T} - \omega + I_{syn},$$
$$\tau_\omega \frac{d\omega}{dt} = a(V_m - V_L) - \omega, \tag{10}$$

with reset condition:

$$\text{if } V_m \geq 20 \text{ mV, then } \begin{cases} V_m \leftarrow V_{reset} \\ \omega \leftarrow \omega + b \end{cases} \tag{11}$$

where ω is the adaptation variable, Δ_T is the slope factor, V_T is threshold potential, a is a level of subthreshold adaptation, and b is spike-triggered adaptation. In the AdEx model, if Δ_T is set to 0 and the ω term is removed, it simplifies down to the LIF model. This model has less computational cost than the Izhikevich model, and requires 10 FLOPS.

Table 2 compares the neurons discussed in this section, evaluating them qualitatively and quantitatively to aid understanding. In this table, neurons are compared based on criteria such as computational complexity, biological accuracy, biological plausibility, the comparison of their equations in terms of dynamics, and the number of parameters. Dynamic parameters are those that change over time to simulate neuron behavior, such as the values of V_m and ω in the AdEx model. In contrast, fixed parameters are those that can be adjusted to control the neuron's output, like the values of a, b, c, and d in the Izhikevich model. Additionally, the compatibility of these models with hardware implementation, which is the primary focus of this paper, is also presented in the table, although further explanations are provided in the following sections.

2.3. Learning rules in SNNs

Translating these principles thus far into a graph theoretic view, neurons can be thought of as nodes while their interconnecting synapses are weighted edges. This view complements artificial neural networks (ANNs) in deep learning, albeit introducing dynamical behaviors into the system. Optimizing an ANN requires a learning rule, which is most often in the form of gradient descent implemented via error backpropagation. Given the limited biological plausibility of backpropagation, the neuroscience community offer alternative theories of learning. They can be broadly classed as supervised and unsupervised, though note that the boundary between the two is exceedingly blurred in the realm of self-supervised (Shwartz Ziv & LeCun, 2024) and weak-supervision (Gunasekaran, Eshraghian, Zhu, & Kuncic, 0000; Yang, Truong, Eshraghian, Nikpour, & Kavehei, 2022; Zhou, 2018).

2.3.1. Unsupervised learning

STDP can be interpreted as an unsupervised learning rule as it, by default, does not rely on a supervisory or error signal. It is also a form of local learning, in that the calculation of synaptic plasticity is only dependent on values that operate on the synapse itself. With local learning, there is an opportunity to move memory and computation units closer together, where the memory unit plays the role of synaptic storage weights, and computations are performed in the neuron. Removing long-range data communication to implement synaptic plasticity is well-suited for energy-efficient on-chip learning. One of the first to present a successful demonstration of unsupervised learning for SNNs was (Diehl & Cook, 2015), which achieved acceptable and comparable accuracy compared to deep learning methods on the MNIST dataset (LeCun, Bottou, Bengio, & Haffner, 1998). However, local learning with SNNs faces a scaling challenge, as an increase of layers drives the probability of spiking in deeper layers to reduce, which is referred to as "*vanishing forward-spike propagation*" (Kheradpisheh, Ganjtabesh, Thorpe, & Masquelier, 2018). In general, STDP has the characteristic of positive reinforcement where strong connections are typically further strengthened. This reinforcement boosts the likelihood of a neuron firing when it encounters the same pattern again. Moreover, STDP, as a type of unsupervised learning algorithm, can enable lifelong learning similar to the biological brain's system, and has been used to fine-tune pre-trained models (Bianchi, Muñoz-Martin, & Ielmini, 2020; Hu et al., 2021).

Given the positive reinforcement of STDP, there must exist homeostatic mechanisms in the brain to prevent an avalanche of neural activity. In general, homeostasis mechanisms refer to processes that maintain balance and stability within biological systems. In neural systems, they preserve an equilibrium state, ensuring that their activity remains stable over time without experiencing extreme fluctuations (Davies, Gait, Rowley, & Di Nuovo, 2024). One such approach to balance network activity accounts for lateral inhibitory mechanisms; that is, the excitation of one neuron suppresses other neurons, and is intended to model the competitive mechanisms of the brain. In this approach, the firing of one neuron inhibits the activity of other neurons. Specifically, a neuron's success in each layer suppresses neighboring neurons in that layer, known as the "*winner-takes-all*" method (Masquelier, Guyonneau, & Thorpe, 2009). There are various ways to implement winner-takes-all, with two of the more common approaches illustrated in Fig. 3. In Fig. 3(a), the weights between excitatory and inhibitory layers are assumed to be fixed values, such that red lines represent positive weights and blue lines represent negative weights. This means that the victory of a neuron in the excitatory layer activates the corresponding neuron in the inhibitory layer and suppresses all its neighboring neurons with a relatively large negative

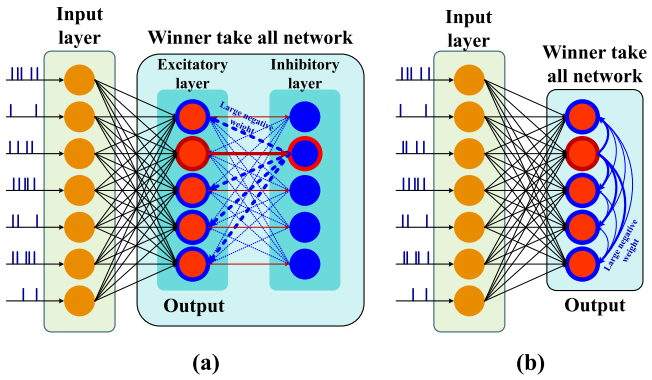

Fig. 3. A demonstration of two types of winner-take-all network mechanisms. (a) Inhibitory layer feedback. (b) Inhibitory recurrence.

weight. In Fig. 3(b), the winning neuron itself sends this negative weight to neighboring neurons through recurrent connections.

The classic STDP model, which is also known as pair-based STDP (PSTDP) based on the dependence on pairs of spike times, was thought to be limited in biological Interpretability (Lammie, Hamilton, van Schaik, & Azghadi, 2018). Therefore, in 2006, Pfister and Gerstner (2006) introduced the triplet-based STDP model (T-STDP), which is capable of producing biological behavior with higher accuracy. This model considers sets of three spikes, two pre- and one post-neuron, or one pre- and two post-neuron. T-STDP is a mechanism that alters the weight of the synapse by taking into account the timing of three consecutive spikes, as described by the following equations:

$$\Delta w = \begin{cases} \Delta w^{+} = e^{\frac{-\Delta t_1}{\tau^{+}}}(A_2^{+} + A_3^{+} e^{\frac{-\Delta t_2}{\tau_y}}) \\ \Delta w^{-} = -e^{\frac{\Delta t_1}{\tau^{-}}}(A_2^{-} + A_3^{-} e^{\frac{-\Delta t_3}{\tau_x}}), \end{cases} \tag{12}$$

where Δw^{+} and Δw^{-} are representative of potentiation and depression, respectively. A_2^{+}, A_3^{+}, A_2^{-} and A_3^{-} are the potentiation and depression strength parameters, and τ^{+}, τ^{-}, τ_x and τ_y are time constants which control the STDP curve window width. Various Δt_i are shown in Fig. 4(a). It has been indicated that the behavior inherent to the spike rate-based Bienenstock–Cooper–Munro (BCM) synaptic plasticity law emerges from the T-STDP rule (Azghadi, Al-Sarawi, Abbott, & Iannella, 2013; Lammie et al., 2018). The BCM theory describes a synaptic plasticity mechanism where the threshold for strengthening or weakening synapses is dynamic and adjusts based on the neuron's activity. This enables synapses to exhibit both long-term potentiation and depression depending on the spike rate, making it an essential model for understanding activity-dependent synaptic changes. It should be noted that a variety of other STDP-based methods such as self-organizing map-STDP (SOM-STDP) (Rumbell, Denham, & Wennekers, 2013), reward-modulated-STDP (R-STDP), reward-modulated self-organizing map-STDP (R-SOM-STDP) (Wang, He et al., 2022), and BCM-STDP (BSTDP) (Liu et al., 2018) have also been proposed, which build upon STDP.

Brader, Senn, and Fusi (2007) presented an alternative learning rule where changes in synaptic strength are determined by the internal state of the post-synaptic neuron during a pre-synaptic spike, rather than relying on timing differences. This method, known as spike-driven synaptic plasticity (SDSP), essentially increases the synaptic strength by one unit when the internal state of the post-synaptic neuron (V_m) is higher than a certain threshold (V_{th}) at the time of pre-synaptic spikes. Conversely, it decreases the synaptic strength by one step when the internal state is below a specific threshold. This method can reduce computational load and hardware complexity by avoiding the computation of precise timing and can enable learning that is both temporal and spatial. It aligns with the asynchronous and event-driven nature of spiking neural networks, making it suitable for efficient hardware implementations and real-time processing. Frenkel, Legat, and Bol (2019) and Frenkel, Lefebvre, Legat, and Bol (2018) have employed this approach for hardware implementation on 65 nm and 28 nm CMOS, and its operation is illustrated in Fig. 4(b).

2.3.2. Supervised learning

In supervised learning, the network is trained with a global signal which can come from labels in data or a loss/error signal. It is the most performant method in the era of deep learning, and so in 2002, *SpikeProp* was developed for SNNs' supervised learning similar to traditional error back-propagation (Bohte, Kok, & La Poutre, 2002). This algorithm showed how SNNs can learn nonlinear tasks such as XOR classification. After introducing SpikeProp, more advanced versions have emerged to generalize single-spike learning, such as *Multi-SpikeProp* (Ghosh-Dastidar & Adeli, 2009).

In 2018, Zenke and Ganguli (2018) introduced *SuperSpike* which trains multilayer SNNs of leaky integrate-and-fire (LIF) neurons on spatiotemporal spike pattern transformation tasks. Using a surrogate gradient approach to overcome the non-differentiable nature of discrete spikes, a nonlinear voltage-based three-factor learning rule is derived. This approach to adapting gradient descent and unrolling the computational graph of LIF neurons has become the most common for training SNNs as it is the most performant, and draws directly from all the advances of SNNs (Eshraghian et al., 2023; Shrestha & Orchard, 2018; Wu, Deng, Li and Shi, 2018). It has enabled SNNs to achieve large-scale tasks competitively with modern deep learning (Bal & Sengupta, 2024; Zhu, Zhao, Li, & Eshraghian, 2023).

Before surrogate gradient descent-based techniques were popularized, the de facto method for training SNNs was to instead train an ANN and convert it to an SNN using weight rescaling and normalization methods to transform continuous non-linear values from the ANN into leakage time constants, refractory periods, membrane thresholds and other parameters required for SNNs (Sengupta, Ye, Wang, Liu, & Roy, 2019). In conversion-based approaches, backpropagation through time (BPTT) is not required which has increasing memory complexity with more sequence steps. This requires approximations, as the ANN necessarily sets an upper-bound on performance before being converted into an approximation as an SNN. SNNs converted from ANNs have classically had very high inference times, leading to increased delay and decreased energy efficiency, and are only performant on time-static datasets which is often thought of as defeating the purpose of SNNs (Ottati et al., 2023; Roy et al., 2019). A detailed overview of deep learning-derived techniques to training SNNs is provided in Ref. Eshraghian et al. (2023).

Deep learning-derived training methods face a slew of challenges with respect to bio-plausibility, and deviate away from one of the initial purposes of developing SNNs: efficient learning. The remote supervised method (*ReSuMe*) was introduced for SNNs to produce desired input–output properties, where STDP and anti-STDP mechanisms (Rumsey & Abbott, 2004) work together with global guidance coming from a target transfer function or desired output (Kasinski & Ponulak, 2005). *Tempotron* is a widely recognized algorithm that neural systems use to effectively decode information concealed in spatiotemporal spike patterns (Gütig & Sompolinsky, 2006). ReSuMe and the Tempotron use supervised variants of STDP to perform classification, enabling STDP to achieve more than it would otherwise be able to with only local plasticity.

Alternative methods that more explicitly define global or reward-modulated forms of STDP have also been proposed in order to make unsupervised methods more performant using a weak supervisory signal. This led to the R-STDP rule (Shahsavari, Thomas, Brown, & Luk, 2021). In this method, incorporating the benefits of reinforcement learning through an external reward signal by modeling dopamine enables STDP to more train a network more effectively (Bing et al., 2019, 2018; He et al., 2021). In R-STDP, the goal of training is to achieve maximum reward. Therefore, when success is achieved, the

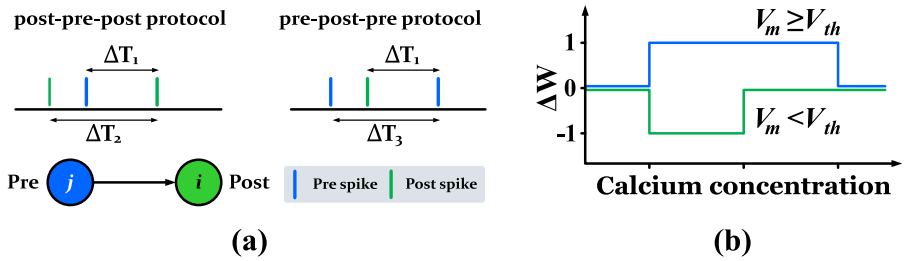

Fig. 4. (a) Determination of the time differences for calculating TSTDP; (b) SDSP learning rule (Brader et al., 2007).

released dopamine reinforces the learning process (Quintana, Perez-Pena, & Galindo, 2022). In R-STDP, the synaptic weights (W) are updated by two metrics: an eligibility trace (E_t) and a reward signal (R). At first, an eligibility trace of the synapse is employed to collect modifications made by STDP. The eligibility trace of a synapse is described as:

$$\dot{E}_t = -\frac{E_t}{\tau_E} + \mathrm{STDP}(t_{post} - t_{pre})\delta(t - t_{pre/post}), \tag{13}$$

where $t_{pre/post}$ is the spike time of a pre or post synaptic neuron. The eligibility trace decays exponentially at a rate defined by the time constant τ_E, which is set to 1 s in its original formulation (Izhikevich, 2007), $\mathrm{STDP}(t_{post} - t_{pre})$ is the synaptic weight change based on the original STDP model, and δ is the Dirac-delta function. By applying the eligibility trace with the reward signal, the weight of the synapses is updated:

$$\dot{W}_{ij} = R(t).E_{t_{ij}}(t) \tag{14}$$

The amount of the reward signal is determined by the amount of dopamine. Each time a reward signal is generated, its value as the concentration of dopamine is shared across all synapses and decays by the time constant τ_R. The amount of reward is calculated based on:

$$\dot{R} = -\frac{R}{\tau_R} + \mathrm{DA}(t), \tag{15}$$

where $\mathrm{DA}(t)$ models the source of dopamine. τ_R is set to 0.2 s (Izhikevich, 2007).

Additional learning rules that are also independent from deep learning for spiking neurons with information temporal coding were introduced by *Chronotrons*; one that provides a high memory capacity (E-learning), and one that has higher biological plausibility (I-learning) (Florian, 2012). This is not an exhaustive overview of learning rules in SNNs, but rather, narrowed to focus on a subset of those that have been generally implemented in hardware. For a more rigorous treatment on brain-inspired learning mechanisms, we refer the reader to Ref. Schmidgall et al. (2024).

2.4. Data encoding

Most SNN hardware require spike-based inputs, and as such, sensory data must often be encoded into spikes as well. The way data is encoded can directly affect network performance, speed, accuracy, and efficiency, but given that most input data is recorded as 'continuous' variables, the conversion into spikes often represents some form of lossy compression. It is generally wiser to process data that is natively captured in the form of spikes (or events), but outside of that, most encoding strategies in the deep learning with SNN domain is done via two strategies.

The first view considers coding based on the rate of neurons firing, which is called *rate coding* (Kim et al., 2022). Another view is that the information is coded in accurate timing between spikes (or the timing of an individual spike) emitted from a neuron which is called *temporal coding* (Ghosh-Dastidar & Adeli, 2009). In the rate coding method, neurons that are exposed to high stimulation generate higher frequency firing than neurons with lower intensity stimulation. The sliding window (SW) algorithm (Webb, Davies, & Lester, 2011) and Ben's Spiker algorithm (BSA) (Schrauwen & Van Campenhout, 2003) are two common algorithms to move between spiking and non-spiking domains using rate codes (Wang et al., 2023). To avoid lossy compression, long spike trains are usually required, leading to increased activity, power consumption, and latency. Therefore, Wang, Gu, Goh, Zhou and Luo (2022) proposed the radix-encoded method, which not only significantly reduced the length of spike trains but also improves accuracy of the models they work with, achieving a 25× speedup and a 1.7× improvement in accuracy compared to state-of-the-art models that work with rate encoded data.

The second dominant approach to encoding data is to use temporal codes, where information is encoded into the firing time, such that a neuron that is stimulated with a higher intensity fires earlier than a neuron with lower stimulation (Saha et al., 2013). This method is more energy effective than rate coding because it encodes information in fewer spikes. Step-forward (SF) encoding, moving-window (MW) encoding (Kasabov, 2014), pulse width modulated-based (PWMB) algorithm (Arriandiaga, Portillo, Espinosa-Ramos, & Kasabov, 2019), rank-order encoding (Van Rullen & Thorpe, 2001), GA-gamma (Sengupta, Scott, & Kasabov, 2015), wavelet encoding (Wang, Guo, & Adjouadi, 2016) and so on, are the most common temporal coding methods. Among the temporal coding methods, phase coding algorithms (Cattani, Einevoll, & Panzeri, 2015) benefit more from the temporal information of neurons and have been used more widely in recent years (Wang et al., 2023). We note that a variety of encoding strategies exist and this is far from an exhaustive treatment of encoding methods.

3. FPGA-based SNNs

Neuromorphic hardware can be divided into three categories: analog, digital and mixed-mode. While neuromorphic computing emerged to emulate the analog capabilities of neural processing, digital implementations of SNNs are easier to scale into large systems.

So far, several ASIC chips with fully custom digital designs have been proposed by researchers and companies to move away from von Neumann architecture towards neuromorphic systems. These chips are specifically designed for brain-inspired processing. Among the most common introduced architectures, several can be highlighted. SpiNNaker (Furber, Galluppi, Temple, & Plana, 2014), developed by the University of Manchester, is designed as a distributed von Neumann architecture that includes 18 ARM cores per chip, implemented in 130 nm technology. This chip is specifically designed for large-scale SNN simulations with high flexibility. The newer hardware of this system, SpiNNaker 2 (Gonzalez et al., 2024), is optimized with ARM Cortex M4F cores to simulate more neurons per chip. IBM TrueNorth (Akopyan et al., 2015) is a design based on a globally asynchronous locally synchronous (GALS) architecture, containing 1 million neurons and 256 million non-plastic synapses per chip, implemented with 5.4 billion transistors using 28 nm technology. The custom-designed Izhikevich neurons in this chip are able to simulate 11 different behaviors of neurons, and by combining three neurons, they can generate up to

20 different behaviors. Intel Loihi (Davies et al., 2018) is also a fully asynchronous design, with up to 180,000 neurons and between 114,000 to 1 million synapses. This chip, due to its on-chip learning and asynchronous design, can be highly efficient for real-time neural processing. Additionally, it uses the STDP model for neural learning and is manufactured with Intel's 14 nm FinFET technology. Although these hardware platforms represent a significant milestone in the development of neuromorphic systems, they face some challenges such as limited or non-programmability and restricted accessibility for many researchers.

FPGAs provide an easy access effective platform for rapidly implementing exploratory algorithms and models at low-cost and high programmability, and with effective optimizations, can outperform CPUs and GPUs on task-specific workloads (Mohaidat & Khalil, 2024). SNNs particularly benefit from FPGA implementations due to their parallel processing capabilities, distributed local memory, and they can be adapted to handle dynamic workloads with branching, which is where GPUs are no longer optimal (Javanshir et al., 2022). For instance, Ju, Fang, Yan, Xu, and Tang (2020) presented an FPGA-based SNN and demonstrated that the implemented network is 41 times faster than a CPU and 22 times more power-efficient than a GPU. Also, Guo, Yantır, Fouda, Eltawil, and Salama (2021) showed that their SNN implementation on FPGA is 180 times faster for training and 20 times faster for inference compared to a PyTorch-based simulation on CPU. On the other hand, Han, Li, Zheng, and Zhang (2020) demonstrated, using the Xilinx ZC706 evaluation board, compared to software simulation via PyTorch running on a Tesla P100 GPU, the FPGA processes 337.6 frames per second per watt, whereas the GPU processes only 42.2 frames. Besides, Fang, Shrestha, Zhao, Li, and Qiu (2019) stated that their FPGA implementation was 196 times more power-efficient and 10.1 times faster compared to GPU execution. Cheslet, Bernert, Beaubois, Yvert, and Lévi (2024) implemented a real-time task using SNNs on the AMD Kria KR260 board, stating that their execution on this board, which is part of the Zynq Ultrascale+ MPSoC, was about 28 times faster than on a personal computer (PC). These examples highlight the superiority of FPGA over other platforms, especially for executing SNNs, proving its selection for this purpose more reasonable.

Given the focus on efficiency, resource-constraints often necessitate many approximations. When implementing algorithms and models on an FPGA, there are often inevitable discrepancies between the software implementation and the hardware. This is because a direct implementation without simplifications, such as quantization or implementing transcendental functions as look-up tables, may not be feasible without a substantial amount of hardware resources (Eshraghian, Lammie, Azghadi and Lu, 2022; Venkatesh, Marinescu, & Eshraghian, 2024).

Over the past few years, there have been numerous attempts to create SNN implementations on FPGA. These attempts can be generally divided into data encoding methods, learning rules, neuron models and network architecture (Pham et al., 2021). In the following subsections, we will examine and review the work that has been carried out in each area in more detail.

3.1. Data encoding

Deep SNNs demand more memory, and there will invariably come a point where on-chip memory is insufficient and a designer will require off-chip memory which increases power consumption. Effective encoding methods of input data along with a good choice of architecture can significantly reduce network parameters and memory (Javanshir et al., 2022). As a result, a considerable amount of research effort has gone into optimizing data encoding strategies, which is in contrast to many software based SNN efforts where inputs are just treated as continuous current injections without encoding.

Poisson distributed spike coding (Cao, Chen, & Khosla, 2015; Diehl et al., 2015) is frequently used as an encoding technique on FPGA that encodes inputs based on the intensity of data. It can be mathematically represented as follows:

$$\text{if } rand() < cx, \text{then fire}, \tag{16}$$

where $rand()$ is a random number sampled from a uniform probability distribution between 0 and 1, c is the firing rate controlling factor, and x represents the normalized input. One approach to generating pseudo-random numbers is the use of a linear-feedback shift register (LFSR) (Linares-Barranco et al., 2007). Likewise, the Poisson encoding method can be produced based on the time interval between two spikes (inter-spike interval (ISI)) (Heeger et al., 2000), as per the following equation:

$$\mathcal{I} = \frac{-\ln(rand())}{\eta}, \tag{17}$$

where $\mathcal{I}$ is the time interval between the prior and current spike and η is the firing rate.

Temporal methods are potentially more biorealistic, and are more appropriate for dynamic input data. However, for static data such as images, rate coding methods can be more effective. For this reason, Carpegna, Savino, and Di Carlo (2022) implemented the Poisson method by considering a time interval Δt for all inputs, during which the inputs are encoded with a certain number of spikes. Then, Δt is divided into steps of dt, where each step includes a maximum of one spike. By multiplying the input value ($rate$) by dt, the average number of spikes can be calculated ($ASPS = total\ spikes/total\ steps = rate.dt$). The value of $ASPS$ is between 0 and 1, so by generating a random number and comparing it, the spike generation condition is checked for each step. This random number is also generated using the LFSR technique, similar to previous methods, with the difference that the generated number is considered the same for all inputs, leading to less hardware complexity and resources while having a minimal impact on accuracy.

In Poisson-based spike trains, the number of spikes generated in a given time window may not be as exact as expected due to randomness. This can result in the generation of an unpredictable number of spikes. Therefore, Xu, Tang, Xing, and Li (2017) proposed an encoding algorithm called fixed uniform spike train generation algorithm (FUSTGA) which converts the inputs into spike trains more precisely. Its performance is shown in Algorithm 1, where T is the desired time window and I is each member of the input vector normalized between 0 and 1. Based on this algorithm, the number of spikes (N_{fire}) and their firing time ($Fire_{times}$) are adjusted based on I and T. Ju et al. (2020) implemented three methods of Poisson-ISI, Poisson-rand and FUSTGA, on their FPGA-based network and compared accuracy. They obtained results of 84.15%, 98.82%, and 98.94%, respectively for MNIST dataset, and demonstrated that the FUSTGA method achieves better accuracy compared to the two other Poisson methods. Given that MNIST is regarded as a simple, toy dataset this result may rule out the usefulness of Poisson-ISI for static data.

Algorithm 1 Fixed Uniform Spike Trains Generation

for each member of the input vector **do**
  Initialize a spike sequence in which no spikes are fired;
  if member value > 0 **then**
    $N_{fire} = Int(T * I_{normalized})$;
    $M_{array} = \frac{1}{I_{normalized}} * ones(N_{fire}, 1)$;
    $Fire_{times} = Int(cumsum(M_{array}))$;
    Adjust fire times based on $Fire_{times}$.
  end if
end for

Zhang et al. (2020) employed a temporal approach of *population coding* to encode the input data. Population coding uses a group of

Table 3
Comparison of encoding methods on FPGA. Provided by Wang et al. (2023).

	SW	BSA	SF	PWMB
Real-time Output	✗	✓	✓	✗
Polarity	Unipolar	Unipolar	Bipolar	Unipolar
Precision	Low	Medium	High	High
Speed	High	High/Low	Low	Medium
Resource consumption	Low	High/Medium	Low	Low
Stability against noise	Medium	High	Low	Low

Gaussian curves with varying means but a constant variance to represent input data through a probability distribution model (Sun et al., 2024; Tang, Kumar, Yoo, & Michmizos, 2021; Zhang et al., 2020). Hence this model maps input information to spike times, and can be expressed as follows:

$$f(x_i) = A\exp(-\frac{(x_i - c_i)^2}{2\sigma}) \rightarrow \begin{cases} \sigma = \frac{1}{\gamma(m+1)} \\ c_i = \frac{i-1}{m-1} \end{cases} \tag{18}$$

where A is the maximum spike time, m is the number of Gaussian reception domain, and γ is a control parameter.

Based on the importance of choosing the input coding type on the performance and efficiency of the network, Wang et al. (2023) implemented four common algorithms on FPGA. They examined these algorithms in terms of accuracy, speed, computations, hardware resources, and noise susceptibility. They have investigated two algorithms, SW (Webb et al., 2011) and BSA (Schrauwen & Van Campenhout, 2003), for rate coding, and two algorithms, PWMB and SF (Kasabov et al., 2016), for temporal coding. These four algorithms are more compatible with FPGA and can be implemented more effectively on it and according to their claim, the complexity of two algorithms, GA-gamma (Sengupta et al., 2015) and wavelet encoding (Wang et al., 2016), is very high, making them less suitable for execution on FPGA. The SW algorithm is only used for reconstructing signals from pulse signals.

- **BSA:** The foundation of the BSA algorithm is a FIR filter. To generate spikes, it is necessary to calculate the sum of the difference between the original signal and its filtered value. When this difference becomes greater than the sum of the original signal at that time point, a spike is generated, and the original signal is reduced by the filter value.
- **SF:** The SF algorithm utilizes a moving threshold. When the signal exceeds the threshold value, a spike is generated, and when the value falls below the threshold, a negative spike is generated, and the threshold value is updated.
- **PWMB:** In the PWMB method, the original signal is compared with a reference value. Pulses with different widths are generated based on the magnitude of the original signal, and a spike is produced at the rising edge of the produced pulse.

Table 3 shows the comparison of these four methods implemented on FPGA that were compared by Wang et al. (2023).

Continuous, real-time signal processing of sensor data such as electroencephalogram (EEG), electrocardiogram (ECG) etc., is a common application use-case of SNNs (Azghadi et al., 2020; Xiaoxue et al., 2023; Yang, Eshraghian, Truong, Nikpour and Kavehei, 2023). Therefore, converting such signals into spikes for real-time processing is crucial (Kasabov, 2014). By accounting for FPGA resource constraints, Leone, Raffo, and Meloni (2023) proposed a new method to reduce energy and the number of operations. The spike detection process is performed in five stages from raw data: (1) Filter: through a second-order moving average difference (MAD), low-frequency components are removed from the input signal; (2) Spike emphasis: removes the negative values of the signal through the absolute value to improve the reliability of signal detection; (3) Threshold: a threshold is determined by the mean value of the signal within a time window of 0.82 ms, so this threshold value is updated every 0.82 ms; (4) Detector: the peak values that exceed the threshold are identified; (5) Spike binning: these identified peaks are transferred as grouped spikes in the output, within the same time intervals where they were detected. Its spike detection process is briefly shown in Fig. 5. Their approach was tested using electrophysiological recordings of two macaque monkeys (Brochier et al., 2018), and the MAD filter proposed by Zhang and Constandinou (2021) is employed as follows:

$$\mathrm{MAD}(n) = x(n) - \frac{1}{N}\sum_{i=1}^{N} x(n-i), \tag{19}$$

where x is the input continuous signal and N is the order of filter. By choosing N as a power of two, the filter is performed with only $N+1$ sums and a right shift, eliminating the need for a multiplier, which is considered $N = 2$ in this article. Chu et al. (2022) converted continuous ECG input data to spikes using a level-crossing analog-to-digital converter (LC-ADC) and injected it as input into the network. LC is a sampling method that generates a pulse when the signal crosses predefined threshold levels (Wang, Ye et al., 2020). In this article, the event of LC-ADC is expressed using a single bit, streamlining the ADC quantization procedure and simultaneously reaching native temporal encoding.

Another approach that can directly encode data on FPGA is to simply use IF and LIF neurons. These neurons are stimulated based on the input signal intensity and membrane potential change, producing spikes at different rates depending on the input magnitude. This method can be implemented using a single neuron or a group of neurons (Fang et al., 2020). Moreover, direct utilization of neurons has shown promising results in signal processing applications (Rathi & Roy, 2020; Wu et al., 2019). Spiking neurons are commonly regarded as delta-sigma ADCs.

3.2. Learning rules

As discussed in Section 2.3, a vast range of methods have been proposed for training SNNs. However, implementing these methods on hardware, including FPGA, requires more considerations. In this section, we will explore some of the recent executed training methods on FPGA and their employed techniques.

3.2.1. Unsupervised learning rules

One of the main challenges in implementing learning rules on FPGA such as STDP is its exponential term, multiplication and solving differential terms (Bahrami & Nazari, 2024; Wu et al., 2021). To solve these problems, several techniques such as implementation by look-up table (LUT) (Chen, Kumar, Sumbul, Knag and Krishnamurthy, 2018), triangular approximation (Cassidy, Georgiou, & Andreou, 2013), Coordinate ROtation DIgital Computer (CORDIC) (Volder, 1959), Piecewise Linear (PWL), Based-2 approximations (Gomar & Ahmadi, 2018) and numerical solvers have been proposed by researchers. Among these methods, Guo et al. (2021) have successfully implemented the triplet STDP model on an FPGA using two numerical computation methods: Euler and third-order Runge–Kutta (RK3). The implementation of the RK3 method consumes more hardware resources but significantly enhances speed in both training and inference times.

As the multiplication operation in hardware implementation consumes a significant amount of resources, Yang, Wang, Deng, Azghadi and Linares-Barranco (2021) simplified the STDP rule using Euler approximation and implemented it using the shift logic multiplier (SLM) block instead of multipliers to prevent excessive resource and power consumption. The STDP equation employed in this article for implementation on FPGA is as follows:

$$W_{ij}^{e}(n+1) = \Delta t.\begin{pmatrix} (W_{\max} - W_{ij}^{e}).A_{+}e^{(-\Delta/\tau_{+})} \\ (W_{\min} - W_{ij}^{e}).A_{-}e^{(+\Delta/\tau_{-})} \end{pmatrix} + W_{ij}^{e}(n) \tag{20}$$

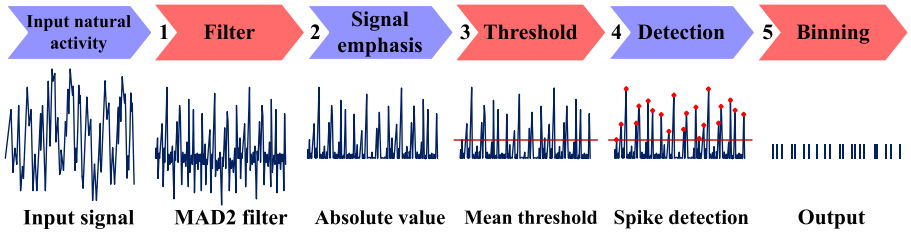

Fig. 5. Spike detection process chain proposed by Leone et al. (2023) for one input channel.
Source: Adopted from Leone et al. (2023).

where W^e is the excitatory weight and the values of $W_{\max}$, $W_{\min}$, A_+, A_- and $\tau_{(+,-)}$ are set to 1, 0, +1.2, −0.4 and 10 ms respectively. Exponential functions are implemented using LUTs in order to save hardware resources. Additionally, barrel shifters (Kotiyal, Thapliyal, & Ranganathan, 2011) are utilized alongside the SLM block to eliminate the need for multipliers. One of the popular hardware-friendly approximations for STDP (Masquelier & Thorpe, 2007) which is also widely used in FPGA (Kheradpisheh et al., 2018; Veeravalli, Fong, et al., 2022) is defined as follows:

$$\Delta w = \begin{cases} \alpha^+ w_{ij}(1 - w_{ij}) & \text{if } t_j - t_i \le 0 \\ \alpha^- w_{ij}(1 - w_{ij}) & \text{otherwise,} \end{cases} \tag{21}$$

where α^+ and α^- are learning rates.

One solution to cope with the computation of the timing of all spike pairs for STDP calculation, which can be a computationally expensive task and consume considerable hardware resources, especially on FPGA, is to use the trace-based STDP rule. It has been demonstrated to be much more efficient than the classical method while maintaining the same level of accuracy (Hu et al., 2021). In the trace-based STDP rule, the term *trace* refers to a temporal variable that records a neuron's recent spiking activity over time. It essentially tracks the decay of a neuron's past spikes and indicates the level of activation it has experienced recently. The trace value is updated each time a spike occurs and diminishes exponentially over time. Because the trace decays exponentially, it can be used as a replacement for the exponential function in the original STDP equation when calculating the weight updates (Gebhardt & Ororbia, 2024). For better comprehension, Fig. 6 illustrates how weight changes based on the trace left by recent pre- and post-synaptic spikes. When a spike occurs in the pre- or post-synaptic neurons, the synaptic weight is either strengthened or weakened by the amount of trace left by the other neuron based on its timing. In this article (Hu et al., 2021), if there is a spike event in the presynaptic neuron, the strength of the synapse is reduced by the value of the postsynaptic trace. On the other hand, if there is a spike event in the postsynaptic cell, the weight of the connection is increased by the value of the presynaptic trace. This means that in the first case, at the moment of the presynaptic spike, if a postsynaptic spike occurs before, it follows the conventional STDP concept, and the weight decreases. Conversely, at the moment of the postsynaptic spike, if a presynaptic spike occurs before, the weight should increase. Eq. (22) represents this concept, and it should be noted that $trace_{pre}$ and $trace_{post}$ decrease exponentially after each spike.

$$\begin{cases} w(t) = w(t-1) - trace_{post} & \text{, presynaptic spike occurs} \\ w(t) = w(t-1) + trace_{pre} & \text{, postsynaptic spike occurs} \end{cases} \tag{22}$$

In other words, the weights update process is conducted only in the occurrence of pre- or post-synaptic spikes, eliminating the need to store all spike times. This leads to improved efficiency and reduced hardware resources (He et al., 2021; Hu et al., 2021).

As mentioned, one of the suggested approaches to address the difficult implementation of exponential functions in FPGA is to utilize the CORDIC method (Bahrami & Nazari, 2024). CORDIC is an iterative algorithm for calculating various complicated functions such as exponential, hyperbolic, and trigonometric. It uses simple shift and add operations, making it simple and efficient to implement in FPGA (Meher,

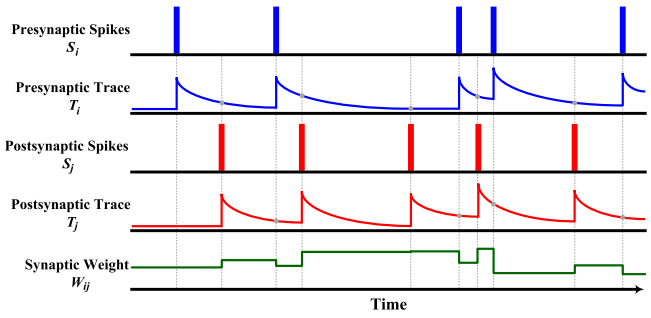

Fig. 6. Overall view of weight update mechanism through STDP trace-based method.

Valls, Juang, Sridharan, & Maharatna, 2009). Therefore, in bio-inspired approaches, exponential functions are utilized to represent the dynamics of neurons and synapses, where the CORDIC method can be highly beneficial for a more accurate and efficient implementation (Wu et al., 2021). Heidarpur, Ahmadi, Ahmadi, and Azghadi (2019) have simplified the traditional CORDIC algorithm and used it to implement STDP rule. More detail of this algorithm can be found in Refs. Heidarpur et al. (2019) and Heidarpour, Ahmadi, and Rashidzadeh (2016).

Wu et al. (2021) implemented a variety of different STDP implementations on FPGA in order to compare them. These methods include the simplified method proposed by Heidarpur et al. (2019), the conventional CORDIC method, and the fast-convergence CORDIC method proposed by Wang, Luo, and Zhou (2018), as well as the LUT-based method presented by Chen, Kumar et al. (2018). They demonstrated that CORDIC methods consume less power and energy than LUT-based, and while they have greater resource utilization, they do not need BRAM. Among the CORDIC methods, the fast-convergence technique consumes the least energy in nearly half the iterations compared to the other two methods.

Bahrami and Nazari (2024) introduced a method to improve the convergence speed of the CORDIC algorithm and tested it for the execution of exponential operators (Exp CORDIC), natural logarithm operators (Ln CORDIC), and arbitrary power functions (Pow CORDIC). Finally, they implemented the Spatial-Pow-STDP learning algorithm using Pow CORDIC. In comparison to traditional CORDIC, the proposed method achieves greater accuracy with fewer parameters and errors while completing the task in just 9 iterations.

To overcome the complexities of implementing nonlinear terms in synaptic plasticity and neuron models, Jokar, Abolfathi, and Ahmadi (2019) proposed a new method based on uniform piecewise linear segmentation (Soleimani, Ahmadi, & Bavandpour, 2012). They tested their proposed method on the calcium-based synaptic plasticity model (Graupner & Brunel, 2012) that can generate STDP curves.

Lammie et al. (2018) demonstrated the first implementation of the T-STDP algorithm on FPGA (Pfister & Gerstner, 2006). They replaced floating-point multipliers with low-resolution unsigned shift-and-add multipliers and substituted the synaptic coefficients (i.e. A and τ in Eq. (12)) with values of 2^N, where $N \in \mathbb{Z}$, mitigating the resource-hungry nature of the multiplication process. In the article

presented by Yang, Wang, Pang, Jin and Linares-Barranco, é (2024), the quadruplet STDP (Q-STDP) learning algorithm is introduced as an efficient method for FPGA implementation. In this implementation of the algorithm, linear differential equations replace complex exponential functions, enabling direct and cost-effective deployment on digital hardware. The synaptic weight update equation in this algorithm is defined as follows:

$$\Delta w = \begin{cases} -o_1(t)\left[A_1^- + A_2^- r_2(t-\epsilon) + A_3^- r_3(t-\epsilon) r_2(t-\epsilon)\right], & \text{if } t = t_{\text{pre}} \\ +r_1(t)\left[A_1^+ + A_2^+ o_2(t-\epsilon) + A_3^+ o_3(t-\epsilon) o_2(t-\epsilon)\right], & \text{if } t = t_{\text{post}} \end{cases} \tag{23}$$

where ϵ represents a time delay used to account for the temporal alignment between pre- and post-synaptic spikes, and r_1, r_2, r_3, o_1, o_2, and o_3 are variables representing the potentiation and depression potentials. These variables are modeled using linear differential equations, which are given as follows:

$$\frac{dr_1}{dt} = -\frac{r_1}{\tau_{x1}}, \quad \frac{do_1}{dt} = -\frac{o_1}{\tau_{y1}}, \quad \frac{dr_2}{dt} = -\frac{r_2}{\tau_{x2}}, \quad \frac{do_2}{dt} = -\frac{o_2}{\tau_{y2}},$$
$$\frac{dr_3}{dt} = -\frac{r_3}{\tau_{x3}}, \quad \frac{do_3}{dt} = -\frac{o_3}{\tau_{y3}} \tag{24}$$

where, $\tau_{x1}, \tau_{x2}, \tau_{x3}$ are the time constants for the potentiation potentials r_1, r_2, r_3, and $\tau_{y1}, \tau_{y2}, \tau_{y3}$ are the time constants for the depression potentials o_1, o_2, o_3. Experimental results in this paper demonstrate that Q-STDP achieves higher learning accuracy than T-STDP and traditional STDP algorithms, and has higher fault-tolerant.

3.2.2. Supervised learning rules

Supervised learning is integral to modern deep learning, even though it is generally far more resource intensive when compared to unsupervised methods. The work in Zhang et al. (2020) generalizes the Tempotron's supervised learning method as inspired by physical laws of inertia. The direction of the final weight update is determined by reserving a certain amount of the previous weight update value while calculating the current weight update value. The magnitude of weight update is calculated as $\Delta W_i = \lambda \sum_{t_i < t_{\max}} K(t_{\max} - t_i)$, where λ is the learning rate and determines the maximum synaptic update size, t_i represents the spike times of the ith afferent, t_{max} shows the time when the postsynaptic membrane potential reaches its maximum value, and K represents the normalized postsynaptic potential kernel function. More information about Tempotron learning is provided in Gütig and Sompolinsky (2006). The work in Ref. Zhang et al. (2020) adapts the original approach by implementing a weight update rule that adds momentum:

$$\Delta W_i = \lambda \sum_{t_i < t_{\max}} K(t_{\max} - t_i) + \Delta W_i^d, \tag{25}$$

where W_i^d is the term of current weight momentum, and calculated as $\Delta W_i^d = d_\lambda \lambda_m \Delta W_i^l$, where ΔW_i^l is the last weight update, d_λ is the attenuation factor of the momentum learning rate and causes the network to eventually converge to a stable value.

Bethi, Xu, Cohen, van Schaik, and Afshar (2022) proposed the optimized deep event-driven spiking neural network architecture (ODESA) method for supervised network training. In this method, networks are trained locally on event-based data. Based on this method, the activity of the next layer is used as the supervisory signal for the previous layer, and synaptic weights are adjusted solely based on the activity of each layer without the need for weights from other layers. In essence, two signals are utilized: Local Attention Signals (LAS) and Global Attention Signals (GAS). LAS is generated by hidden layers, while GAS is produced by the output layer. These signals are used to apply rewards and punishments to neurons during training. LAS is applied to the previous layer whenever a spike is generated in a layer, rewarding neurons that contribute to the spiking activity. Additionally, the GAS is generated by the output layer and applied to all layers in the network. This signal rewards active neurons when the correct label is assigned for input and punishes them if the correct label is not assigned or if there is no activity in the output layer. ODESA is inspired by Feature Extraction using Adaptive Selection Thresholds (FEAST) approach proposed by Afshar et al. (2020).

Mehrabi, Bethi, van Schaik, Wabnitz and Afshar (2023) successfully implemented the ODESA algorithm on FPGA. This work includes a multi-layer network where each layer has a separate training module, enabling online supervised training without the need for backpropagation. In this architecture, they implemented their network using local adaptive selection thresholds, applied Winner-Takes-All (WTA) constraints for each layer, and employed an updated weight adjustment rule that is more hardware-friendly. This paper demonstrates the good compatibility of the ODESA algorithm with hardware and its suitability for online training. Furthermore, Mehrabi, Bethi, van Schaik and Afshar (2023) have improved and optimized their hardware implementation by replacing costly dot products in the hardware neurons with low-cost shift registers. With this strategy, fewer resources are consumed at the FPGA level, enabling the implementation of more complex SNN networks.

Quintana et al. (2022) successfully implemented the learning of reward-modulated spike-timing-dependent plasticity (R-STDP) on FPGA. As observed in Fig. 7, calculating R-STDP requires computing the trace of STDP initially at each time step, which is done in parallel for each input synapse population. However, time multiplexing has been utilized to reduce hardware components for pre- and post-synaptic computations. When an input spike occurs, the value of $weight_{plus}$ increases by a constant value determined by the user. The e_{trace} register accumulates the $weight_{minus}$ value. When a postsynaptic spike occurs, the value of $weight_{minus}$ increases, and e_{trace} accumulates the value of $weight_{plus}$. In each time step, all registers are updated using the Euler method based on Eq. (22). After calculating the eligibility trace, its value is multiplied by the global dopamine level to determine the synaptic weight changes.

To achieve better accuracy without high resource utilization, He et al. (2021) combined trace-assisting, triplet-based STDP and R-STDP methods to train their network. This work uses the trace-assisting technique to prevent excessive storage of times and R-STDP to improve accuracy. The weights in the input layer are randomly initialized and fixed, and only the weights between the hidden and output layers are trained and updated based on the aforementioned methods.

As stated, presenting a supervised method to enhance adaptation to realistic cognitive applications, capable of adhering to biological optimality principles while also supporting online learning, is highly desirable and essential. Most gradient descent-based methods are typically executed offline, which can adversely affect the final accuracy of the system. Therefore, Tavanaei and Maida (2019) combined the STDP approach with backpropagation and proposed the BP-STDP method. This method is not only based on bio-inspired principles but also performs supervised backpropagation operations to achieve higher levels of accuracy. In this algorithm, the weights between the hidden layer and the output layer ($\Delta w_{h,o}$) are first updated, and then the weights between the input and hidden layers are ($\Delta w_{i,h}$) updated as follows:

$$\Delta w_{h,o}(t) = \alpha . \epsilon_o(t) . \sum_{t-4}^{t} S_h(t),$$
$$\Delta w_{i,h}(t) = \begin{cases} \alpha \sum_o (\epsilon_o(t) . w_{h,o}(t)) . \sum_{t-4}^{t} S_i(t), & S_h(t) = 1 \text{ in } [t-4, t] \\ 0, & \text{otherwise,} \end{cases} \tag{26}$$

where α is the learning rate, $S_h(t)$ represents whether in hidden neuron h spike is generated or not at timestamp $t \in \{0, 1\}$, $\epsilon_o(t)$ specifies the direction of changes, such that the output neuron is predetermined based on the target class. If the target neuron at output layer (o) has not spiked in the last four timestamps as the STDP window, the STDP rule is applied. If a non-target neuron spikes, the anti-STDP rule is applied,

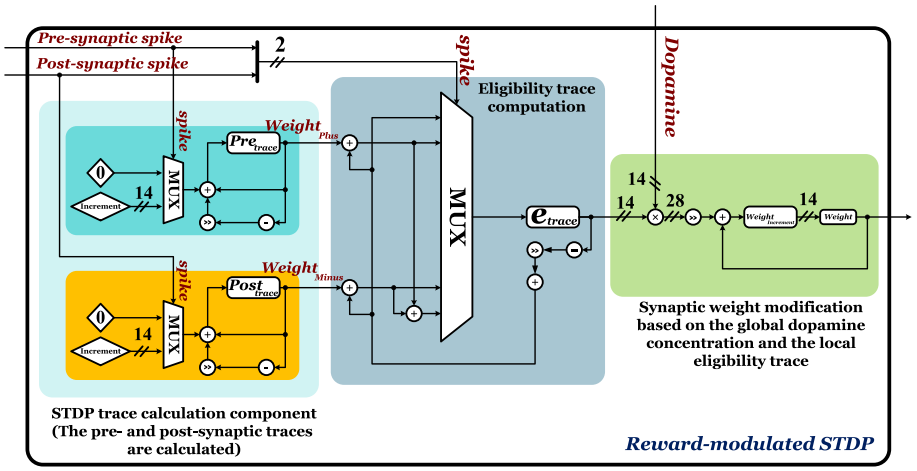

Fig. 7. Components of R-STDP trace calculation block.
Source: Adopted from Quintana et al. (2022).

whose values are as follows:

$$\epsilon_o(t) = \begin{cases} 1, & \text{if } S_o = 0 \text{ in } [t-4, t] \text{ for target neuron} \\ -1, & \text{if } S_o = 1 \text{ in } [t-4, t] \text{ for non-target neuron} \\ 0, & \text{otherwise.} \end{cases} \tag{27}$$

The error of the output neuron can be approximated by ϵ_o, which represents the difference between the target class neuron and the spiking neuron, and can be applied as a variant of gradient descent as shown in Eq. (26). Zhang et al. (2021) trained their network on an FPGA using the BP-STDP method. Instead of rate coding, they proposed the weighted neuron model, where the magnitude of each spike is proportional to the input. Therefore, the spike counts of neurons in the past four timesteps ($\sum_{t-4}^{t} S_i(t)$) in Eqs. (27) and (26) can be substituted by WS_i.

Prior studies indicate that online learning can be achieved through feedback signals that convey neural information about credit to calculate local error signals in hidden layers (Lillicrap, Cownden, Tweed, & Akerman, 2016). However, these models face challenges in mimicking the actual brain's performance, as they require separate feedback pathways to transmit neural information for determining local error signals. This approach, due to the need to pair each hidden layer neuron with a corresponding feedback error signal, is incompatible with the actual brain. Therefore, Yang, Wang, Pang, Azghadi and Linares-Barranco (2024) proposed the neuromorphic architecture for dendrite-oriented learning system (NADOL), designed with inspiration from the biological learning mechanisms in the human brain. This architecture integrates feedback and feedforward signals within distinct dendritic compartments, including apical and basal segments, thereby enhancing credit assignment and spike-based learning. By leveraging the unique properties of dendritic compartments, NADOL simulates the brain's ability to process and learn from sensory and feedback signals separately. This separation allows for accurate error computation without relying on inefficient methods like backpropagation, enhancing compatibility with biological processes. The semi-supervised NADOL training algorithm has been implemented on the FPGA-based BiCoSS platform (Yang, Wang, Hao et al., 2021), utilizing time-multiplexing, PWL, and Euler techniques for digital optimization.

To gain a better understanding of the methods discussed in this section, Table 4 offers a concise comparison of some rules which are adapted for FPGA implementation. In this table, the resources consumed in each study are gathered according to the employed rule and implementation technique. Additionally, the advantages and disadvantages of each study are outlined in a general manner. It is important to note that the main impact of the presented learning rules is on the overall network performance and the accuracy achieved, which are presented and compared in Section 3.4 (Table 9). The network accuracy achieved using these methods is also provided.

3.3. Neuron models

Neurons are the core building blocks of a spiking neural network, and their capabilities significantly impact a network's power, accuracy, and performance. As observed in Section 2.2, various types of spiking neurons have been introduced, differing in biological plausibility and computational complexity. When accounting for the resource constraints on lightweight FPGAs, the compute cost of neuron models becomes a critical consideration. A lot of prior research targets the reduction of the computational demands by introducing techniques for reducing hardware consumption while preserving an adequate level of biological accuracy. In this section, we highlight several neurons proposed in recent works and their implementation methods on FPGA will be analyzed.

The Izhikevich model expressed in Eq. (5) in the previous section shows the continuous form of the equation, and must be discretized if it is to be used in a digital system like FPGAs. Accordingly, one of the simplest methods for discretizing the model is using the Euler method, as Humaidi, Kadhim, Hasan, Ibraheem, and Azar (2020) have derives its approximate solution as follows:

$$\begin{aligned} v(n+1) &= v(n) + (0.04v^2(n) + 5v(n) + 140 - u(n) + I)T, \\ u(n+1) &= u(n) + a(bv(n) - u(n))T, \end{aligned} \tag{28}$$

where T is a fixed time interval for sampling. Also, the reset after the spike is performed as follows:

$$\text{if } v_{n+1} \geq 30\,\text{mV}, \text{ then } \begin{cases} v_{n+1} \leftarrow c \\ u_{n+1} \leftarrow u_{n+1} + d. \end{cases} \tag{29}$$

Since the computation of signed floating points in FPGA is expensive, fixed point representations are used for neuron values in the range of $[-1, 1]$. For this purpose, u and v should be scaled down by 100 to fit within the required range of the arithmetic system. Also, by considering the timestep as a power of 2, the integration operation can be simplified to a right shift. Therefore, in Humaidi et al. (2020), this value is considered $1/16$. In Quintana et al. (2022), this timestep is considered to be equal to $1/8$, and based on the down-scaling operator, the Izhikevich equation is converted as follows:

$$\begin{aligned} v &= (4v^2 + 4v + v + 1.4 - u + I)/8 \\ u &= a(bv - u)/8. \end{aligned} \tag{30}$$

Table 4
Comparison of some spiking neural network training rules adapted for FPGA implementation.

Ref.	Learning Rule	Supv. /Unsupv.	Technique Used	Resource Usage				Advantages	Disadvantages
				LUT	FF	DSP	BRAM		
Chen, Kumar et al. (2018)	STDP	Unsupv.	LUT-based	180	86	3	1	Simple hardware implementation	High memory usage for storing function values, leading to potential accuracy-memory trade-off
Hu et al. (2021)	STDP	Unsupv.	Trace-based	Moderate resource required				Faster and easier computation of weight change in STDP because there is no need to calculate *exp*	Needing a mechanism to have a trace of all neurons after each spike
Jokar et al. (2019)	STDP	Unsupv.	PWL	274	257	2	0	Simplified Implementation and reduced resource usage	Loss of accuracy due to approximation; for better accuracy, more pieces are required
Guo et al. (2021)	STDP	Unsupv.	RK3	High resource required				Suitable for online training in real-time applications due to higher speed than the Euler method	Higher resource and power costs; unsuitable for low-power inference applications
Wu et al. (2021) Heidarpur et al. (2019)	STDP	Unsupv.	CORDIC	244 611	123 163	3 3	0 0	More accurate calculation of *exp* compared to other methods; No need for BRAM	Accuracy has a direct relationship to the number of iterations; challenging for low-power designs
Wang et al. (2018)	STDP	Unsupv.	Fast-Convergence CORDIC	533	178	3	0	Faster than conventional CORDIC; No need for BRAM	More complex design increases difficulty in hardware implementation
Lammie et al. (2018)	TSTDP	Unsupv.	Triple-based STDP	18[a] 1943[b]	34 1498	0 0	0 0	Higher biological accuracy than basic STDP	Greater memory usage due to time storage, more complex than regular STDP
Quintana et al. (2022)	R-STDP	Supv.	Reward-modulated STDP	394	133	2	0.5	By combining the advantages of reinforcement learning with STDP, enables online learning for real-world scenarios	More complex calculations, increased resources and reduced learning speed
He et al. (2021)	TR-STDP	Supv.	Triplet-based reward-modulated STDP	Moderate resource required				Improving TSTDP performance using reward module; balances resource and accuracy trade-offs	Increased complexity over STDP and TSTDP
Zhang et al. (2020)	Tem-potron	Supv.	Momentum-based updates	3528	768	0	0	High-speed convergence due to momentum updates	Requires careful tuning for different network tasks and datasets; not suitable for deep networks
Mehrabi, Bethi, van Schaik, Wabnitz et al. (2023) and Afshar et al. (2020)	ODESA	Supv.	Event-based local training	Low resource required				Simple implementation and suitable for multi-layer local training	Low accuracy for complicated tasks
Tavanaei and Maida (2019)	BP-STDP	Supv.	Backpropagation with STDP	High resource required				High accuracy for deep networks	Slow convergence and increased resources and complexity

[a] Proposed digital optimization.
[b] Digital full resolution.

For a more biorealistic network, Ambroise, Levi, Bornat, and Saighi (2013) divided the input current of Izhikevich neurons into three parts: I_{stat}, representing the bias current; I_{exc}, applying the positive influence of excitatory synapses; and I_{inh}, applying the negative impact of inhibitory synapses. The Izhikevich equation, after incorporating changes and considering the approximate discrete solution $\frac{dv}{dt} = \frac{v[n+1]-v[n]}{\Delta t}$ is expressed as follows:

$$v[n+1] = \frac{1}{32}v[n]^2 + 4v[n] + 109.375 - u[n] + I_{stat}[n] + I_{exc}[n] + I_{inh}[n], \quad (31)$$

where in this equation, compared to the original Izhikevich equation (Eq. (5)), the value 1/32 substitutes the decimal 0.04 such that five right-shifts are used instead of multiplication. Additionally, the value of 5 is replaced with 4 to transform it into two left-shifts (Cassidy & Andreou, 2008). This implementation of the Izhikevich neuron is illustrated in Fig. 8. Updating v and u are done as a pipeline, utilizing only one multiplier in different steps. After calculating v and u in six stages, the final stage compares the value of v with the threshold to determine spike generation conditions and the subsequent values of v and u. Values are assumed to be 18-bit, with 10 bits for the integer and 8 bits for the fractional component. Therefore, 18×18 multipliers have been used in this implementation. I_{exc} and I_{inh}, derived from synaptic connections of pre- and post-neurons, take 1 ms to increase their

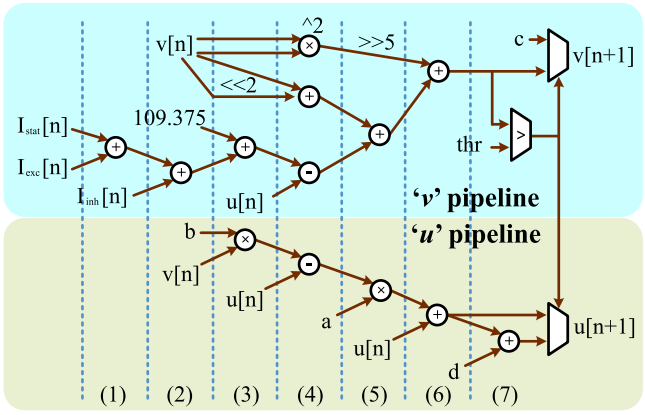

Fig. 8. Displaying the calculation of u and v values in a pipelined manner in seven steps.
Source: Adopted from Ambroise et al. (2013).

currents. However, they exhibit different leakage behaviors with time constants of 3 ms and 10 ms, respectively. For this reason, in Eq. (31), they are written separately and embedded in the equation.

So far, numerous articles have worked on the mathematical modeling of the Izhikevich neuron, such as Ghanbarpour, Naderi, Ghanbari, Haghiri, and Ahmadi (2023), Heidarpur, Ahmadi, and Ahmadi (2024), Lin, Lu, Pi, and Wang (2020) and Pi and Lin (2022). Among them, Leigh, Mirhassani, and Muscedere (2020) proposed a method called Hardware-Oriented Modified Izhikevich Neuron (HOMIN). In digital systems, multiplying numbers by 2^x, where $x \in \mathbb{Z}$, is achievable through simple shifting operations. Therefore, they approximated the constant numbers present in the Izhikevich model to the nearest 2^x values to reduce the need for multiplication units. Below the simplified equation of the HOMIN model is provided, discretized using the Euler method where $dt = 2^{-5} = 0.03125$:

$$\begin{aligned} v[n+1] &= v[n] + 2^{-5}(2^{-2}v^2 + 2^2 v[n] + v[n] + 14 - u[n] + I[n]), \\ u[n+1] &= u[n] + 2^{-11}(2-2v[n] - u[n]) \end{aligned} \tag{32}$$

A fixed point 16-bit signed model is used, consisting of one sign bit, six integer bits, and nine fractional bits.

To simplify the LIF neurons as much as possible, Carpegna et al. (2022) applied an offset to the internal parameters of the LIF equation, aiming to set v_{rest} to zero, which also leads to changes in v_{reset} and v_{thr} values. The equation is simplified as follows:

$$v[n] = v[n-1] - \frac{dt}{\tau}.v[n-1]. \tag{33}$$

In this equation, the implementation of $\frac{dt}{\tau}$ consumes a lot of hardware resources due to the multiplication. Therefore, its value is approximated to the nearest negative power of 2 to facilitate implementation through right shifts. After applying the offset and approximation, the parameter values are set to $v_{rest} = 0$, $v_{reset} = 5\text{mV}$, $v_{thr} = 13\text{mV}$, and $\frac{dt}{\tau} = 2^{-10}$.

To reduce power consumption, enhance speed, and utilize fewer and simpler hardware resources, Liang et al. (2021) introduced a single-spike IF neuron for their network. This neuron generates at most one spike in each recognition cycle. They highlighted the principle that the first generated spike carries the most information (Thorpe, Delorme, & Van Rullen, 2001). The classification is completed when a neuron in the output layer produces a spike. Thus, it leads to a more rapid classification. In the single-spike IF neuron, after the first spike, its value remains unchanged and no new values are accumulated until the classification period ends. The update equation for this neuron is as follows:

$$v_i(t) = \begin{cases} v_i(t-1) + \sum_{j=0}^{n-1} w_{ji}.s_i(t) & , v_i(t-1) < \theta \\ v_i(t-1) & , v_i(t-1) \geq \theta \end{cases} \tag{34}$$

Table 5
Comparison of different parameters of neurons implemented on Xilinx Zynq UltraScale+ ZCU102 FPGA. Provided by Nitzsche et al. (2021).

Neuron model	Bit width	Freq. (MHz.)	Power (W)	Resources		Biological plausibility
				LUT	FF	
LIF	9	542.9	0.016	19	2	3
	16	589.3	0.030	30	6	
Quadratic-IAF[a]	7	193.6	0.028	178	22	6
	16	172.2	0.075	520	63	
Izhikevich	10	127.4	0.055	534	74	21
	16	123.6	0.082	1051	170	
Hodgkin–Huxley	16	8.6	2.317	49,817	7035	19-22
	24	5.4	3.325	93,758	15,289	

[a] Quadratic Integrate-and-Fire (Ermentrout & Kopell, 1986).

$$s_i(t) = \begin{cases} 1 & , \text{first time } v_i(t) \geq \theta \\ 0 & , \text{else} \end{cases} \tag{35}$$

where $v_i(t)$ is the membrane voltage of the ith neuron at time t, $s_i(t)$ is the firing signal of the ith neuron, and θ is threshold. In this model, due to the generation of one spike in each cycle, the read rate from memory and the spike generation frequency decreases, which reduces power consumption. Furthermore, the accumulation process for the membrane potential only involves adders, eliminating the need for a multiplier.

Jokar et al. (2019) used the uniform piecewise linear segmentation method to tackle the nonlinear terms in two-dimensional models of Izikevich and FitzHugh–Nagumo (FHN) neurons (FitzHugh, 1961), as well as in a more complex three-dimensional model of the Hindmarsh–Rose (HR) neuron (Rose & Hindmarsh, 1989) to simplify its FPGA implementation.

Nitzsche et al. (2021) implemented various neuron models on the Xilinx Zynq UltraScale+ ZCU102 FPGA development board and conducted a comparative analysis. They used fixed-point representation for values and determined each model's optimum bit width and decimal point position based on the error level. They did not use DSPs such that the models could be fairly compared and performed all operations using logic cells. Additionally, in implementing each model, they leveraged powers of 2 as much as possible to eliminate the need for multipliers. Such a comparison is of great importance given that different DSPs optimize for different operations. Table 5 provides each neuron's bit width, achievable frequency, power consumption, and resource utilization in the same FPGA model. Also, biological plausibility has been noted for each model to provide a better comparison.

For an extremely efficient LIF neuron, Ye, Chen, and Liu (2022) proposed an extended prediction correction (EPC) method instead of the common Euler method. They utilized second-order EPC and, after obtaining the parameters, performed operations using shifts. They claimed that using the EPC method not only eliminates differential calculations but also reduces hardware requirements and improves neuron stability against errors. They proposed their LIF neuron with 18 flip-flop (FF) and 86 LUTs. Furthermore, they utilized the Genetic algorithm (Lambora, Gupta, & Chopra, 2019) to optimize neuron thresholds. This is not too distinct from optimizing thresholds during hyperparameter searches, given that such sweeps often employ evolutionary-based strategies.

Typically, two methods for updating neuron states are used in FPGA implementation: time-driven and event-driven update approaches. In the time-driven algorithm, all states of neurons must be checked at each time step to determine whether a spike is produced or not. Below the time-driven update equation is provided, in which a simplified LIF neuron type as $V_{t+1} = V_t \times \exp(\frac{-\Delta_t}{\tau})$ is assumed:

$$V_{(t+1,j)} = V_{(t,j)} \times \exp(\frac{-\Delta_t}{\tau}) + \sum_{i=0}^{n-1} S_i(t) \times W_{i,j}, \tag{36}$$

Table 6
Employed techniques for more optimal implementation of synapses and neurons.

Categories	Techniques	Purpose
Hardware implementation	Bit shifting	With each left bit shift, the number is multiplied by 2, and each right shift is divided by 2 to reduce the need for multipliers.
	Barrel shifter	Used to replace multipliers of smaller dimensions.
	Time multiplexing	Using limited hardware resources to perform several different operations at various times.
	Fixed-point Representation	It is used for decimal values instead of floating points to reduce resource consumption and complexity.
Approximations/ Numerical calculations	Euler Runge–Kutta (RK) 3rd-order RK (Rk3) Taylor series Extended prediction correction (EPC)	They are used to simplify differential equations and more complex functions for implementation.
	CORDIC	Computing math functions like Exp using only add and shift operations iteratively.
	LUT-based	Discrete points of the desired function are stored in LUTs.
	Finite-Difference method (FDM)	Used for approximate solving partial differential equations at a higher speed and lower complexity.
	Based-2	Converts the nonlinear functions like Exe to a 2^x function to simplify the digital implementation.
	Piecewise linear (PWL)	To simplify the nonlinear behavior of the equations, using several first-order lines.
	Linear regression	Finds a linear model that best fits the target function (it is used to model the Exp function).
	Nearest power of two	Coefficients are approximated to the nearest value of 2^N to change the multiplication operation to a simple shift.

where V_{t+1} is the membrane potential of the jth postsynaptic neuron at timestep $t+1$, $S_i(t) \in \{0,1\}$ is the state of the presynaptic neuron at t, $W_{i,j}$ is the connection weight between ith and jth neuron, n is the number of neurons, and τ is the leakage time constant. This method is often wasteful as it is unnecessary to update neurons that do not receive any input spikes (Liu et al., 2022), though it may benefit in networks with high firing activity, or where procedural workloads with deterministic execution is important.

On the other hand, in the event-driven algorithm, neurons are updated at the onset of an event. If an event occurs and a spike is triggered from a neuron, the events are queued and downstreams neurons are updated. This method can be much more efficient than the time-driven method, but sorting events can increase the hardware complexity. However, this process can also be executed in parallel, akin to biological systems (Han et al., 2020). The event-driven update is carried out based on:

$$V_{(t+1,j)} = V_{(t,j)} \times \exp(\frac{-\Delta_t}{\tau}) + W_{i,j}. \tag{37}$$

In Carpegna et al. (2022), a clock-driven approach is employed to update the membrane potential of neurons. This means that the membrane potential is evaluated for each clock cycle based on the exponential relationship of the LIF neuron. However, inputs are only examined in the presence of a spike. This method increases power consumption but utilizes fewer hardware resources.

Implementing neurons and learning rules in a precise manner similar to software simulation is complex for FPGAs and requires simplifications and techniques to reduce this complexity. Many of these approximations are tolerable in the context of data-driven tasks. Therefore, Table 6 presents a concise summary of the techniques used in the reviewed articles, along with their objectives.

3.3.1. Evaluation metrics

Given the large number of approximations made, we provide three main evaluation criteria commonly used:

- Mean Absolute Error (MAE):

$$\text{MAE}\% = \frac{1}{n}\sum_{i=1}^{n} |N_p - N_o| \times 100, \tag{38}$$

where N_p is desired parameter of proposed neuron, N_o is original parameter value of its neuron and n is time steps.

- Mean Relative Error (MRE):

$$\text{MRE}\% = \frac{\sum_{n}^{i=1} \frac{N_p - No}{N_o}}{n} \times 100. \tag{39}$$

- Normalized Root Mean Square Error (NRMSE):

$$\text{NRMSE}\% = \frac{\sqrt{\frac{1}{n}\sum_{i=1}^{n}(N_p(n) - N_o(n))^2}}{N_{o_{\max}} - N_{o_{\min}}} \times 100. \tag{40}$$

Various parameters can be used to evaluate the proposed neuron and determine its optimality. In addition to the error metrics above, factors such as the hardware resources utilized are also of paramount importance. A summary of the neurons whose data was available is presented in Table 7. This table summarizes the proposed neuron type, the number of bits, the simplification and implementation techniques employed, and the achieved accuracy with respect to a full precision implementation. The maximum achievable operating frequency and execution time should also be considered, but these have been excluded due to their dependence on the type of FPGA device. On the other hand, the values listed represent the best available performance under different execution conditions in the articles. It should be noted that the biological plausibility of a neuron is an important criterion for selecting and designing a neuron, indicating how many biological features of real neurons can reproduce. In Table 7, HH, HR, IZH, FHN, AdEx, and LIF neurons can be prioritized based on their biological plausibility, though designers may be concerned about neuron models that are compatible with gradient-based or local learning rules which will also sway which model is optimal for a given application.

3.4. Network architecture and system design

Even if individual blocks are well-designed and optimized, the network's overall architecture can often be determinative in overall performance. In this section, several recent FPGA system implementations are reviewed, and the method of designing a complete network and

Table 7
Comparison of basic parameters of proposed neurons and their employed implementation techniques.

Ref.	Neuron type	Bit width		Technique used	Resources			Error
		Integer part[a]	Fraction part		LUT	FF	DSP	
Ambroise et al. (2013)	IZH	10	8	• Linear regression method	473	749	1	MRE: 1.78%
Leigh et al. (2020)	IZH	7	9	• Euler method • Nearest power of two for coefficients	702	41	0	MRE: 2.08%
Ghanbarpour et al. (2023)	IZH HH	16	20	• Based-2 • Euler method	173 352	153 408	0 0	MAE: 3.50% MAE: 4.20%
Heidarpour et al. (2016)	AdEx	14	23	• CORDIC • Euler method • Shift and add operations instead of multipliers	829	1221	0	NRMSE: 0.039%
Jokar et al. (2019)	IZH FHN HR	N/A	N/A	• Uniform piecewise linear (PWL) segmentation • Barrel shifter • Segment Address Encoder • Euler method • Nearest power of two	203 183 220	182 193 253	0 0 0	MAE: 1.36% MAE: 0.99% MAE: 1.93%
Heidarpur et al. (2024)	IZH	14	16	• CORDIC • Euler method • Approximation of coefficients using the sum of based-2 values	532	297	0	NRMSE: 0.11%
Carpegna et al. (2022)	LIF	13	3	• Using an offset to obtain zero V_{rest} • Nearest power of two	62	40	0	N/A
Ye et al. (2022)	LIF	16		• Extended prediction correction (EPC)	86	18	0	MAE: 0.89%

[a] With the sign bit.

the interaction of its components and sub-blocks are investigated. We find that for more effective interaction between different sub-system across the FPGA, it is essential to employ techniques and additional components that might not be apparent in software frameworks.

Han et al. (2020) combined event-driven and time-driven methods to implement the network, utilizing multiple event queues each marked with a timestamp Q_n, where the range of n is from 0 to $D-1$, and D denotes the maximum delay. Therefore, the events that occur at the same time are placed in a queue and there is no need to sort them. During each time step, the active queue is Q_0 where event processing takes place and neuron states are updated. If a neuron fires, a new event is placed in the events queue with a delay. The current step does not end until Q_0 is empty. Once the event queue becomes empty, the labels are decreased by one and Q_0 is then reused in a circular form, i.e. Q_{D-1}. With the implementation of this hybrid update algorithm, the sorting process is eliminated, resulting in a decrease in the system's delay during run-time. This structure consists of four main components, each of which has a controller. The event queues sub-module is responsible for the hardware implementation of the event queues, while the event controller sub-module manages these queues. Also, weight, state, and delay memory sub-modules are employed to keep the weights and states of neurons and the delay of various events, respectively. The main part of this structure is the state updater, which checks the state of neurons to set their value to a predetermined value when they are activated and define a new event. The state updater can perform in parallel and check the state of several neurons at the same time.

Inspired by Han et al. (2020) and Liu et al. (2022) proposed a new structure with a time and event hybrid algorithm. The design of this structure consists of three parts: neuron computation architecture, driven algorithm design, and platform construction and testing. In the neuron design part, this structure supports both IZH and LIF neurons. In the driven algorithm design, three parts of Poisson spike algorithm, address event representation (AER) bus and ring-shape first in-first out (FIFO) are used for its implementation. And finally, by integrating all the parts, the accelerator platform is completed. An AER address bus is used for optimal transmission of spikes (Boahen, 2000). A ring-shaped FIFO array is employed as a buffering of input spike events and internal spike events. The neuron update will continue as long as the FIFO is not empty.

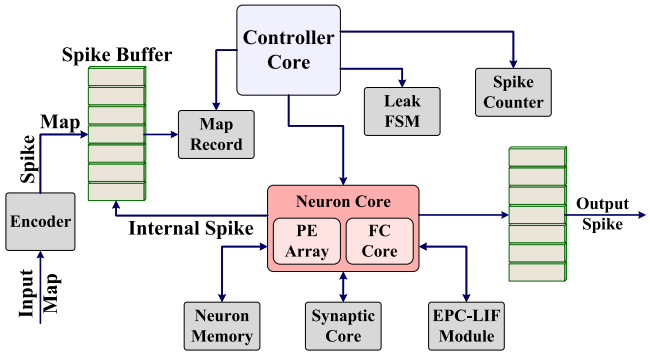

Fig. 9. Architecture of ELIF-NHAS system.
Source: Adopted from Ye et al. (2022).

In the article (Ye et al., 2022), the authors propose an optimized architecture named ELIF-NHAS, capable of performing multi-layer perceptron (MLP) and convolution computations using a systolic array-based architecture (Xu, Ma, Guo, & Li, 2023). The overall view of this architecture is illustrated in Fig. 9. For proper system functionality, various components and modules need to be designed and placed alongside each other. The network's input, which in this article is image data, is fed into the *Encoder* unit, converted into spikes using Poisson rate coding, and then saved in *Spike Buffer*. The *Map Record* module rearranges the data based on the network's type and requirements. The input data arrangement varies for FC layers and different convolutional windows. The *Neuron Core* consists of *FC Core* for fully connected computations and *PE Array* for convolutional computations. The computation process is performed in a pipelined and parallel manner, based on Liu et al. (2022). The neurons are located in the *EPC-LIF Module*, the details of which are provided in Section 3.3. The membrane voltage and neuron states are updated and stored in *Neuron Memory* at each time step. When the system performs an update, the weight corresponding to the address is collected via the *Synaptic Core*. The *Leak FSM* module is responsible for controlling the synchronization of different neuron leakage. The *Spike Counter* counts the number of spikes

emitted by the neuron, providing a basis for determining the final outcome. Finally, the *Controller Core* is responsible for coordinating all the different system modules. Additionally, this module contains a memory that stores all the parameters of the network topology, which can be configured via the UART bus, allowing for easy modification of the network topology.

In Mitchell, Schuman, and Potok (2020), the architecture named μCaspian is presented, along with the corresponding PCB development board design. This design offers a cost-effective size, weight, and power-optimized neuromorphic hardware platform. The primary objective of this paper is to offer researchers an affordable hardware option and an open-source FPGA workflow. This is because popular hardware options, such as IBM's TrueNorth (Akopyan et al., 2015) and Intel's Loihi (Davies et al., 2018), are not readily accessible to the public. As a result, deploying or assessing them in a real-world application can be challenging or even impossible. Xia, Levi, and Kohno (2020) have provided this capability for their network using the Ethernet interface to communicate with other FPGAs and to be able to form a large-scale network.

Tang and Han (2023) proposes a weight-binarized spiking neural network (WB-SNN) to reduce memory footprint and hardware resources. In this network, spikes are considered as 0 and 1, representing the absence and presence of spikes, and the weights are constrained to −1 and 1. Furthermore, to avoid considering the signed bit, −1 is represented as 0, requiring only one bit of memory to represent both weight and spike. Prior work has shown that SNNs are highly tolerant to weight binarization as they project information into both the state of the neuron and across time (Eshraghian & Lu, 2022). In this architecture, a priority encoder (PE) has been employed for communication between the layers in such a way that the PE encodes the active bit index with the highest priority. The encoded bit indicates the memory address location, so the bit width is reduced from N input channels to $\log_2 N$. When the PE completes encoding the bit with the highest priority, a priority-resolving circuit (PRI) clears this bit, allowing the PE to proceed to encode the bit with the second-highest priority. Additionally, a counter has been utilized for the implementation of IF neurons to achieve the simplest and most resource-efficient implementation for all elements.

Binarizing and quantizing weights is a common technique to reduce memory consumption. However, it should be noted that this technique needs to be applied during training, as Qiao et al. (2020) demonstrated that applying this technique after training leads to a significant reduction in network accuracy. In the same vein, Hu et al. (2021) trained their network using quantized and binarized weights based on STDP. They demonstrated that the network with 4-bit quantized weights achieved an accuracy of 93.8% for the MNIST dataset, while the network with 32-bit weights achieved an accuracy of 94.1%. This indicates that the drop in accuracy, compared to the significant reduction in memory usage, can be negligible. Additionally, the binary network reached an accuracy of 92.9% which remarkably reduced memory usage and transformed the input-weight multiplication processes into simple AND operations.

In the article (Chu et al., 2022), a recurrent network based on Yan et al. (2021) and Corradi et al. (2019) has been employed for the hidden layer, featuring recurrent and sparse connections, known as a 'liquid pool'. As illustrated in Fig. 10, the input spikes connect to the liquid pool, leading to the processing of temporal features and consequently improving classification accuracy. The last two layers are responsible for classification tasks. The output of the liquid pool connecting to the classifier, as well as the input layer to the liquid pool, are designed with sparse connections to reduce processing time and cost. Dense connections are only present in the output layer to reduce compute demands. Also, all weights in this network have been trained using the spatio-temporal backpropagation (STBP) algorithm (for more information about STBP see Wu, Deng, Li, Zhu and Shi, 2018).

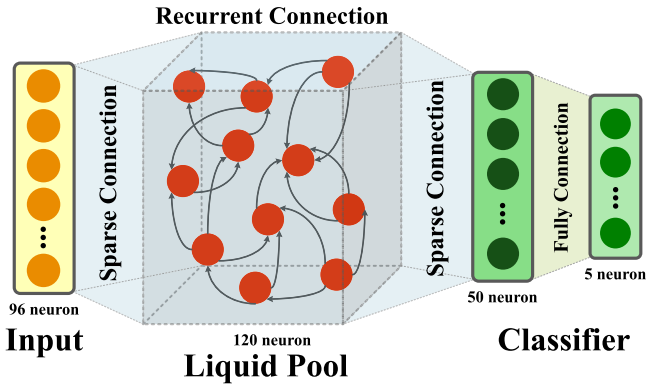

Fig. 10. Network topology of the proposed spiking recurrent MLP model.
Source: Adopted from Chu et al. (2022).

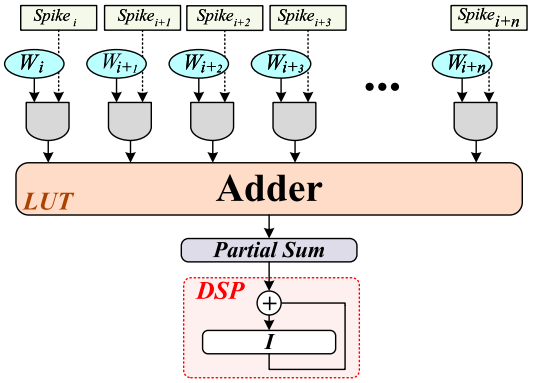

Fig. 11. Synapse current integration module architecture for communication between layers.
Source: Adopted from Leone, Raffo, and Meloni (2022).

To create a large-scale network, Leone et al. (2022) proposed a fully connected network using off-chip double data rate (DDR) memory because they demonstrated that the bandwidth for data transfer is sufficient to avoid any disruptions in the system. They stored the synaptic weights in DDR memory, which communicates with the FPGA through advanced extensible interface (AXI) High-Performance ports. However, they stored the Izhikevich neurons' parameters and synaptic currents in Block RAM (BRAM). Since the input to the Izhikevich neuron must be current, they operated according to Fig. 11 for inter-layer communication. Weights are fetched from DDR memory as 64-bit values and utilized directly without the need for buffering. Since the weights are 8-bits, so 8 weight bits are passed as input to the AND gates simultaneously as shown in Fig. 11. When a spike is applied through each neuron, its weight is summed using an adder implemented with a LUT, and then accumulation is performed with a DSP.

As mentioned, neuromorphic hardware platforms, predominantly digital, have been introduced by various individuals and companies, each presenting unique advantages and disadvantages. Yang, Wang, Hao et al. (2021) introduced a biological-inspired cognitive supercomputing system (BiCoSS) platform that utilizes 35 Cyclone IV FPGAs, each connected to two synchronous dynamic random-access memories (SDRAMs) and additional components and peripheral circuits. Leveraging a heterogeneous multi-core architecture, this platform can support more than four million spiking neurons in real-time. One of their main goals has been to develop a platform with multigranular capabilities, a feature that almost none of the previous platforms fully provided. Additionally, they aimed to bridge the gap between neuroscience and neuromorphic architecture. A multigranular neuromorphic architecture in this study is divided into six parts. From the neural morphology

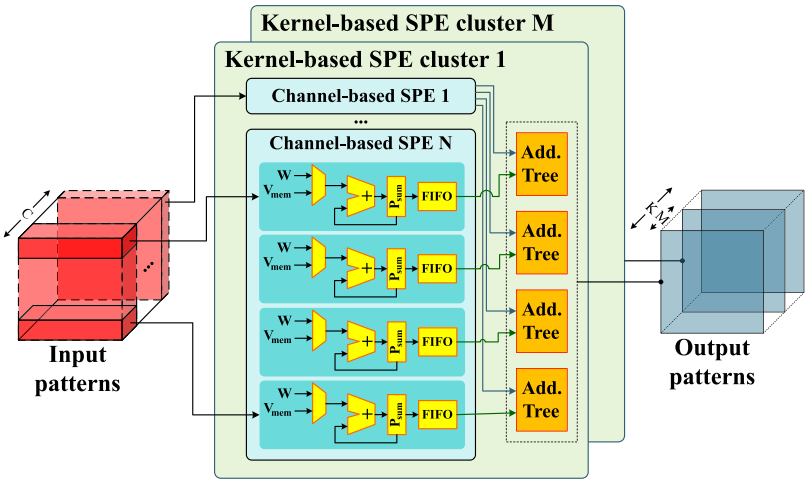

Fig. 12. The filter-based spiking processing unit structure.
Source: Adopted from Chen, Gao, Fang et al. (2022).

perspective, the architecture includes both point neuron models and compartmental neuron models (Sun, Zeng, Zhao, & Zhao, 2023; Yang, Deng et al., 2019); in terms of biological plausibility, it supports various neuron models, such as LIF, IZ, HH, and others; at the network topology level, it encompasses feedforward and recurrent networks; at the synaptic level, it includes both electrical and chemical synapses; and at the learning rule level, it supports a variety of learning rules.

3.4.1. Convolutional spiking neural network

Most efforts in FPGA-based SNNs have focused on investigating and implementing MLPs, whereas CNNs tend to dominate the number of operations in computer vision tasks (Aung et al., 2021; Lemaire et al., 2020). This enhanced feature extraction capability comes at the cost of increased operations. As such, CNNs stand to benefit significantly from activation sparsity which can significantly reduce the total number of operations required (Tapiador-Morales, Linares-Barranco, Jimenez-Fernandez, & Jimenez-Moreno, 2018; Veeravalli et al., 2022).

Chen, Gao, Fang and Luan (2022) proposed a convolutional SNN in which processing elements are constructed with several filter-based spiking processing element (SPE) clusters, illustrated in Fig. 12. As evident, each cluster consists of several channel-based SPEs and adder trees. Each channel-based SPE receives a subset of kernels within a filter and computes partial membrane potential sums, where each is divided into 4 streams. The total partial sums from each stream are managed by an adder tree, ultimately calculating the output membrane voltage of specific channel in each cluster. Each SPE cluster is connected to a memory bank for storing and applying weights. They demonstrated that since the input channel sparsity varies, SPEs with more zeros perform computations faster, and SPEs with non-zero values create the most delay, leading to an imbalance. To address the issue of unbalanced event-driven workloads, this paper proposes the channel-balanced workload schedule (CBWS) method, in which input channels are divided into N groups with approximately equal workloads and processed by N SPEs. The approximate relative workload in this paper is obtained using the approximate proportional relation construction (APRC) method and is utilized in CBWS. After executing these two methods, the balance ratio increased from 79.63% to 94.14%.

Aung et al. (2021) proposed a high-performance RTL IP for accelerating convolutional SNN inference by integrating IF spiking neurons into a CNN. The neuron architecture is configured based on Pani et al. (2017), and it utilizes a three-stage adder tree for integrating weights and inputs. The first stage of the adder is implemented with a DSP, and the other two stages are executed with configurable logic blocks (CLBs). Additionally, the network training is performed using standard back-propagation.

To achieve better feature extraction, Lemaire et al. (2020) initially processed input images by utilizing a CNN and then employed an SNN for optimized classification. The network structure is illustrated in Fig. 13. As depicted in the figure, RGB images are applied as inputs to the CNN network with dimensions of 28×28. Subsequently, convolutional and max-pooling layers extract the latent features with dimensions of $4 \times 4 \times 5$ (Hamdan, 2018). These features are flattened and converted into spikes using a spike generator. These spikes are passed through two layers of SNNs with IF neurons, and the output of the final layer is used by the Terminate Delta Module to determine the output class and perform classification.

Ju et al. (2020) implemented a deep SNN that was trained using ANN-SNN conversion (Rueckauer, Lungu, Hu, Pfeiffer, & Liu, 2017). Max-pooling helps reduce the parameters of the network (Scherer, Müller, & Behnke, 2010). Nevertheless, its hardware implementation poses challenges as the firing rate of neurons must be considered at each time step. Inspired by Hu and Pfeiffer (2016), the firing rate is calculated according to the proposed Eq. (41). In this equation, $x_i(t)$ represents the presence or absence of spikes for neuron i at time t, and T is the time window length for a presentation.

$$f_i(t) = f_i(t-1) + x_i(t)(T-t) \tag{41}$$

Li, Shen, Zhao, Zhang, and Zeng (2023) proposed solutions to address mathematical computation challenges in FPGA and memory constraints. They implemented a spiking convolutional neural network (SCNN) on a Xilinx Ultrascale FPGA. They bundled synapses and fed them to DSP48E2, and then enabled or disabled these DSPs using spikes, providing a high-speed crossbar with fewer hardware resources. For a 16×4 crossbar, only 8 DSPs are required, with no use of LUTs and FFs. Additionally, they achieved better memory efficiency by balancing between off-chip memory bandwidth and on-chip memory consumption for weights and neuron states through a hierarchical delivery system. To accelerate the process of inferring SCNN from DNN to SNN, Wang, Wang et al. (2020) proposed an SNN inference engine called SIES. They integrated all computational units into a unified systolic array to accelerate membrane potential computation. They added an optional max-pooling module to minimize unneeded data movement and developed a hardware mechanism for faster input spike preparation. Their proposed design has significantly enhanced data processing efficiency and increased hardware parallelization of SNNs.

To leverage the capabilities of CNN while preventing their hyperparameter-heavy nature for efficient execution on FPGA, Chen, Liu, Ye, and Chang (2023) proposed a network using parameter fusion and quantization techniques to utilize fewer resources. Their

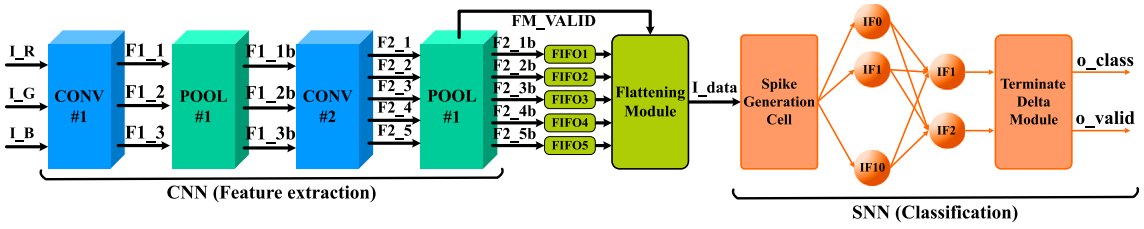

Fig. 13. The topology of hybrid CNN-SNN network and its various components and tasks.
Source: Adopted from Lemaire et al. (2020).

Table 8
A general summary of the number of used hardware resources based on network size.

Article	FPGA type	Neuron model	#synapses	#neurons	Component			
					LUT	FF	BRAM	DSP
Han et al. (2020)	Xilinx Zynq 7000 (XC7Z045)	LIF	16.8 million	16,384	5381	7309	40.5	–
Liu et al. (2022)	Xilinx Kintex-7 (XC7K325T)	LIF/IZH	16.77 million	16,384	46,371	30,417	150	65
Zhang et al. (2020)	Xilinx Vertix-7	LIF	51	144	13,862	5,487	–	144
He et al. (2021)	Xilinx Zynq 7000 (XC7Z045)	LIF	1.81 million	2,932	15,621[a]		129	8
Asgari, Maybodi, Payvand, and Azghadi (2020)	Xilinx Kintex-7	LIF	122	16	34,646	5088	–	256
Guo et al. (2021)	Xilinx Virtex-7 (VC709)	LIF	N/A	[b]800[c]	45,531	50,614	289	N/A
				[d]800	88,053	91,281	289	N/A
Tang and Han (2023)	Xilinx Virtex-7 (XC7VX485T)	IF	1.85 million	2844	24,784	14,603	56.6	–
Liang et al. (2021)	Xilinx Virtex-7	single-spike IF	0.4 million	512	16,324	11,612	–	–
Carpegna et al. (2022)	Xilinx Artix-7	LIF	313,600	1384	29,145	26,853	45	–
Gupta et al. (2020)	Xilinx Virtex-6	Simplified LIF	12,544	800	56,230	23,238	16	64
Veeravalli et al. (2022)	Xilinx Zynq 7000 (XC7Z020)	IF	N/A	32,768	23,375	29,526	97.5	22
Huang et al. (2020)	Xilinx Artix-7	IF	4320	148	986	481	–	–
Chu et al. (2022)	Xilinx Artix-7 (XC7A100T)	LIF	N/A	271	619	783	17/40[e]	–
Leone et al. (2022)	Xilinx Zynq 7000 (XC7Z020)	IZH	9.6 million	3098	24,480	53,158	130.5	140
Corradi, Adriaans, and Stuijk (2021)	Xilinx	LIF	1.6 million	2954	140,206	131,977	306	10
Deng, Fan, Wang, and Yang (2021)	Cyclone-4	Modified-LIF	N/A	66	49,082	19,325	–	–
Carpegna, Savino, and Di Carlo (2024)	Xilinx Zynq 7000 (XC7Z020)	LIF	101,632	922	4314	3298	18	0

[a] Sum of LUT & FF.
[b] Euler method.
[c] Output neurons.
[d] RK3 method.
[e] LUTRAM.

proposed network has the capability to execute standard and residual convolution. Furthermore, articles such as Chen, Gao and Fu (2022), Gerlinghoff, Wang, Gu, Goh, and Luo (2022), Khodamoradi, Denolf, and Kastner (2021), Sommer, Özkan, Keszocze, and Teich (2022) and Irmak, Corradi, Detterer, Alachiotis, and Ziener (2021) have also focused on implementing convolutional SNNs on FPGA, which can be used for further understanding and development.

For a general comparison and better understanding of SNN implementation and execution on FPGA by various studies in the Network Architecture and System Design section, articles that provided information are summarized in Table 8. They are outlined based on the FPGA type and the number of hardware resources used according to the number of synapses and neurons. Also, information related to the classification tasks of the reviewed articles, including the type of training, learning method, network accuracy, and hardware description language used, is summarized in Table 9. Furthermore, the dataset used for classification operations is mentioned and referenced to introduce common datasets to researchers for these tasks. We note that these all use time-static datasets, which does not fully exploit the potential of spike-based temporal computing, and so we encourage the research community to test their implementations on time-varying data, as vanilla ANNs are able to process images without projecting them into the time dimension, often with lower latency (Ottati et al., 2023).

On the other hand, recent studies have focused on emerging and more complex networks, such as transformer-based networks, implemented on FPGAs, such as Refs. Udeji and Margala (2024) and Vishwamith, Isik, and Dikmen (2024). These efforts hold promise for future advancements. Therefore, we also encourage researchers to go beyond implementing simpler networks and take steps toward handling more complex tasks, addressing the challenges of such implementations on hardware and FPGAs.

3.5. Exploitation of sparsity on FPGA

While FPGAs are inherently designed for synchronous processing, which poses challenges for asynchronous computation typical in sparse neural networks, strategic design choices can mitigate these limitations (Shahsavari, Beaumont, Thomas, & Brown, 2020). Where the overhead of additional circuitry for exploiting sparsity, e.g. power and clock gating, multiplexers that deactivate further processing in absence of data inputs (sparse activations), and hardware that encodes sparse data formats, is less than the overhead from executing all computations and memory accesses, then it makes sense to implement acceleration techniques for sparse workloads (Chen, Ye, Liu, & Zhou, 2024). Power and clock gating techniques can deactivate idle functional units within the FPGA when they are not processing active neurons, effectively reducing dynamic power consumption without disrupting the synchronous operation of the device (Leone et al., 2024).

By partitioning the FPGA fabric into regions that can be independently clock-gated based on workload, designers can create a more power-efficient implementation that capitalizes on the irregular computation patterns of sparse activations (Blott et al., 2018). Additionally, employing asynchronous FIFO buffers and handshaking protocols within a predominantly synchronous design can help manage data flows resulting from sparsity.

Leveraging sparse activations in neural networks presents significant opportunities to optimize memory access patterns and computational efficiency on FPGAs (Li, Li et al., 2024). Sparse activations—where many neurons output zero values—are common in networks

Table 9
A comparison of the proposed classification networks based on the training method and the dataset used.

Article	Training	Learning method	Datasets	Accuracy	Coding
Zhang et al. (2020)	Supervised	Tempotron	N/A	96%	Verilog
He et al. (2021)	Supervised	Triplet R-STDP	MNIST (LeCun et al., 1998)	93%	N/A
			Fashion-MNIST (Xiao, Rasul, & Vollgraf, 2017)	78.5%	N/A
Mehrabi, Bethi, van Schaik, Wabnitz et al. (2023)	Supervised	ODESA	Iris (Fisher, 1936)	80%	N/A
Quintana et al. (2022)	Supervised	R-STDP	N/A	95%	N/A
Guo et al. (2021)	Unsupervised	Triplet STDP	MNIST	89.5%[a]– 89.7%[b]	Verilog
Liang et al. (2021)	Unsupervised	N/A	MNIST	96%	N/A
Carpegna et al. (2022)	Unsupervised	STDP (offline)	MNIST	80.2%	N/A
Veeravalli et al. (2022)	Unsupervised	STDP	Caltech-101 (Fei-Fei, Fergus, & Perona, 2004)	95.7%	SystemVerilog
Ju et al. (2020)	Supervised	Backpropagation (CNN-to-SNN)	MNIST	98.94%	Verilog
Chu et al. (2022)	Supervised	STBP	MIT-BIH (Moody & Mark, 2001)	98.22%	Verilog
Xing et al. (2022)	Supervised	Backpropagation (CNN-to-SNN)	MIT-BIH	98.26%	N/A
Li et al. (2023)	Supervised	Backpropagation (CNN-to-SNN)	MNIST	98.12%	Verilog
			CIFAR10	91.36%	N/A
			CIFAR100	64.28%	N/A
Ye et al. (2022)	Supervised	Backpropagation (CNN-to-SNN)	MNIST	98.45%	Verilog
			Fashion-MNIST	85.38%	N/A
			SVHN (Netzer et al., 2011)	72.63%	N/A
Carpegna et al. (2024)	Supervised	Back-Propagation Through Time	MNIST	96.83%	VHDL
			SHD (Cramer, Stradmann, Schemmel, & Zenke, 2020)	75.44%	

[a] Euler method.
[b] RK3 method.

using activation functions like ReLU, and especially in SNNs. Exploiting this sparsity can reduce memory bandwidth requirements and computational workload, which is particularly beneficial given the limited on-chip memory and parallel processing capabilities of FPGAs (Liu, Cui et al., 2024). Depending on the size and demands of the neural network in question, different approaches to leveraging sparsity will yield different benefits.

• **Reducing memory access**: For small-scale models that fit entirely within the FPGA's on-chip memory, techniques such as zero-skipping can be implemented to minimize SRAM accesses and computations. By designing custom processing elements that detect zero-valued activations, the FPGA can bypass unnecessary computations and memory reads/writes associated with these zeros (Jiang et al., 2024). This selective computation reduces the number of operations and conserves power by preventing the toggling of logic gates and memory elements. With respect to computations, sparse matrix–vector multiplication (SpMV) optimizations can also be used effectively for fully connected layers where the density of non-zero elements is low (Liu, Chen et al., 2024). Specific implementations can go beyond RTL that naively describe SpMV, and can express such operations as architectures that incorporate zero-skipping mechanisms (Ma, Cao, Vrudhula, & Seo, 2017). In these designs, processing elements are equipped with control logic to detect zero-valued inputs and outputs, allowing the array to bypass unnecessary computations and data movements.

• **Reduced memory usage**: SNN-based acceleration techniques can draw upon inspiration from deeply pruned neural networks on FPGAs. Pruning involves removing less significant weights and neurons from the network, resulting in sparser models that require fewer computational resources. Fine-grained pruning can significantly reduce the model size and computation requirements, enabling the entire model to fit into on-chip memory without accessing external DRAM. For example, FINN (Fast, Scalable Quantized Neural Network Inference) leverages both extreme quantization and weight sparsity (Blott et al., 2018). However, weight sparsity may still require a memory access to check whether the weight exists before skipping downstream operations, unless the weights are so sparse that the addresses of non-zero weights can be encoded in a compressed representation that fits on-chip SRAM.

In large-scale models that require external DRAM due to their substantial size, exploiting activation sparsity becomes even more crucial. Compressed sparse formats, such as compressed sparse row (CSR) or coordinate (COO) formats, can efficiently represent activation and weight matrices by storing only non-zero elements and their indices. This significantly reduces the volume of data transferred between the FPGA and DRAM (Li, Li et al., 2024).

Specialized memory controllers and dataflow architectures can handle these sparse data formats. By fetching and processing only the non-zero elements, the system reduces DRAM bandwidth requirements and lowers energy consumption associated with memory transactions. Techniques like memory tiling and blocking further optimize memory access patterns by ensuring that data fetched from DRAM is effectively utilized before being replaced.

• **Power gating strategies**: Power gating strategies are particularly effective for SNN implementations on FPGAs. By deactivating idle neurons or synapses until a spike occurs, the network significantly reduces power consumption during periods of inactivity. This approach aligns with the event-driven nature of SNNs and leverages the synchronous operation of FPGAs to achieve energy efficiency. However, it should be noted that due to the fixed structure of FPGAs, applying power gating techniques is somewhat more challenging and differs from similar methods used in circuit-level implementations. These techniques tend to be more effective in ASIC designs (Jahanirad, 2023).

4. SNN applications

With the further development of SNNs and their capabilities for more advanced tasks, efforts have extended beyond expanding the

network and designing its components, and into real-world application. In this section, a portion of the proposed applications that have been implemented on FPGA in recent years is examined.

Portable healthcare devices are one of the promising applications of SNNs due to their efficiency and low power consumption compared to ANNs, making them a good candidate for continuous patient monitoring, control, and adaptation (Chu et al., 2022; Corradi et al., 2019). However, shallow networks proposed on hardware have not yet been fully exploited for biomedical signal processing. In this vein, Xing et al. (2022) have utilized an SNN for the real-time portable processing of ECG classification on FPGA. To address the low accuracy of previous SNN work (Amirshahi & Hashemi, 2019), they utilize a channel attention module (CAM) to extract key information from the ECG channels and incorporate it into the morphological features of the signal. By doing so, the distinctive features become more prominent, aiding in the optimal and more accurate detection of ECG arrhythmia. In this article, the conversion from ANN to SNN has been employed. Also, Chu et al. (2022) attempted to implement ECG classification for continuous health monitoring on FPGA. They converted input ECG signals to spikes using an LC-ADC and trained their lightweight SNN models using the STBP learning algorithm, achieving an accuracy of 98.22% on the MIT-BIH dataset for seizure detection (Moody & Mark, 2001).

Beyond biomedical signal analysis, SNNs have gathered much attention in robotics as this is a domain in which the available compute resources are significantly constrained. For example, Walravens, Verreyken, and Steckel (2020) proposed the use of SNNs on FPGA with reinforcement learning for collision avoidance in robots. Refs. Hao et al. (2019) and Wang et al. (2023) were able to control a robotic arm in real-time by modeling the cerebellum on FPGA. Also, in Canas-Moreno et al. (2023), a spike-based proportional–integral–derivative (PID) controller is applied to a Scorbot ER-VII robotic arm with 6 degrees of freedom. In this controller, motors are fed with pulse frequency modulation instead of pulse width modulation, so that their behavior mimics the activity of motor neurons on muscles. This controller is implemented on two types of Xilinx Spartan and Zynq MPSoC FPGA and has been tested. Gutierrez-Galan, Dominguez-Morales, Perez-Peña, Jimenez-Fernandez, and Linares-Barranco (2020) presented NeuroPod, a real-time neuromorphic spiking central pattern generator (CPG) used in robotics applications. Inspired by biological examples, such as insects, the study aims to advance robotics beyond repetitive tasks and towards more complex behaviors. Their hexapod robot system is composed and controlled by SpiNNaker (Furber et al., 2014) and FPGA boards. Real-time measurements validate simulation results, demonstrating the robot's ability to perform gaits seamlessly without balance loss or delays.

The auditory system is one of the principle components of the human brain, capable of receiving and understanding sounds through the ear. Sound recognition is a highly temporal task, leading (Deng et al., 2021) to propose an auditory system that benefits from the temporal properties of SNN for implementation on FPGA. They converted acoustic input waves into spikes based on intensity and frequency using Poisson coding and classified it through a network inspired by Khatami and Escabí (2020). This network comprised of 6 layers of 11 modified LIF neurons and classified the input using Bayesian inference. Their hierarchical structure enabled them to achieve acceptable accuracy in the presence of noise. They also expanded their work in the paper (Deng, Fan, Wang, & Yang, 2023) by developing a larger network consisting of 10 FPGAs and improving its components to achieve even higher performance compared to their previous work. They achieved an accuracy of 85.75% for TIMIT (Garofolo, 1993) speech detection in the presence of noise.

In Asgari et al. (2020), an SNN for context-dependent tasks is presented in which reinforcement learning is used. In this paper, LIF neurons are approximated and reduced to only two sum operations. Also, STDP is implemented using LUT-based approximation, which leads to the reduction of hardware cost and power consumption. Yang,

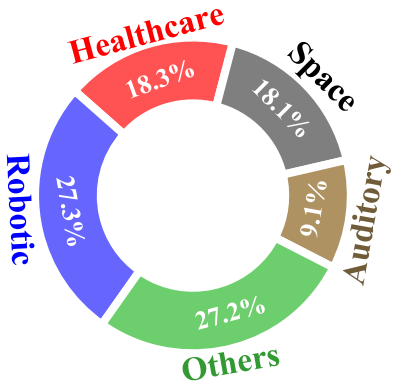

Fig. 14. Percentage of each specific application used on the FPGA and reviewed in this article.

Wang et al. (2021) also implemented their neuromorphic system on FPGA with the aim of context-dependent learning and fault-tolerance.

Lemaire et al. (2020) addressed the requirement for a low-power embedded network with high accuracy in satellite image classification for space applications by integrating a CNN for feature extraction and an SNN for efficient classification (more details are provided in Section 3.4.1). Lent introduced a cognitive network controller (CNC) in Lent (2020, 2021) that uses SNNs to make adaptive routing decisions in challenging scenarios using FPGA-based acceleration. The study by George, Soci, Miscuglio, and Sorger (2021) explored symmetry perception using SNN, which leverages the coincidence detection property of SNNs to detect mirror symmetry. They tested their proposed system on geospatial satellite images and demonstrated automated recognition of man-made structures over vegetation through different hardware, including FPGA. This capability holds promise for image rendering, robotics, brain-machine interfacing, and bridging the gap between sensor inputs and high-level interpretation.

Huang et al. (2020) implemented the detection of radioisotopes using SNN on FPGA due to the event-driven nature of these networks. They initially transformed the results from an ANN to an SNN using SpiNNaker (Furber et al., 2014) to improve the execution speed. Subsequently, they implemented the overall results on an FPGA. Also, in Huang et al. (2021), they proposed another structure to perform this task and improved their previous design.

Structural health monitoring (SHM) evaluate damages that influence structural reliability. To check for damage due to load and everyday use, modern SHMs require continuous monitoring. In Javed, Harkin, McDaid, and Liu (2021), a damage classification model based on SNN for SHM on FPGA is proposed. This model offers low cost, energy-efficient performance, and a high accuracy of 99.46% in classifying earthquake-induced damage on a 7-story concrete building.

Unmanned aerial vehicles (UAVs) interact with various sensors to collect physical data from the surrounding environment, typically utilizing DNN for identification purposes. Corradi et al. (2021) have improved the power-efficiency of UAVs with limited energy resources by implementing SNNs instead of DNNs. They classified pixelwise agricultural products using the bi-temporal dataset of optical-radar. Yang, Tan, Lei and Linares-Barranco (2023) developed an intelligent navigation system for transportation using the Internet of Vehicles (IoV) and implemented fault-tolerant edge computing on a LaCSNN system consisting of six Altera Stratix III FPGA chips (Yang, Wang et al., 2019). This system, utilizing bio-inspired computational methods, enables precise navigation and obstacle avoidance in complex environments. The key innovation of this research is the development of a fault-tolerant routing algorithm for spiking information transfer, along with the design of a framework for intelligent edge computing systems. Experimental results show that this system improves resource efficiency, decreases processing delays, and enhances fault tolerance for

real-time processing tasks. The proposed model can be a foundation for intelligent navigation systems and obstacle avoidance in autonomous vehicles, utilizing bio-inspired methods in smart transportation.

Fig. 14 shows the percentage of FPGA utilization for each application for better comprehension, as discussed in this section. It should be noted that most articles in previous sections had worked on classifying images including MNIST, which is a type of image processing. However, in this section, more focus is placed on real-world tasks that move beyond toy datasets.

5. Future works, gaps and opportunities

In this section, the challenges of developing SNNs on FPGA are investigated. These challenges are based on the issues and gaps identified in recent articles, emphasizing the need for further attention and efforts to address these problems in future works.

• **Brain-inspired multilayer supervised learning:** The training of SNNs remains an open research question, and local training algorithms that perform to the same efficacy as error backpropagation have not yet been effectively developed (Mehrabi, Bethi, van Schaik, Wabnitz et al., 2023). Surrogate gradient descent has scaled to large architectures to overcome the non-differentiability of spikes, though this workload is typically performed off-chip (Neftci, Mostafa, & Zenke, 2019). The ReckOn chip proposes the use of an approximation of real-time recurrent learning (RTRL) to avoid increasing memory complexity, though the added approximations make it challenging to scale up RTRL-based learning rules to large-scale architectures (Frenkel & Indiveri, 2022). The implementation has recently been ported to an FPGA, showing promising pathways forward for alternative learning rules to backprop that can still perform quite effectively (Linares-Barranco, Prono, Lengenstein, Indiveri, & Frenkel, 2024).

While RTRL-based techniques address the issue of temporal locality in backpropagation through time, they still suffer from a lack of spatial locality, update locking, and requiring symmetric backward paths. Techniques that have a stronger degree of spatial locality, like ReSuMe (Ponulak & Kasiński, 2010), Chronotron (Florian, 2012), Tempotron (Gütig & Sompolinsky, 2006), and SpikeProp (Bohte, Kok, & La Poutré, 2000), using various loss functions, have managed to adapt the backpropagation algorithm for single-layer supervised training networks. It is worth noting that these approaches demand extensive computations for calculating output errors and updating the layers, typically executed on GPUs and occupying significant resources on FPGAs.

Considering the above, developing an appropriate supervised method that aligns more closely with the biological principles of the brain and supports multi-layer training can be essential for enhancing computational performance and energy efficiency. As a result, this purpose can improve hardware efficiency, reduce power consumption and lead to better compatibility of the system with hardware such as FPGA. Additionally, it offers improved solutions to address the deficiency of brain-inspired algorithms in supervised learning that can significantly reduce training complexities.

• **Hardware availability:** A hindrance to the progress of neuromorphic engineering is the limited availability of hardware. Despite the substantial research in advancing neuromorphic hardware and the creation of neuromorphic chips such as IBM's TrueNorth and Intel's Loihi, gaining access and utilization of neuromorphic hardware remains challenging for neuromorphic and non-neuromorphic researchers (Mitchell et al., 2020). Promising efforts have been made in enabling widespread academic access to custom hardware, such as SpiNNaker 2 (Gonzalez et al., 2024) and Loihi, but reconfigurable hardware solutions are required to continue driving forward the exploration of new and effective learning rules, since adding these to ASICs is too slow a process. A promising effort is the DeepSouth large-scale FPGA system (ICNS, 2024), which will be made publicly available in 2025.

• **Memory bottleneck:** One of the primary concerns in developing SNNs on FPGA arises from the memory limitations as a bottleneck. As network sizes grow, this bottleneck restricts further performance improvements. The growing number of parameters poses challenges for hardware design, particularly in efficiently accessing distributed memories. This inefficiency has become a primary constraint in implementing SNNs on platforms with hardware constraints, especially for FPGAs with limited resources (Tang & Han, 2023). Hence, providing methods such as simplifying the network and elements, leading to reduced memory and resource requirements without significantly compromising network accuracy, can prove highly efficient and crucial.

One of the solutions is to use off-chip memory such as DDR. However, using it also has disadvantages such as high power consumption and bandwidth limitations in case of further parallelization of the network. Therefore, an appropriate method for accessing off-chip memory, on-chip data buffering, and managing it properly should be developed (Li et al., 2023).

• **Security issues:** Given the great potential of SNNs and their increasing use and development, it is expected that they will receive more attention in future cyber–physical systems, particularly in the Internet of Things (IoT). Therefore, security issues must be considered seriously and novel approaches must be assessed to mitigate side-channel attacks. Garaffa et al. (2021) demonstrated that information leakage can occur through the timing and power consumption of the system by implementing the Izhikevich neuron on FPGA, and that it is possible to extract internal system information and neuron weights through reverse engineering attacks.

• **Error tolerance:** It is commonly believed that SNNs are resistant to hardware faults that can occur during the manufacturing process, due to their similarity to the biological brain. However, recent research has shown that this assumption is not always true. These networks can still experience faults caused by radiation, aging, and other factors (Putra, Hanif, & Shafique, 2021; Spyrou et al., 2021). Moreover, standard error tolerance techniques commonly used in traditional computations, such as triple modular redundancy (TMR) and error-correction codes (ECC) for memories, are generally ineffective for hardware accelerators (Spyrou et al., 2022). Therefore, the performance-under-error scenarios needs to be further evaluated for cost-effective fault tolerance designs. Additionally, hardware-friendly fault tolerance techniques should be assessed and recommended for practical implementation. On the other hand, when designing large-scale neuromorphic systems, two important features should be considered: (1) the system should have online learning capability and, (2) the system should have fault tolerance (Yang, Wang et al., 2021). It is of great importance for a system to support both capabilities so as to adapt to error-prone environments.

• **Reinforcement learning:** One of the primary and crucial applications of SNNs is online learning and their use in real-life scenarios. Reinforcement learning is a key feature of these networks, requiring them to learn online through interaction with the environment. This aspect is particularly more prominent in applications related to robotics, autonomous driving and natural language processing (Lagani, Falchi, Gennaro, & Amato, 2023). Although implementing reinforcement learning online has its advantages, it can be costly for several reasons. Firstly, the process is time-consuming since the concurrency of events for learning rarely occurs. Secondly, the high cost of errors is a significant concern because mistakes can lead to damage to the system (Walravens et al., 2020). Therefore, using simulation to acquire knowledge and then applying it to the system is one of the possible solutions to mitigate these challenges. Nonetheless, the disparity between simulation and reality is a concern that causes fault. One solution is transfer learning (Ghaffari et al., 2023), but it is crucial to develop online learning systems that minimize the aforementioned challenges. This online learning feature distinguishes SNNs from traditional neural networks and makes them a powerful tool for making quick decisions in unpredictable and changing scenarios.

• **Healthcare portable devices:** Power consumption in portable devices is crucial, and this issue becomes even more critical in health monitoring devices, as these devices need to continuously monitor the condition of individuals or patients (Zompanti et al., 2021). SNNs have a lot of promise for patient adaptation and for disease detection and prediction, e.g., through continuous monitoring of EEG and ECG signals (Amirshahi & Hashemi, 2019; Luo et al., 2020). SNNs have the potential to offer lower energy consumption for disease detection and prediction in scenarios where continuous health monitoring is required. However, most of the work conducted so far faces the challenge of accuracy due to their shallow networks (Xing et al., 2022). As advances in FPGA technology progresses, larger-scale embedded SNNs will become possible such that the efficacy of these models also improves to a standard necessary for real-world portable deployment.

• **Compiler frameworks:** Implementing neural networks on FPGA is a more tedious task because frameworks such as Tensorflow, Cafe and Pytorch do not cover FPGA implementation, and it is much easier to work with them that run on CPUs and GPUs (Javanshir et al., 2022). However, BindsNET (Hazan et al., 2018) was introduced in recent years and can interface with hardware such as FPGA and ASIC to run on it, but needs to be further developed. On the other hand, high-level synthesis (HLS) tools have been introduced by FPGA development companies to facilitate working with them through software coding, but, compared to hardware description languages (HDL), they still lack precise control over all design components by the user (Cong et al., 2022; Gerlinghoff, Wang, Gu, Goh, & Luo, 2021).

FPGA compiler frameworks are vital for the widespread utilization of accelerators. They enable researchers and developers who may not be acquainted with hardware engineering to implement SNNs on FPGAs more easily (Chen, Moreau et al., 2018). On the other hand, hardware-implemented SNNs lag behind software simulation in terms of executing more complex operations and still performing simpler tasks. One reason is the lack of more advanced frameworks for FPGA SNN execution with more flexibility and the challenges of hardware description languages. Nevertheless, articles like Gerlinghoff et al. (2021), Gillet, Vincent, Le Gal, and Saïghi (2022) have strived to develop a software framework for simplicity in FPGA implementation through PyTorch, and also with PYNQ boards (Li, Wan et al., 2024; Stornaiuolo, Santambrogio, & Sciuto, 2018), it is possible to program an FPGA through the Python language. Despite promising results, the gap is still strongly felt, and further development is needed. A promising avenue is the development of the neuromorphic intermediate representation (NIR) which acts as an exchange format for common SNN libraries to be cross-compatible (Pedersen et al., 2023).

• **Limitations and advancements in commercial FPGAs:** Commercial FPGAs, despite offering unparalleled flexibility and reconfigurability, inherently possess limitations that can significantly impact the design and implementation of SNNs. SNNs, particularly those with intricate dynamics, often necessitate substantial resources for real-time processing. However, the finite number of available logic blocks (e.g., FFs, LUTs, and DSPs) and internal memory (BRAM) can constrain the complexity of neural network architectures, especially when considering deep or biologically accurate models. Furthermore, limitations in external memory bandwidth and input/output capabilities restrict the communication speed between processing units and memory, potentially hampering performance in large-scale SNNs and diminishing the network's parallelization capabilities (Tang & Han, 2023).

Notwithstanding these challenges, recent advancements in FPGA technology have addressed or mitigated many of these limitations. The advent of ultra-scale architectures, exemplified by Xilinx UltraScale+ and Intel Stratix 10, provides significantly enhanced logic resources, more accessible BRAMs, and more efficient DSP slices. These improvements facilitate the deployment of increasingly complex SNNs with deeper layers. Contemporary SoC FPGAs, such as the Xilinx Zynq family, integrate ARM processors, offering a hybrid platform that combines hardware acceleration with software flexibility. This heterogeneous architecture enables more efficient task partitioning, thereby reducing latency and enhancing power efficiency for SNNs.

Recent technological breakthroughs have also introduced adaptive compute acceleration platforms (ACAPs), like Xilinx Versal, which ingeniously combine FPGA logic with integrated CPUs and dedicated AI processing engines (Huang et al., 2024). Moreover, the incorporation of high bandwidth memory (HBM) in cutting-edge FPGA-based accelerators, such as Xilinx Alveo, substantially enhances memory access speeds—a crucial factor in neural computations (Kim & Park, 2024).

These technological advancements have opened new vistas for SNN implementation, enabling the creation of more sophisticated and efficient artificial neural systems. As a result, they are pushing the boundaries of brain-inspired computing, paving the way for groundbreaking applications in fields ranging from robotics and autonomous systems to advanced pattern recognition and cognitive computing.

6. Conclusion

Spiking neural networks, as the third generation of neural networks, have generated high expectations and hopes for low-power and high-speed computations due to their utilization of brain-inspired computational patterns. Mimicking the brain to achieve more efficient computations requires parallel processing, a task difficult to achieve in conventional processors which consume high power. FPGAs have a lot of utility for SNN acceleration, prototyping, and iterative testing, with its unique capabilities and support for parallel processing, and rapid design cycles. This paper presents an effort to synthesize the vast amount of literature on SNN acceleration and modeling on FPGAs. We have identified a series of challenges in the use of SNNs, and concluded by motivating several specific applications, followed by a discussion of upcoming challenges that warrant further investigation in the future.

CRediT authorship contribution statement

Mehrzad Karamimanesh: Writing – original draft, Methodology, Investigation, Conceptualization. **Ebrahim Abiri:** Writing – review & editing, Validation, Supervision. **Mahyar Shahsavari:** Writing – review & editing, Validation. **Kourosh Hassanli:** Writing – review & editing, Validation. **André van Schaik:** Writing – review & editing, Validation. **Jason Eshraghian:** Writing – review & editing, Validation, Methodology.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

References

Abbott, L. F., & Nelson, S. B. (2000). Synaptic plasticity: taming the beast. *Nature Neuroscience*, *3*(11), 1178–1183.

Abbott, L., & Regehr, W. G. (2004). Synaptic computation. *Nature*, *431*(7010), 796–803.

Afshar, S., Ralph, N., Xu, Y., Tapson, J., van Schaik, A., & Cohen, G. (2020). Event-based feature extraction using adaptive selection thresholds. *Sensors*, *20*(6), 1600.

Akopyan, F., Sawada, J., Cassidy, A., Alvarez-Icaza, R., Arthur, J., Merolla, P., et al. (2015). Truenorth: Design and tool flow of a 65 mw 1 million neuron programmable neurosynaptic chip. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, *34*(10), 1537–1557.

Ambroise, M., Levi, T., Bornat, Y., & Saighi, S. (2013). Biorealistic spiking neural network on FPGA. In *2013 47th annual conference on information sciences and systems* (pp. 1–6). IEEE.

Amirshahi, A., & Hashemi, M. (2019). ECG classification algorithm based on STDP and R-STDP neural networks for real-time monitoring on ultra low-power personal wearable devices. *IEEE Transactions on Biomedical Circuits and Systems*, *13*(6), 1483–1493.

An, H., Ehsan, M. A., Zhou, Z., & Yi, Y. (2017). Electrical modeling and analysis of 3D synaptic array using vertical RRAM structure. In *2017 18th international symposium on quality electronic design* (pp. 1–6). IEEE.

Arriandiaga, A., Portillo, E., Espinosa-Ramos, J. I., & Kasabov, N. K. (2019). Pulsewidth modulation-based algorithm for spike phase encoding and decoding of time-dependent analog data. *IEEE Transactions on Neural Networks and Learning Systems*, *31*(10), 3920–3931.

Asgari, H., Maybodi, B. M.-N., Payvand, M., & Azghadi, M. R. (2020). Low-energy and fast spiking neural network for context-dependent learning on FPGA. *IEEE Transactions on Circuits and Systems II: Express Briefs*, *67*(11), 2697–2701.

Aung, M. T. L., Qu, C., Yang, L., Luo, T., Goh, R. S. M., & Wong, W.-F. (2021). DeepFire: Acceleration of convolutional spiking neural network on modern field programmable gate arrays. In *2021 31st international conference on field-programmable logic and applications* (pp. 28–32). IEEE.

Azghadi, M. R., Al-Sarawi, S., Abbott, D., & Iannella, N. (2013). A neuromorphic VLSI design for spike timing and rate based synaptic plasticity. *Neural Networks*, *45*, 70–82.

Azghadi, M. R., Lammie, C., Eshraghian, J. K., Payvand, M., Donati, E., Linares-Barranco, B., et al. (2020). Hardware implementation of deep network accelerators towards healthcare and biomedical applications. *IEEE Transactions on Biomedical Circuits and Systems*, *14*(6), 1138–1159.

Bahrami, M. K., & Nazari, S. (2024). Digital design of a spatial-pow-STDP learning block with high accuracy utilizing pow CORDIC for large-scale image classifier spatiotemporal SNN. *Scientific Reports*, *14*(1), 3388.

Bal, M., & Sengupta, A. (2024). Spikingbert: Distilling bert to train spiking language models using implicit differentiation. *vol. 38*, In *Proceedings of the AAAI conference on artificial intelligence* (pp. 10998–11006).

Basu, A., Deng, L., Frenkel, C., & Zhang, X. (2022). Spiking neural network integrated circuits: A review of trends and future directions. In *2022 IEEE custom integrated circuits conference* (pp. 1–8). IEEE.

Bear, M. F., & Malenka, R. C. (1994). Synaptic plasticity: LTP and LTD. *Current Opinion in Neurobiology*, *4*(3), 389–399.

Bethi, Y., Xu, Y., Cohen, G., van Schaik, A., & Afshar, S. (2022). An optimized deep spiking neural network architecture without gradients. *IEEE Access*, *10*, 97912–97929.

Bi, G.-q., & Poo, M.-m. (1998). Synaptic modifications in cultured hippocampal neurons: dependence on spike timing, synaptic strength, and postsynaptic cell type. *Journal of Neuroscience*, *18*(24), 10464–10472.

Bianchi, S., Muñoz-Martin, I., & Ielmini, D. (2020). Bio-inspired techniques in a fully digital approach for lifelong learning. *Frontiers in Neuroscience*, *14*, 379.

Bing, Z., Jiang, Z., Cheng, L., Cai, C., Huang, K., & Knoll, A. (2019). End to end learning of a multi-layered snn based on r-stdp for a target tracking snake-like robot. In *2019 international conference on robotics and automation* (pp. 9645–9651). IEEE.

Bing, Z., Meschede, C., Huang, K., Chen, G., Rohrbein, F., Akl, M., et al. (2018). End to end learning of spiking neural network based on r-stdp for a lane keeping vehicle. In *2018 IEEE international conference on robotics and automation* (pp. 4725–4732). IEEE.

Blott, M., Preußer, T. B., Fraser, N. J., Gambardella, G., O'brien, K., Umuroglu, Y., et al. (2018). FINN-R: An end-to-end deep-learning framework for fast exploration of quantized neural networks. *ACM Transactions on Reconfigurable Technology and Systems (TRETS)*, *11*(3), 1–23.

Boahen, K. A. (2000). Point-to-point connectivity between neuromorphic chips using address events. *IEEE Transactions on Circuits and Systems II: Analog and Digital Signal Processing*, *47*(5), 416–434.

Bohte, S. M., Kok, J. N., & La Poutré, J. A. (2000). SpikeProp: backpropagation for networks of spiking neurons. *vol. 48*, In *ESANN* (pp. 419–424). Bruges.

Bohte, S. M., Kok, J. N., & La Poutre, H. (2002). Error-backpropagation in temporally encoded networks of spiking neurons. *Neurocomputing*, *48*(1–4), 17–37.

Bouvier, M., Valentian, A., Mesquida, T., Rummens, F., Reyboz, M., Vianello, E., et al. (2019). Spiking neural networks hardware implementations and challenges: A survey. *ACM Journal on Emerging Technologies in Computing Systems (JETC)*, *15*(2), 1–35.

Brader, J. M., Senn, W., & Fusi, S. (2007). Learning real-world stimuli in a neural network with spike-driven synaptic dynamics. *Neural Computation*, *19*(11), 2881–2912.

Brenowitz, S. D., & Regehr, W. G. (2005). Associative short-term synaptic plasticity mediated by endocannabinoids. *Neuron*, *45*(3), 419–431.

Brette, R., & Gerstner, W. (2005). Adaptive exponential integrate-and-fire model as an effective description of neuronal activity. *Journal of Neurophysiology*, *94*(5), 3637–3642.

Brochier, T., Zehl, L., Hao, Y., Duret, M., Sprenger, J., Denker, M., et al. (2018). Massively parallel recordings in macaque motor cortex during an instructed delayed reach-to-grasp task. *Scientific Data*, *5*(1), 1–23.

Canas-Moreno, S., Piñero-Fuentes, E., Rios-Navarro, A., Cascado-Caballero, D., Perez-Peña, F., & Linares-Barranco, A. (2023). Towards neuromorphic FPGA-based infrastructures for a robotic arm. *Autonomous Robots*, *47*(7), 947–961.

Cao, Y., Chen, Y., & Khosla, D. (2015). Spiking deep convolutional neural networks for energy-efficient object recognition. *International Journal of Computer Vision*, *113*, 54–66.

Capra, M., Bussolino, B., Marchisio, A., Masera, G., Martina, M., & Shafique, M. (2020). Hardware and software optimizations for accelerating deep neural networks: Survey of current trends, challenges, and the road ahead. *IEEE Access*, *8*, 225134–225180.

Carpegna, A., Savino, A., & Di Carlo, S. (2022). Spiker: an fpga-optimized hardware accelerator for spiking neural networks. In *2022 IEEE computer society annual symposium on VLSI* (pp. 14–19). IEEE.

Carpegna, A., Savino, A., & Di Carlo, S. (2024). Spiker+: a framework for the generation of efficient Spiking Neural Networks FPGA accelerators for inference at the edge. arXiv preprint arXiv:2401.01141.

Cassidy, A., & Andreou, A. G. (2008). Dynamical digital silicon neurons. In *2008 IEEE biomedical circuits and systems conference* (pp. 289–292). IEEE.

Cassidy, A. S., Georgiou, J., & Andreou, A. G. (2013). Design of silicon brains in the nano-CMOS era: Spiking neurons, learning synapses and neural architecture optimization. *Neural Networks*, *45*, 4–26.

Cattani, A., Einevoll, G. T., & Panzeri, S. (2015). Phase-of-firing code. arXiv preprint arXiv:1504.03954.

Chen, Q., Gao, C., Fang, X., & Luan, H. (2022). Skydiver: A spiking neural network accelerator exploiting spatio-temporal workload balance. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, *41*(12), 5732–5736.

Chen, Q., Gao, C., & Fu, Y. (2022). Cerebron: A reconfigurable architecture for spatiotemporal sparse spiking neural networks. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, *30*(10), 1425–1437.

Chen, G. K., Kumar, R., Sumbul, H. E., Knag, P. C., & Krishnamurthy, R. K. (2018). A 4096-neuron 1M-synapse 3.8-pJ/SOP spiking neural network with on-chip STDP learning and sparse weights in 10-nm FinFET CMOS. *IEEE Journal of Solid-State Circuits*, *54*(4), 992–1002.

Chen, Y., Liu, Y., Ye, W., & Chang, C.-C. (2023). The high-performance design of a general spiking convolution computation unit for supporting neuromorphic hardware acceleration. *IEEE Transactions on Circuits and Systems II: Express Briefs*, *70*(9), 3634–3638.

Chen, T., Moreau, T., Jiang, Z., Zheng, L., Yan, E., Shen, H., et al. (2018). {TVM}: An automated {End-to-End} optimizing compiler for deep learning. In *13th USeNIX symposium on operating systems design and implementation* (pp. 578–594).

Chen, L., Xiong, X., & Liu, J. (2022). A survey of intelligent chip design research based on spiking neural networks. *IEEE Access*, *10*, 89663–89686.

Chen, Y., Ye, W., Liu, Y., & Zhou, H. (2024). Sibrain: A sparse spatio-temporal parallel neuromorphic architecture for accelerating spiking convolution neural networks with low latency. *IEEE Transactions on Circuits and Systems. I. Regular Papers*.

Cheslet, J., Bernert, M., Beaubois, R., Yvert, B., & Lévi, T. (2024). Real-time spike sorting using an optimized STDP Spiking Neural Network on FPGA. In *2024 international joint conference on neural networks* (pp. 1–7). IEEE.

Chu, H., Yan, Y., Gan, L., Jia, H., Qian, L., Huan, Y., et al. (2022). A neuromorphic processing system with spike-driven SNN processor for wearable ECG classification. *IEEE Transactions on Biomedical Circuits and Systems*, *16*(4), 511–523.

Citri, A., & Malenka, R. C. (2008). Synaptic plasticity: multiple forms, functions, and mechanisms. *Neuropsychopharmacology*, *33*(1), 18–41.

Cong, J., Lau, J., Liu, G., Neuendorffer, S., Pan, P., Vissers, K., et al. (2022). FPGA HLS today: successes, challenges, and opportunities. *ACM Transactions on Reconfigurable Technology and Systems (TRETS)*, *15*(4), 1–42.

Corradi, F., Adriaans, G., & Stuijk, S. (2021). Gyro: A digital spiking neural network architecture for multi-sensory data analytics. In *Proceedings of the 2021 drone systems engineering and rapid simulation and performance evaluation: methods and tools proceedings* (pp. 9–15).

Corradi, F., Pande, S., Stuijt, J., Qiao, N., Schaafsma, S., Indiveri, G., et al. (2019). ECG-based heartbeat classification in neuromorphic hardware. In *2019 international joint conference on neural networks* (pp. 1–8). IEEE.

Cramer, B., Stradmann, Y., Schemmel, J., & Zenke, F. (2020). The heidelberg spiking data sets for the systematic evaluation of spiking neural networks. *IEEE Transactions on Neural Networks and Learning Systems*, *33*(7), 2744–2757.

Dabbagh, A., Karamimanesh, M., Hassanli, K., & Abiri, E. (2025). CD-MAC: mixed-signal binary/ternary in-memory computing accelerator for power-constrained MAC processing. *Journal of Supercomputing*, *81*(1), 190.

Davies, S., Gait, A., Rowley, A., & Di Nuovo, A. (2024). Supervised learning of spatial features with STDP and homeostasis using Spiking Neural Networks on SpiNNaker. *Neurocomputing*, Article 128650.

Davies, M., Srinivasa, N., Lin, T.-H., Chinya, G., Cao, Y., Choday, S. H., et al. (2018). Loihi: A neuromorphic manycore processor with on-chip learning. *Ieee Micro*, *38*(1), 82–99.

Deng, B., Fan, Y., Wang, J., & Yang, S. (2021). Reconstruction of a fully paralleled auditory spiking neural network and FPGA implementation. *IEEE Transactions on Biomedical Circuits and Systems*, *15*(6), 1320–1331.

Deng, B., Fan, Y., Wang, J., & Yang, S. (2023). Auditory perception architecture with spiking neural network and implementation on FPGA. *Neural Networks*, *165*, 31–42.

Diehl, P. U., & Cook, M. (2015). Unsupervised learning of digit recognition using spike-timing-dependent plasticity. *Frontiers in Computational Neuroscience*, *9*, 99.

Diehl, P. U., Neil, D., Binas, J., Cook, M., Liu, S.-C., & Pfeiffer, M. (2015). Fast-classifying, high-accuracy spiking deep networks through weight and threshold balancing. In *2015 international joint conference on neural networks* (pp. 1–8). ieee.

Ermentrout, G. B., & Kopell, N. (1986). Parabolic bursting in an excitable system coupled with a slow oscillation. *SIAM Journal on Applied Mathematics*, *46*(2), 233–253.

Eshraghian, J. K., Lammie, C., Azghadi, M. R., & Lu, W. D. (2022). Navigating local minima in quantized spiking neural networks. In *2022 IEEE 4th international conference on artificial intelligence circuits and systems* (pp. 352–355). IEEE.

Eshraghian, J. K., & Lu, W. D. (2022). The fine line between dead neurons and sparsity in binarized spiking neural networks. arXiv preprint arXiv:2201.11915.

Eshraghian, J. K., Wang, X., & Lu, W. D. (2022). Memristor-based binarized spiking neural networks: Challenges and applications. *IEEE Nanotechnology Magazine*, *16*(2), 14–23.

Eshraghian, J. K., Ward, M., Neftci, E. O., Wang, X., Lenz, G., Dwivedi, G., et al. (2023). Training spiking neural networks using lessons from deep learning. *Proceedings of the IEEE*.

Fang, H., Mei, Z., Shrestha, A., Zhao, Z., Li, Y., & Qiu, Q. (2020). Encoding, model, and architecture: Systematic optimization for spiking neural network in FPGAs. In *Proceedings of the 39th international conference on computer-aided design* (pp. 1–9).

Fang, H., Shrestha, A., Zhao, Z., Li, Y., & Qiu, Q. (2019). An event-driven neuromorphic system with biologically plausible temporal dynamics. In *2019 IEEE/ACM international conference on computer-aided design* (pp. 1–8). IEEE.

Fei-Fei, L., Fergus, R., & Perona, P. (2004). Learning generative visual models from few training examples: An incremental bayesian approach tested on 101 object categories. In *2004 conference on computer vision and pattern recognition workshop*. IEEE, 178–178.

Fisher, R. A. (1936). The use of multiple measurements in taxonomic problems. *Annals of Eugenics*, *7*(2), 179–188.

FitzHugh, R. (1961). Impulses and physiological states in theoretical models of nerve membrane. *Biophysical Journal*, *1*(6), 445–466.

Florian, R. V. (2012). The chronotron: A neuron that learns to fire temporally precise spike patterns.

Fourcaud-Trocmé, N., Hansel, D., Van Vreeswijk, C., & Brunel, N. (2003). How spike generation mechanisms determine the neuronal response to fluctuating inputs. *Journal of Neuroscience*, *23*(37), 11628–11640.

Frenkel, C., Bol, D., & Indiveri, G. (2023). Bottom-up and top-down approaches for the design of neuromorphic processing systems: tradeoffs and synergies between natural and artificial intelligence. *Proceedings of the IEEE*, *111*(6), 623–652.

Frenkel, C., & Indiveri, G. (2022). ReckOn: A 28nm sub-mm2 task-agnostic spiking recurrent neural network processor enabling on-chip learning over second-long timescales. *vol. 65*, In *2022 IEEE international solid-state circuits conference* (pp. 1–3). IEEE.

Frenkel, C., Lefebvre, M., Legat, J.-D., & Bol, D. (2018). A 0.086-mm^2 12.7-pJ/SOP 64k-synapse 256-neuron online-learning digital spiking neuromorphic processor in 28-nm CMOS. *IEEE Transactions on Biomedical Circuits and Systems*, *13*(1), 145–158.

Frenkel, C., Legat, J.-D., & Bol, D. (2019). Morphic: A 65-nm 738k-Synapse/mm^2 quad-core binary-weight digital neuromorphic processor with stochastic spike-driven online learning. *IEEE Transactions on Biomedical Circuits and Systems*, *13*(5), 999–1010.

Furber, S. B., Galluppi, F., Temple, S., & Plana, L. A. (2014). The spinnaker project. *Proceedings of the IEEE*, *102*(5), 652–665.

Garaffa, L. C., Aljuffri, A., Reinbrecht, C., Hamdioui, S., Taouil, M., & Sepulveda, J. (2021). Revealing the secrets of spiking neural networks: The case of izhikevich neuron. In *2021 24th euromicro conference on digital system design* (pp. 514–518). IEEE.

Garofolo, J. S. (1993). *Timit acoustic phonetic continuous speech corpus*. Linguistic Data Consortium, 1993.

Gebhardt, W., & Ororbia, A. G. (2024). Time-integrated spike-timing-dependent-plasticity. arXiv preprint arXiv:2407.10028.

George, J. K., Soci, C., Miscuglio, M., & Sorger, V. J. (2021). Symmetry perception with spiking neural networks. *Scientific Reports*, *11*(1), 5776.

Gerlinghoff, D., Wang, Z., Gu, X., Goh, R. S. M., & Luo, T. (2021). E3NE: An end-to-end framework for accelerating spiking neural networks with emerging neural encoding on FPGAs. *IEEE Transactions on Parallel and Distributed Systems*, *33*(11), 3207–3219.

Gerlinghoff, D., Wang, Z., Gu, X., Goh, R. S. M., & Luo, T. (2022). A resource-efficient spiking neural network accelerator supporting emerging neural encoding. In *2022 design, automation & test in europe conference & exhibition* (pp. 92–95). IEEE.

Gerstner, W., Kistler, W. M., Naud, R., & Paninski, L. (2014). *Neuronal dynamics: From single neurons to networks and models of cognition*. Cambridge University Press.

Ghaffari, R., Helfroush, M. S., Khosravi, A., Kazemi, K., Danyali, H., & Rutkowski, L. (2023). Towards domain adaptation with open-set target data: Review of theory and computer vision applications. *Information Fusion*, Article 101912.

Ghanbarpour, M., Naderi, A., Ghanbari, B., Haghiri, S., & Ahmadi, A. (2023). Digital hardware implementation of Morris-Lecar, Izhikevich, and Hodgkin-Huxley neuron models with high accuracy and low resources. *IEEE Transactions on Circuits and Systems. I. Regular Papers*.

Ghosh-Dastidar, S., & Adeli, H. (2009). Spiking neural networks. *International Journal of Neural Systems*, *19*(04), 295–308.

Gillet, C., Vincent, A. F., Le Gal, B., & Saïghi, S. (2022). Flexible design methodology for spike encoding implementation on FPGA. In *2022 IEEE biomedical circuits and systems conference* (pp. 379–383). IEEE.

Gomar, S., & Ahmadi, M. (2018). Digital realization of PSTDP and TSTDP learning. In *2018 international joint conference on neural networks* (pp. 1–5). IEEE.

Gonzalez, H. A., Huang, J., Kelber, F., Nazeer, K. K., Langer, T., Liu, C., et al. (2024). Spinnaker2: A large-scale neuromorphic system for event-based and asynchronous machine learning. arXiv preprint arXiv:2401.04491.

Graupner, M., & Brunel, N. (2012). Calcium-based plasticity model explains sensitivity of synaptic changes to spike pattern, rate, and dendritic location. *Proceedings of the National Academy of Sciences*, *109*(10), 3991–3996.

Gunasekaran, S., Eshraghian, J., Zhu, R., & Kuncic, Z. (0000). Knowledge distillation through time for future event prediction. In: *The second tiny papers track at ICLR 2024*. (n.d.).

Guo, W., Yantır, H. E., Fouda, M. E., Eltawil, A. M., & Salama, K. N. (2021). Toward the optimal design and FPGA implementation of spiking neural networks. *IEEE Transactions on Neural Networks and Learning Systems*, *33*(8), 3988–4002.

Gupta, S., Vyas, A., & Trivedi, G. (2020). FPGA implementation of simplified spiking neural network. In *2020 27th IEEE international conference on electronics, circuits and systems* (pp. 1–4). IEEE.

Gutierrez-Galan, D., Dominguez-Morales, J. P., Perez-Peña, F., Jimenez-Fernandez, A., & Linares-Barranco, A. (2020). NeuroPod: a real-time neuromorphic spiking CPG applied to robotics. *Neurocomputing*, *381*, 10–19.

Gütig, R., & Sompolinsky, H. (2006). The tempotron: a neuron that learns spike timing–based decisions. *Nature Neuroscience*, *9*(3), 420–428.

Hamdan, M. K. (2018). *VHDL auto-generation tool for optimized hardware acceleration of convolutional neural networks on FPGA (VGT)* (Unpublished doctoral dissertation), Iowa State University.

Han, J., Li, Z., Zheng, W., & Zhang, Y. (2020). Hardware implementation of spiking neural networks on FPGA. *Tsinghua Science and Technology*, *25*(4), 479–486.

Hao, X., Wang, J., Yang, S., Deng, B., Wei, X., & Yi, G. (2019). Real-time implementation of the cerebellum neural network. In *2019 Chinese control and decision conference* (pp. 3595–3599). IEEE.

Hazan, H., Saunders, D. J., Khan, H., Patel, D., Sanghavi, D. T., Siegelmann, H. T., et al. (2018). Bindsnet: A machine learning-oriented spiking neural networks library in python. *Frontiers in Neuroinformatics*, *12*, 89.

He, Z., Shi, C., Wang, T., Wang, Y., Tian, M., Zhou, X., et al. (2021). A low-cost FPGA implementation of spiking extreme learning machine with on-chip reward-modulated STDP learning. *IEEE Transactions on Circuits and Systems II: Express Briefs*, *69*(3), 1657–1661.

Hebb, D. O. (2005). *The organization of behavior: A neuropsychological theory*. Psychology Press.

Heeger, D., et al. (2000). Poisson model of spike generation. *Handout, University of Standford*, *5*(1–13), 76.

Heidarpour, M., Ahmadi, A., & Rashidzadeh, R. (2016). A CORDIC based digital hardware for adaptive exponential integrate and fire neuron. *IEEE Transactions on Circuits and Systems. I. Regular Papers*, *63*(11), 1986–1996.

Heidarpur, M., Ahmadi, A., & Ahmadi, M. (2024). The silence of the neurons: an application to enhance performance and energy efficiency. *Frontiers in Neuroscience*, *17*, Article 1333238.

Heidarpur, M., Ahmadi, A., Ahmadi, M., & Azghadi, M. R. (2019). CORDIC-SNN: On-FPGA STDP learning with izhikevich neurons. *IEEE Transactions on Circuits and Systems. I. Regular Papers*, *66*(7), 2651–2661.

Hodgkin, A. L., & Huxley, A. F. (1952). A quantitative description of membrane current and its application to conduction and excitation in nerve. *The Journal of Physiology*, *117*(4), 500.

Hu, Y., & Pfeiffer, M. (2016). Max-pooling operations in deep spiking neural networks. In *Neural syst. comput. project rep*.

Hu, S., Qiao, G., Chen, T., Yu, Q., Liu, Y., & Rong, L. (2021). Quantized STDP-based online-learning spiking neural network. *Neural Computing and Applications*, *33*(19), 12317–12332.

Huang, X., Jones, E., Zhang, S., Xie, S., Furber, S., Goulermas, Y., et al. (2020). Spiking neural network based low-power radioisotope identification using fpga. In *2020 27th IEEE international conference on electronics, circuits and systems* (pp. 1–4). IEEE.

Huang, X., Jones, E., Zhang, S., Xie, S., Furber, S., Goulermas, Y., et al. (2021). An FPGA implementation of convolutional spiking neural networks for radioisotope identification. In *2021 IEEE international symposium on circuits and systems* (pp. 1–5). IEEE.

Huang, B.-Y., Lyubomirsky, S., Li, Y., He, M., Smith, G. H., Tambe, T., et al. (2024). Application-level validation of accelerator designs using a formal software/hardware interface. *ACM Transactions on Design Automation of Electronic Systems*, *29*(2), 1–25.

Humaidi, A. J., Kadhim, T. M., Hasan, S., Ibraheem, I. K., & Azar, A. T. (2020). A generic izhikevich-modelled FPGA-realized architecture: a case study of printed english letter recognition. In *2020 24th international conference on system theory, control and computing* (pp. 825–830). IEEE.

Hunsberger, E., & Eliasmith, C. (2016). Training spiking deep networks for neuromorphic hardware. arXiv preprint arXiv:1611.05141.

Huynh, P. K., Varshika, M. L., Paul, A., Isik, M., Balaji, A., & Das, A. (2022). Implementing spiking neural networks on neuromorphic architectures: A review. arXiv preprint arXiv:2202.08897.

ICNS (2024). DeepSouth. Retrieved from https://www.deepsouth.org.au.

Irmak, H., Corradi, F., Detterer, P., Alachiotis, N., & Ziener, D. (2021). A dynamic reconfigurable architecture for hybrid spiking and convolutional fpga-based neural network designs. *Journal of Low Power Electronics and Applications*, *11*(3), 32.

Izhikevich, E. M. (2003). Simple model of spiking neurons. *IEEE Transactions on Neural Networks*, *14*(6), 1569–1572.

Izhikevich, E. M. (2007). Solving the distal reward problem through linkage of STDP and dopamine signaling. *Cerebral Cortex*, *17*(10), 2443–2452.

Jahanirad, H. (2023). Dynamic power-gating for leakage power reduction in FPGAs. *Frontiers of Information Technology & Electronic Engineering*, *24*(4), 582–598.

Javanshir, A., Nguyen, T. T., Mahmud, M. P., & Kouzani, A. Z. (2022). Advancements in algorithms and neuromorphic hardware for spiking neural networks. *Neural Computation*, *34*(6), 1289–1328.

Javed, A., Harkin, J., McDaid, L., & Liu, J. (2021). Spiking Neural Network-based structural health monitoring hardware system. In *2021 IEEE symposium series on computational intelligence* (pp. 01–07). IEEE.

Jiang, Y., Lun, L., Wang, J., Yin, M., Liu, H., Dai, Z., et al. (2024). SPAT: FPGA-based sparsity-optimized Spiking Neural Network training accelerator with temporal parallel dataflow. In *2024 IEEE international symposium on circuits and systems* (pp. 1–5). IEEE.

Jokar, E., Abolfathi, H., & Ahmadi, A. (2019). A novel nonlinear function evaluation approach for efficient FPGA mapping of neuron and synaptic plasticity models. *IEEE Transactions on Biomedical Circuits and Systems*, *13*(2), 454–469.

Ju, X., Fang, B., Yan, R., Xu, X., & Tang, H. (2020). An FPGA implementation of deep spiking neural networks for low-power and fast classification. *Neural Computation*, *32*(1), 182–204.

Kasabov, N. K. (2014). NeuCube: A spiking neural network architecture for mapping, learning and understanding of spatio-temporal brain data. *Neural Networks*, *52*, 62–76.

Kasabov, N., Scott, N. M., Tu, E., Marks, S., Sengupta, N., Capecci, E., et al. (2016). Evolving spatio-temporal data machines based on the NeuCube neuromorphic framework: Design methodology and selected applications. *Neural Networks*, *78*, 1–14.

Kasinski, A., & Ponulak, F. (2005). Experimental demonstration of learning properties of a new supervised learning method for the spiking neural networks. In *International conference on artificial neural networks* (pp. 145–152). Springer.

Khatami, F., & Escabí, M. A. (2020). Spiking network optimized for word recognition in noise predicts auditory system hierarchy. *PLoS Computational Biology*, *16*(6), Article e1007558.

Kheradpisheh, S. R., Ganjtabesh, M., Thorpe, S. J., & Masquelier, T. (2018). STDP-based spiking deep convolutional neural networks for object recognition. *Neural Networks*, *99*, 56–67.

Khodamoradi, A., Denolf, K., & Kastner, R. (2021). S2n2: A fpga accelerator for streaming spiking neural networks. In *The 2021 ACM/SIGDA international symposium on field-programmable gate arrays* (pp. 194–205).

Kim, K., & Park, M.-j. (2024). Present and future, challenges of High Bandwith Memory (HBM). In *2024 IEEE international memory workshop* (pp. 1–4). IEEE.

Kim, Y., Park, H., Moitra, A., Bhattacharjee, A., Venkatesha, Y., & Panda, P. (2022). Rate coding or direct coding: Which one is better for accurate, robust, and energy-efficient spiking neural networks? In *ICASSP 2022-2022 IEEE international conference on acoustics, speech and signal processing* (pp. 71–75). IEEE.

Kotiyal, S., Thapliyal, H., & Ranganathan, N. (2011). Design of a reversible bidirectional barrel shifter. In *2011 11th IEEE international conference on nanotechnology* (pp. 463–468). IEEE.

Lagani, G., Falchi, F., Gennaro, C., & Amato, G. (2023). Spiking neural networks and bio-inspired supervised deep learning: A survey. arXiv preprint arXiv:2307.16235.

Lambora, A., Gupta, K., & Chopra, K. (2019). Genetic algorithm-a literature review. In *2019 international conference on machine learning, big data, cloud and parallel computing* (pp. 380–384). IEEE.

Lammie, C., Hamilton, T. J., van Schaik, A., & Azghadi, M. R. (2018). Efficient FPGA implementations of pair and triplet-based STDP for neuromorphic architectures. *IEEE Transactions on Circuits and Systems. I. Regular Papers*, *66*(4), 1558–1570.

LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, *86*(11), 2278–2324.

Leigh, A. J., Mirhassani, M., & Muscedere, R. (2020). An efficient spiking neuron hardware system based on the hardware-oriented modified Izhikevich neuron (HOMIN) model. *IEEE Transactions on Circuits and Systems II: Express Briefs*, *67*(12), 3377–3381.

Lemaire, E., Moretti, M., Daniel, L., Miramond, B., Millet, P., Feresin, F., et al. (2020). An FPGA-based hybrid neural network accelerator for embedded satellite image classification. In *2020 IEEE international symposium on circuits and systems* (pp. 1–5). IEEE.

Lent, R. (2020). Evaluating the cognitive network controller with an SNN on FPGA. In *2020 IEEE international conference on wireless for space and extreme environments* (pp. 106–111). IEEE.

Lent, R. (2021). Towards cognitive networking using FPGA-based acceleration. *IEEE Journal of Radio Frequency Identification*, *5*(3), 222–231.

Leone, G., Raffo, L., & Meloni, P. (2022). A bandwidth-efficient emulator of biologically-relevant spiking neural networks on FPGA. *IEEE Access*, *10*, 76780–76793.

Leone, G., Raffo, L., & Meloni, P. (2023). On-FPGA spiking neural networks for end-to-end neural decoding. *IEEE Access*.

Leone, G., Scrugli, M. A., Badas, L., Martis, L., Raffo, L., & Meloni, P. (2024). Syntzulu: A tiny risc-v-Controlled SNN processor for real-time sensor data analysis on low-power FPGAs. *IEEE Transactions on Circuits and Systems. I. Regular Papers*.

Li, T., Li, J., Shen, G., Zhao, D., Zhang, Q., & Zeng, Y. (2024). FireFly-S: Exploiting dual-side sparsity for spiking neural networks acceleration with reconfigurable spatial architecture. arXiv preprint arXiv:2408.15578.

Li, J., Shen, G., Zhao, D., Zhang, Q., & Zeng, Y. (2023). Firefly: A high-throughput hardware accelerator for spiking neural networks with efficient dsp and memory optimization. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, *31*(8), 1178–1191.

Li, H., Wan, B., Fang, Y., Li, Q., Liu, J. K., & An, L. (2024). An FPGA implementation of Bayesian inference with spiking neural networks. *Frontiers in Neuroscience*, *17*, Article 1291051.

Liang, M., Zhang, J., & Chen, H. (2021). A 1.13 μJ/classification spiking neural network accelerator with a single-spike neuron model and sparse weights. In *2021 IEEE international symposium on circuits and systems* (pp. 1–5). IEEE.

Lillicrap, T. P., Cownden, D., Tweed, D. B., & Akerman, C. J. (2016). Random synaptic feedback weights support error backpropagation for deep learning. *Nature Communications*, *7*(1), 13276.

Lin, X., Lu, H., Pi, X., & Wang, X. (2020). An FPGA-based implementation method for quadratic spiking neuron model. In *2020 11th IEEE annual ubiquitous computing, electronics & mobile communication conference* (pp. 0621–0627). IEEE.

Linares-Barranco, A., Oster, M., Cascado, D., Jiménez, G., Civit, A., & Linares-Barranco, B. (2007). Inter-spike-intervals analysis of AER Poisson-like generator hardware. *Neurocomputing*, *70*(16–18), 2692–2700.

Linares-Barranco, A., Prono, L., Lengenstein, R., Indiveri, G., & Frenkel, C. (2024). Training and inference in the ReckON RSNN architecture implemented on a MPSoC. arXiv preprint arXiv:2405.12849.

Liu, Y., Chen, R., Li, S., Yang, J., Li, S., & Silva, B. d. (2024). FPGA-based sparse matrix multiplication accelerators: From state-of-the-art to future opportunities. *ACM Transactions on Reconfigurable Technology and Systems*.

Liu, Y., Chen, Y., Ye, W., & Gui, Y. (2022). FPGA-NHAP: A general FPGA-based neuromorphic hardware acceleration platform with high speed and low power. *IEEE Transactions on Circuits and Systems. I. Regular Papers*, *69*(6), 2553–2566.

Liu, H., Cui, X., Zhang, S., Yin, M., Jiang, Y., & Cui, X. (2024). A convolutional Spiking Neural Network accelerator with the sparsity-aware memory and compressed weights. In *2024 IEEE 35th international conference on application-specific systems, architectures and processors* (pp. 163–171). IEEE.

Liu, J., McDaid, L. J., Harkin, J., Karim, S., Johnson, A. P., Millard, A. G., et al. (2018). Exploring self-repair in a coupled spiking astrocyte neural network. *IEEE Transactions on Neural Networks and Learning Systems*, *30*(3), 865–875.

Luo, Y., Fu, Q., Xie, J., Qin, Y., Wu, G., Liu, J., et al. (2020). EEG-based emotion classification using spiking neural networks. *IEEE Access*, *8*, 46007–46016.

Lynch, G. S., Dunwiddie, T., & Gribkoff, V. (1977). Heterosynaptic depression: a postsynaptic correlate of long-term potentiation. *Nature*, *266*(5604), 737–739.

Ma, Y., Cao, Y., Vrudhula, S., & Seo, J.-s. (2017). An automatic RTL compiler for high-throughput FPGA implementation of diverse deep convolutional neural networks. In *2017 27th international conference on field programmable logic and applications* (pp. 1–8). IEEE.

Maass, W. (1997). Networks of spiking neurons: the third generation of neural network models. *Neural Networks*, *10*(9), 1659–1671.

Masquelier, T., Guyonneau, R., & Thorpe, S. J. (2009). Competitive STDP-based spike pattern learning. *Neural Computation*, *21*(5), 1259–1276.

Masquelier, T., & Thorpe, S. J. (2007). Unsupervised learning of visual features through spike timing dependent plasticity. *PLoS Computational Biology*, *3*(2), Article e31.

Meher, P. K., Valls, J., Juang, T.-B., Sridharan, K., & Maharatna, K. (2009). 50 years of CORDIC: Algorithms, architectures, and applications. *IEEE Transactions on Circuits and Systems. I. Regular Papers*, *56*(9), 1893–1907.

Mehrabi, A., Bethi, Y., van Schaik, A., & Afshar, S. (2023). An optimized multi-layer Spiking Neural Network implementation in FPGA without multipliers. *Procedia Computer Science*, *222*, 407–414.

Mehrabi, A., Bethi, Y., van Schaik, A., Wabnitz, A., & Afshar, S. (2023). Efficient implementation of a multi-layer gradient-free online-trainable Spiking Neural Network on FPGA. arXiv preprint arXiv:2305.19468.

Mitchell, J. P., Schuman, C. D., & Potok, T. E. (2020). A small, low cost event-driven architecture for spiking neural networks on fpgas. In *International conference on neuromorphic systems 2020* (pp. 1–4).

Mohaidat, T., & Khalil, K. (2024). A survey on neural network hardware accelerators. *IEEE Transactions on Artificial Intelligence*.

Moody, G. B., & Mark, R. G. (2001). The impact of the MIT-BIH arrhythmia database. *IEEE Engineering in Medicine and Biology Magazine*, *20*(3), 45–50.

Neftci, E. O., Mostafa, H., & Zenke, F. (2019). Surrogate gradient learning in spiking neural networks: Bringing the power of gradient-based optimization to spiking neural networks. *IEEE Signal Processing Magazine*, *36*(6), 51–63.

Netzer, Y., Wang, T., Coates, A., Bissacco, A., Wu, B., Ng, A. Y., et al. (2011). Reading digits in natural images with unsupervised feature learning. *vol. 2011*, In *NIPS workshop on deep learning and unsupervised feature learning* (p. 4). Granada.

Nguyen, D.-A., Tran, X.-T., & Iacopi, F. (2021). A review of algorithms and hardware implementations for spiking neural networks. *Journal of Low Power Electronics and Applications*, *11*(2), 23.

Nitzsche, S., Pachideh, B., Luhn, N., & Becker, J. (2021). Digital hardware implementation of optimized spiking neurons. In *2021 international conference on neuromorphic computing* (pp. 126–134). IEEE.

Ottati, F., Gao, C., Chen, Q., Brignone, G., Casu, M. R., Eshraghian, J. K., et al. (2023). To spike or not to spike: A digital hardware perspective on deep learning acceleration. *IEEE Journal on Emerging and Selected Topics in Circuits and Systems*.

Pani, D., Meloni, P., Tuveri, G., Palumbo, F., Massobrio, P., & Raffo, L. (2017). An FPGA platform for real-time simulation of spiking neuronal networks. *Frontiers in Neuroscience*, *11*, 90.

Paugam-Moisy, H., & Bohte, S. M. (2012). Computing with spiking neuron networks. *vol. 1*, In *Handbook of natural computing* (pp. 1–47).

Pedersen, J. E., Abreu, S., Jobst, M., Lenz, G., Fra, V., Bauer, F. C., et al. (2023). Neuromorphic intermediate representation: a unified instruction set for interoperable brain-inspired computing. arXiv preprint arXiv:2311.14641.

Pfister, J.-P., & Gerstner, W. (2006). Triplets of spikes in a model of spike timing-dependent plasticity. *Journal of Neuroscience*, *26*(38), 9673–9682.

Pham, Q. T., Nguyen, T. Q., Hoang, P. C., Dang, Q. H., Nguyen, D. M., & Nguyen, H. H. (2021). A review of SNN implementation on FPGA. In *2021 international conference on multimedia analysis and pattern recognition* (pp. 1–6). IEEE.

Pi, X., & Lin, X. (2022). An FPGA-based piecewise linear spiking neuron for simulating bursting behavior. In *2022 IEEE 12th annual computing and communication workshop and conference* (pp. 0415–0420). IEEE.

Pietrzak, P., Szczesny, S., Huderek, D., & Przyborowski, L. (2023). Overview of Spiking Neural Network learning approaches and their computational complexities. *Sensors*, *23*(6), 3037.

Ponulak, F., & Kasiński, A. (2010). Supervised learning in spiking neural networks with resume: sequence learning, classification, and spike shifting. *Neural Computation*, *22*(2), 467–510.

Putra, R. V. W., Hanif, M. A., & Shafique, M. (2021). Respawn: Energy-efficient fault-tolerance for spiking neural networks considering unreliable memories. In *2021 IEEE/ACM international conference on computer aided design* (pp. 1–9). IEEE.

Qiao, G., Hu, S., Chen, T., Rong, L., Ning, N., Yu, Q., et al. (2020). Stbnn: Hardware-friendly spatio-temporal binary neural network with high pattern recognition accuracy. *Neurocomputing*, *409*, 351–360.

Quintana, F. M., Perez-Pena, F., & Galindo, P. L. (2022). Bio-plausible digital implementation of a reward modulated STDP synapse. *Neural Computing and Applications*, *34*(18), 15649–15660.

Rathi, N., & Roy, K. (2020). Diet-snn: Direct input encoding with leakage and threshold optimization in deep spiking neural networks. arXiv preprint arXiv:2008.03658.

Rose, R., & Hindmarsh, J. (1989). The assembly of ionic currents in a thalamic neuron I. The three-dimensional model. *Proceedings of the Royal Society of London. B. Biological Sciences*, *237*(1288), 267–288.

Roy, K., Jaiswal, A., & Panda, P. (2019). Towards spike-based machine intelligence with neuromorphic computing. *Nature*, *575*(7784), 607–617.

Rueckauer, B., Lungu, I.-A., Hu, Y., Pfeiffer, M., & Liu, S.-C. (2017). Conversion of continuous-valued deep networks to efficient event-driven networks for image classification. *Frontiers in Neuroscience*, *11*, 682.

Rumbell, T., Denham, S. L., & Wennekers, T. (2013). A spiking self-organizing map combining STDP, oscillations, and continuous learning. *IEEE Transactions on Neural Networks and Learning Systems*, *25*(5), 894–907.

Rumsey, C. C., & Abbott, L. (2004). Synaptic equalization by anti-STDP. *Neurocomputing*, *58*, 359–364.

Saha, D., Leong, K., Li, C., Peterson, S., Siegel, G., & Raman, B. (2013). A spatiotemporal coding mechanism for background-invariant odor recognition. *Nature Neuroscience*, *16*(12), 1830–1839.

Scherer, D., Müller, A., & Behnke, S. (2010). Evaluation of pooling operations in convolutional architectures for object recognition. In *International conference on artificial neural networks* (pp. 92–101). Springer.

Schmidgall, S., Ziaei, R., Achterberg, J., Kirsch, L., Hajiseyedrazi, S., & Eshraghian, J. (2024). Brain-inspired learning in artificial neural networks: a review. *APL Machine Learning*, *2*(2).

Schrauwen, B., & Van Campenhout, J. (2003). BSA, a fast and accurate spike train encoding scheme. *vol. 4*, In *Proceedings of the international joint conference on neural networks, 2003* (pp. 2825–2830). IEEE.

Sengupta, N., Scott, N., & Kasabov, N. (2015). Framework for knowledge driven optimisation based data encoding for brain data modelling using spiking neural network architecture. In *Proceedings of the fifth international conference on fuzzy and neuro computing* (pp. 109–118). Springer.

Sengupta, A., Ye, Y., Wang, R., Liu, C., & Roy, K. (2019). Going deeper in spiking neural networks: VGG and residual architectures. *Frontiers in Neuroscience*, *13*, 95.

Shahsavari, M., Beaumont, J., Thomas, D., & Brown, A. (2020). Poets: A parallel cluster architecture for Spiking Neural Network. *International Journal of Machine Learning and Computing*.

Shahsavari, M., Devienne, P., & Boulet, P. (2019). Spiking neural computing in memristive neuromorphic platforms. In *Handbook of memristor networks* (pp. 691–728). Springer.

Shahsavari, M., Faisal Nadeem, M., Arash Ostadzadeh, S., Devienne, P., & Boulet, P. (2015). Unconventional digital computing approach: memristive nanodevice platform. *Physica Status Solidi (C)*, *12*(1–2), 222–228.

Shahsavari, M., Thomas, D., Brown, A., & Luk, W. (2021). Neuromorphic design using reward-based STDP learning on event-based reconfigurable cluster architecture. In *International conference on neuromorphic systems 2021* (pp. 1–8).

Shahsavari, M., Thomas, D., van Gerven, M., Brown, A., & Luk, W. (2023). Advancements in spiking neural network communication and synchronization techniques for event-driven neuromorphic systems. *Array*, *20*, Article 100323.

Shatz, C. J. (1992). The developing brain. *Scientific American*, *267*(3), 60–67.

Shouval, H. Z., Wang, S. S.-H., & Wittenberg, G. M. (2010). Spike timing dependent plasticity: a consequence of more fundamental learning rules. *Frontiers in Computational Neuroscience*, *4*, 19.

Shrestha, S. B., & Orchard, G. (2018). Slayer: Spike layer error reassignment in time. *Advances in Neural Information Processing Systems*, *31*.

Shwartz Ziv, R., & LeCun, Y. (2024). To compress or not to compress—self-supervised learning and information theory: A review. *Entropy*, *26*(3), 252.

Sjostrom, P. J., Rancz, E. A., Roth, A., & Hausser, M. (2008). Dendritic excitability and synaptic plasticity. *Physiological Reviews*, *88*(2), 769–840.

Soleimani, H., Ahmadi, A., & Bavandpour, M. (2012). Biologically inspired spiking neurons: Piecewise linear models and digital implementation. *IEEE Transactions on Circuits and Systems. I. Regular Papers*, *59*(12), 2991–3004.

Sommer, J., Özkan, M. A., Keszocze, O., & Teich, J. (2022). Efficient hardware acceleration of sparsely active convolutional spiking neural networks. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, *41*(11), 3767–3778.

Spyrou, T., El-Sayed, S. A., Afacan, E., Camuñas-Mesa, L. A., Linares-Barranco, B., & Stratigopoulos, H.-G. (2021). Neuron fault tolerance in spiking neural networks. In *2021 design, automation & test in europe conference & exhibition* (pp. 743–748). IEEE.

Spyrou, T., El-Sayed, S. A., Afacan, E., Camuñas-Mesa, L. A., Linares-Barranco, B., & Stratigopoulos, H.-G. (2022). Reliability analysis of a spiking neural network hardware accelerator. In *2022 design, automation & test in europe conference & exhibition* (pp. 370–375). IEEE.

Stornaiuolo, L., Santambrogio, M., & Sciuto, D. (2018). On how to efficiently implement deep learning algorithms on pynq platform. In *2018 IEEE computer society annual symposium on VLSI* (pp. 587–590). IEEE.

Sun, P.-S. V., Titterton, A., Gopiani, A., Santos, T., Basu, A., Lu, W. D., et al. (2024). Exploiting deep learning accelerators for neuromorphic workloads. *Neuromorphic Computing and Engineering*, *4*(1), Article 014004.

Sun, Y., Zeng, Y., Zhao, F., & Zhao, Z. (2023). Multi-compartment neuron and population encoding improved spiking neural network for deep distributional reinforcement learning. arXiv preprint arXiv:2301.07275.

Tang, C., & Han, J. (2023). Hardware efficient weight-binarized spiking neural networks. In *2023 design, automation & test in europe conference & exhibition* (pp. 1–6). IEEE.

Tang, G., Kumar, N., Yoo, R., & Michmizos, K. (2021). Deep reinforcement learning with population-coded spiking neural network for continuous control. In *Conference on robot learning* (pp. 2016–2029). PMLR.

Tapiador-Morales, R., Linares-Barranco, A., Jimenez-Fernandez, A., & Jimenez-Moreno, G. (2018). Neuromorphic LIF row-by-row multiconvolution processor for FPGA. *IEEE Transactions on Biomedical Circuits and Systems*, *13*(1), 159–169.

Tavanaei, A., & Maida, A. (2019). BP-STDP: Approximating backpropagation using spike timing dependent plasticity. *Neurocomputing*, *330*, 39–47.

Thorpe, S., Delorme, A., & Van Rullen, R. (2001). Spike-based strategies for rapid processing. *Neural Networks*, *14*(6–7), 715–725.

Udeji, U. L., & Margala, M. (2024). Spiking-transformer optimization on FPGA for image classification and captioning. In *SoutheastCon 2024* (pp. 1353–1357). IEEE.

Van Rullen, R., & Thorpe, S. J. (2001). Rate coding versus temporal order coding: what the retinal ganglion cells tell the visual cortex. *Neural Computation*, *13*(6), 1255–1283.

Veeravalli, B., Fong, X., et al. (2022). An FPGA-based co-processor for spiking neural networks with on-chip stdp-based learning. In *2022 IEEE international symposium on circuits and systems* (pp. 2157–2161). IEEE.

Venkatesh, S., Marinescu, R., & Eshraghian, J. K. (2024). SQUAT: Stateful quantization-aware training in recurrent spiking neural networks. In *2024 neuro inspired computational elements conference* (pp. 1–10). IEEE.

Vishwamith, H., Isik, M., & Dikmen, I. C. (2024). HPCNeuroNet: Advancing neuromorphic audio signal processing with transformer-enhanced spiking neural networks. In *2024 4th interdisciplinary conference on electrics and computer* (pp. 1–7). IEEE.

Volder, J. E. (1959). The CORDIC trigonometric computing technique. *IRE Transactions on Electronic Computers*, (3), 330–334.

Walravens, M., Verreyken, E., & Steckel, J. (2020). Spiking neural network implementation on fpga for robotic behaviour. In *Advances on p2P, parallel, grid, cloud and internet computing: proceedings of the 14th international conference on p2P, parallel, grid, cloud and internet computing (3PGCIC-2019) 14* (pp. 694–703). Springer.

Wang, Z., Gu, X., Goh, R. S. M., Zhou, J. T., & Luo, T. (2022). Efficient spiking neural networks with radix encoding. *IEEE Transactions on Neural Networks and Learning Systems*, *35*(3), 3689–3701.

Wang, Z., Guo, L., & Adjouadi, M. (2016). Wavelet decomposition and phase encoding of temporal signals using spiking neurons. *Neurocomputing*, *173*, 1203–1210.

Wang, K., Hao, X., Wang, J., & Deng, B. (2023). Comparison and selection of spike encoding algorithms for SNN on FPGA. *IEEE Transactions on Biomedical Circuits and Systems*, *17*(1), 129–141.

Wang, H., He, Z., Wang, T., He, J., Zhou, X., Wang, Y., et al. (2022). TripleBrain: A compact neuromorphic hardware core with fast on-chip self-organizing and reinforcement spike-timing dependent plasticity. *IEEE Transactions on Biomedical Circuits and Systems*, *16*(4), 636–650.

Wang, C., Luo, J., & Zhou, J. (2018). A 1-V to 0.29-V sub-100-pJ/operation ultra-low power fast-convergence CORDIC processor in 0.18-μm CMOS. *Microelectronics Journal*, *76*, 52–62.

Wang, S.-Q., Wang, L., Deng, Y., Yang, Z.-J., Guo, S.-S., Kang, Z.-Y., et al. (2020). SIES: A novel implementation of spiking convolutional neural network inference engine on field-programmable gate array. *Journal of Computer Science and Technology*, *35*, 475–489.

Wang, Z., Ye, L., Zhang, H., Ru, J., Fan, H., Wang, Y., et al. (2020). 20.2 A 57 nW software-defined always-on wake-up chip for IoT devices with asynchronous pipelined event-driven architecture and time-shielding level-crossing ADC. In *2020 IEEE international solid-state circuits conference-* (pp. 314–316). IEEE.

Webb, A., Davies, S., & Lester, D. (2011). Spiking neural PID controllers. In *Neural information processing: 18th international conference, ICONIP 2011, shanghai, China, November 13-17, 2011, proceedings, part iii 18* (pp. 259–267). Springer.

Wu, Y., Deng, L., Li, G., & Shi, L. (2018). Spatio-temporal backpropagation for training high-performance spiking neural networks. *Frontiers in Neuroscience*, *12*, Article 323875.

Wu, Y., Deng, L., Li, G., Zhu, J., & Shi, L. (2018). Spatio-temporal backpropagation for training high-performance spiking neural networks. *Frontiers in Neuroscience*, *12*, 331.

Wu, Y., Deng, L., Li, G., Zhu, J., Xie, Y., & Shi, L. (2019). Direct training for spiking neural networks: Faster, larger, better. *vol. 33*, In *Proceedings of the AAAI conference on artificial intelligence* (pp. 1311–1318).

Wu, J., Zhan, Y., Peng, Z., Ji, X., Yu, G., Zhao, R., et al. (2021). Efficient design of spiking neural network with STDP learning based on fast CORDIC. *IEEE Transactions on Circuits and Systems. I. Regular Papers*, *68*(6), 2522–2534.

Xia, Y., Levi, T., & Kohno, T. (2020). Digital hardware spiking neuronal network with STDP for real-time pattern recognition. *Journal of Robotics, Networking and Artificial Life*, *7*(2), 121–124.

Xiao, H., Rasul, K., & Vollgraf, R. (2017). Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms. arXiv preprint arXiv:1708.07747.

Xiaoxue, L., Xiaofan, Z., Xin, Y., Dan, L., He, W., Bowen, Z., et al. (2023). Review of medical data analysis based on spiking neural networks. *Procedia Computer Science*, *221*, 1527–1538.

Xing, Y., Zhang, L., Hou, Z., Li, X., Shi, Y., Yuan, Y., et al. (2022). Accurate ECG classification based on spiking neural network and attentional mechanism for real-time implementation on personal portable devices. *Electronics*, *11*(12), 1889.

Xu, R., Ma, S., Guo, Y., & Li, D. (2023). A survey of design and optimization for systolic array-based DNN accelerators. *ACM Computing Surveys*, *56*(1), 1–37.

Xu, Y., Tang, H., Xing, J., & Li, H. (2017). Spike trains encoding and threshold rescaling method for deep spiking neural networks. In *2017 IEEE symposium series on computational intelligence* (pp. 1–6). IEEE.

Yamazaki, K., Vo-Ho, V.-K., Bulsara, D., & Le, N. (2022). Spiking neural networks and their applications: A review. *Brain Sciences*, *12*(7), 863.

Yan, Y., Chu, H., Chen, X., Jin, Y., Huan, Y., Zheng, L., et al. (2021). Graph-based spatio-temporal backpropagation for training spiking neural networks. In *2021 IEEE 3rd international conference on artificial intelligence circuits and systems* (pp. 1–4). IEEE.

Yang, S., & Chen, B. (2024). *Neuromorphic intelligence: Learning, architectures and large-scale systems*. Springer Nature.

Yang, S., Deng, B., Wang, J., Li, H., Lu, M., Che, Y., et al. (2019). Scalable digital neuromorphic architecture for large-scale biophysically meaningful neural network with multi-compartment neurons. *IEEE Transactions on Neural Networks and Learning Systems*, *31*(1), 148–162.

Yang, Y., Eshraghian, J. K., Truong, N. D., Nikpour, A., & Kavehei, O. (2023). Neuromorphic deep spiking neural networks for seizure detection. *Neuromorphic Computing and Engineering*, *3*(1), Article 014010.

Yang, S., Gao, T., Wang, J., Deng, B., Lansdell, B., & Linares-Barranco, B. (2021). Efficient spike-driven learning with dendritic event-based processing. *Frontiers in Neuroscience*, *15*, Article 601109.

Yang, S., Tan, J., Lei, T., & Linares-Barranco, B. (2023). Smart traffic navigation system for fault-tolerant edge computing of internet of vehicle in intelligent transportation gateway. *IEEE Transactions on Intelligent Transportation Systems*, *24*(11), 13011–13022.

Yang, Y., Truong, N. D., Eshraghian, J. K., Nikpour, A., & Kavehei, O. (2022). Weak self-supervised learning for seizure forecasting: a feasibility study. *Royal Society Open Science*, *9*(8), Article 220374.

Yang, S., Wang, J., Deng, B., Azghadi, M. R., & Linares-Barranco, B. (2021). Neuromorphic context-dependent learning framework with fault-tolerant spike routing. *IEEE Transactions on Neural Networks and Learning Systems*, *33*(12), 7126–7140.

Yang, S., Wang, J., Deng, B., Liu, C., Li, H., Fietkiewicz, C., et al. (2019). Real-time neuromorphic system for large-scale conductance-based spiking neural networks. *IEEE Transactions on Cybernetics*, *49*(7), 2490–2503.

Yang, S., Wang, J., Hao, X., Li, H., Wei, X., Deng, B., et al. (2021). BiCoSS: toward large-scale cognition brain with multigranular neuromorphic architecture. *IEEE Transactions on Neural Networks and Learning Systems*, *33*(7), 2801–2815.

Yang, S., Wang, H., Pang, Y., Azghadi, M. R., & Linares-Barranco, B. (2024). NADOL: Neuromorphic architecture for spike-driven online learning by dendrites. *IEEE Transactions on Biomedical Circuits and Systems*, *18*(1), 186–199.

Yang, S., Wang, H., Pang, Y., Jin, Y., & Linares-Barranco, B. (2024). Integrating visual perception with decision making in neuromorphic fault-tolerant quadruplet-spike learning framework. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*.

Yang, J.-Q., Wang, R., Ren, Y., Mao, J.-Y., Wang, Z.-P., Zhou, Y., et al. (2020). Neuromorphic engineering: from biological to spike-based hardware nervous systems. *Advanced Materials*, *32*(52), Article 2003610.

Ye, W., Chen, Y., & Liu, Y. (2022). The implementation and optimization of neuromorphic hardware for supporting spiking neural networks with MLP and CNN topologies. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, *42*(2), 448–461.

Zenke, F., & Ganguli, S. (2018). Superspike: Supervised learning in multilayer spiking neural networks. *Neural Computation*, *30*(6), 1514–1541.

Zhang, Z., & Constandinou, T. G. (2021). Adaptive spike detection and hardware optimization towards autonomous, high-channel-count BMIs. *Journal of Neuroscience Methods*, *354*, Article 109103.

Zhang, G., Li, B., Wu, J., Wang, R., Lan, Y., Sun, L., et al. (2020). A low-cost and high-speed hardware implementation of spiking neural network. *Neurocomputing*, *382*, 106–115.

Zhang, J., Wang, R., Pei, X., Luo, D., Hussain, S., & Zhang, G. (2021). A fast spiking neural network accelerator based on BP-STDP algorithm and weighted neuron model. *IEEE Transactions on Circuits and Systems II: Express Briefs*, *69*(4), 2271–2275.

Zhou, Z.-H. (2018). A brief introduction to weakly supervised learning. *National Science Review*, *5*(1), 44–53.

Zhu, R.-J., Zhao, Q., Li, G., & Eshraghian, J. K. (2023). Spikegpt: Generative pre-trained language model with spiking neural networks. arXiv preprint arXiv:2302.13939.

Zompanti, A., Sabatini, A., Grasso, S., Pennazza, G., Ferri, G., Barile, G., et al. (2021). Development and test of a portable ECG device with dry capacitive electrodes and driven right leg circuit. *Sensors*, *21*(8), 2777.